# ML Unsupervised Techniques

Master Summary

Stefan Eder

Based on Hochreiter, *Machine Learning: Unsupervised Methods*, JKU Linz

This document is a summary of the course, written to be *understood* rather than merely consulted: the prose tries to explain why each method works before it states what it does, and the coloured boxes flag the definitions, key formulas, worked examples, mental models, and exam traps. Long proofs are condensed or omitted in favour of clarity. Use it as a study aid alongside the lecture notes and slides, not as a replacement for them.

If you spot an error or a gap, please open an issue or pull request at **github.com/stevetgit** — corrections are always welcome.

Good study materials should be shared, not gatekept. If this helped you, take it as a reason to help the next colleague you see struggling with something.

Compiled July 20, 2026

# Contents

# 1

# Introduction

## 1.1 Machine Learning Introduction

Machine learning is currently a major research topic in academia and industry alike. Applications span computer vision, speech recognition, recommender systems, and the analysis of large biological datasets. In the biomedical context, modern measurement technologies — mRNA concentrations measured via microarrays or next-generation sequencing, SNP arrays, copy number variation data — create a demand for methods that can extract structure from high-dimensional data without relying on labeled training examples.

The machine learning course series at JKU Linz comprises four complementary courses:

- **Basic Methods of Data Analysis:** summary statistics, PCA, ICA, multidimensional scaling, clustering, biclustering.
- **ML Supervised Methods:** classification and regression, SVMs, neural networks, deep learning, LSTM, ARMA/ARIMA.
- **ML Unsupervised Methods** (this course): MLE, MAP, maximum entropy, EM, PCA, ICA, factor analysis, mixture models, HMMs, Boltzmann machines.
- **Theoretical Concepts of Machine Learning:** estimation theory, VC-dimension, generalization bounds, convex optimization, Bayes theory.

The goal is not only to know individual methods, but to be able to *choose* the right approach for a given problem, understand the mathematical foundations well enough to adapt standard algorithms, and recognize when a model is likely to fail.

## 1.2 Course Specific Introduction

A learning problem is called *unsupervised* when the training data contain no target label or desired output. The algorithm must discover useful structure from the observations themselves. That makes evaluation less direct than in supervised learning: there is usually no single per-sample answer to compare against. Depending on the goal, we therefore inspect held-out likelihood, reconstruction error, cluster stability, neighbourhood preservation, or whether the representation helps a downstream task. Modern generative models such as VAEs and diffusion models belong to this broad family, although they use additional ideas beyond the classical methods in this script.

**Objectives.** Unsupervised methods are driven by different optimality criteria, depending on the problem:

- information content (maximizing information preserved after projection)
- orthogonality of extracted components
- statistical independence of components
- explained variance
- entropy of the representation
- likelihood: probability that the model produces the observed data
- distances within and between clusters

**Use cases.** Unsupervised methods are used to: explore data, find latent structure, visualize high-dimensional data in low dimensions, and compress representations. The unifying goal is to *understand and explore the data and generate new knowledge.*

**Key concepts in this course.** Maximum likelihood (ML), maximum a posteriori (MAP), maximum entropy, expectation maximization (EM), maximal variance, statistical independence, non-Gaussianity, sub- and super-Gaussian distributions, sparse codes and population codes.

**Model selection and overfitting.** The goal of learning is to select, from a model class, the model with the highest *generalization performance* — the model that best explains *future* data. These terms should not be conflated. *Training* estimates parameters for a fixed model and objective; *model selection* chooses hyperparameters or even a model family, ideally using held-out data; *learning* is the umbrella term for both. *Overfitting* occurs when the model is fitted to idiosyncratic training characteristics (noise, outliers, labeling errors) rather than to the true underlying structure; the model has high training performance but poor generalization.

**Intuition & Mental Model**

Underfitting (model too simple) and overfitting (model too complex) are the two failure modes. The goal is the best bias-variance trade-off: a model complex enough to capture the true structure but not so complex that it memorizes noise.

**Parametric vs. nonparametric models.** An important early decision is the *model class*:

- **Parametric models:** each parameter vector $\boldsymbol{w}$ represents one model. Examples: neural networks (synaptic weights), SVMs (weight vector $\boldsymbol{w}$), factor analysis. Learning corresponds to a path through parameter space. Disadvantages: different parameterizations can represent the same function; model complexity is implicitly controlled through the parameters.
- **Nonparametric models:** their effective number of parameters can grow with the amount of data. Examples are $k$-nearest neighbours, kernel density estimation, and many decision-tree methods. "Nonparametric" does *not* mean "has no parameters": choices such as $k$, bandwidth, or tree depth still matter and are commonly selected from data by validation.

**Unsupervised methods covered in this course.** Principal component analysis (PCA), independent component analysis (ICA), factor analysis, projection pursuit, $k$-means clustering, hierarchical clustering, mixture models (Gaussian, Poisson), self-organizing maps (SOMs), kernel density estimation, hidden Markov models (HMMs, factorial HMMs, IO-HMMs), Markov networks, restricted Boltzmann machines, auto-associators.

Methods fall into two broad families:

- **Projection methods:** produce a new representation of objects by down-projecting feature vectors into a lower-dimensional space while preserving neighborhoods or other desired prop-

erties. Examples: PCA (orthogonal components of maximal variance) and ICA (statistically independent components). Factor analysis is related but is better viewed as a probabilistic latent-variable model with an explicit noise model.

- **Generative models:** build an explicit probabilistic model of the observed data; the fitted model should match the observed data density.

## 1.3 Generative vs. Descriptive Models

**Descriptive (Recoding) Model**

A **descriptive model** doesn't try to explain where the data *came from* — it just re-describes it in a more useful form. It maps each observation into a new space with some desired property baked in: uncorrelated axes (PCA) or statistically independent ones (ICA).

**Generative Model**

A **generative model** goes the other way: it models *how the data was made*. The story is that hidden source components $\boldsymbol{u}$ are drawn from a prior $p_{\boldsymbol{u}}$, pushed through a model $g(\boldsymbol{u}; \boldsymbol{w})$, and out comes the data. Training then tunes $\boldsymbol{w}$ so that the model's distribution $p(\boldsymbol{x}; \boldsymbol{w})$ matches what we actually observe, $p(\boldsymbol{x})$. Examples: factor analysis, latent variable models, Boltzmann machines, HMMs.

Generative models allow the use of prior knowledge about the world and enable the prediction of new states of the data-generating process (e.g. predicting gene expression or brain activity). The natural training objective is maximum likelihood or MAP.

Descriptive models aim to find a richer or more interpretable representation rather than to model the generative process. Their objectives are properties of the representation: maximal variance (PCA), statistical independence (ICA), maximal non-Gaussianity (projection pursuit).

**Exam trap**

Parametric/nonparametric is *not* the same as generative/descriptive. Factor analysis is parametric *and* generative. $k$-NN is nonparametric *and* descriptive (no explicit density).

# 2

# Basic Terms and Concepts

This chapter is the toolbox chapter. Nothing here is a method you would apply to a dataset — instead it is the machinery that every later chapter quietly assumes: how we judge whether an estimate is any good, how we turn "fit the model to the data" into a concrete objective (maximum likelihood), what to do when the model has variables we never get to see (EM), how to measure information and distance between distributions, and how to pick a distribution when we know almost nothing (maximum entropy).

If you find yourself lost in a later chapter, the confusion usually traces back to something in here. EM in particular shows up again as Baum–Welch for HMMs, as the fitting procedure for Gaussian mixtures, for factor analysis, and for FABIA — learn it once, properly, and four later chapters become re-runs of the same idea.

## 2.1 Unsupervised Learning in Bioinformatics

The primary driving application for many unsupervised methods is gene expression data. A microarray experiment produces a matrix of expression values: rows are genes (features), columns are samples (conditions, patients, time points). The data are high-dimensional (thousands of genes), noisy, and unlabeled.

Typical tasks include:

- finding groups of co-expressed genes (clustering, biclustering)
- identifying tissue types or disease subtypes (mixture models, ICA)
- visualizing the data in 2–3 dimensions (PCA, $t$-SNE, Isomap)
- detecting regulatory modules that activate in a subset of samples (FABIA)

A classical result is Spellman's cell-cycle data: PCA applied to yeast expression data over the cell cycle reveals the dominant temporal oscillation in the first two principal components, confirming the cyclic structure. Hierarchical clustering of microarray data yields dendrograms that group both genes and samples; the correlation coefficient bar on the dendrogram indicates relatedness of branches.

## 2.2 Unsupervised Learning Categories

For this course it is useful to organise most methods into two frameworks. This is a mental map, not an exhaustive taxonomy: methods such as clustering can be descriptive ($k$-means) or generative (mixture models), and some methods combine both views.

**Two Frameworks of Unsupervised Learning**

- **Generative framework:** density estimation, hidden Markov models, mixture models. Objective: *maximum likelihood* or *maximum a posteriori*.
- **Recoding (descriptive) framework:** projection methods, PCA, ICA. Objective: *maximal variance*, orthogonality, independence, *maximum entropy*.

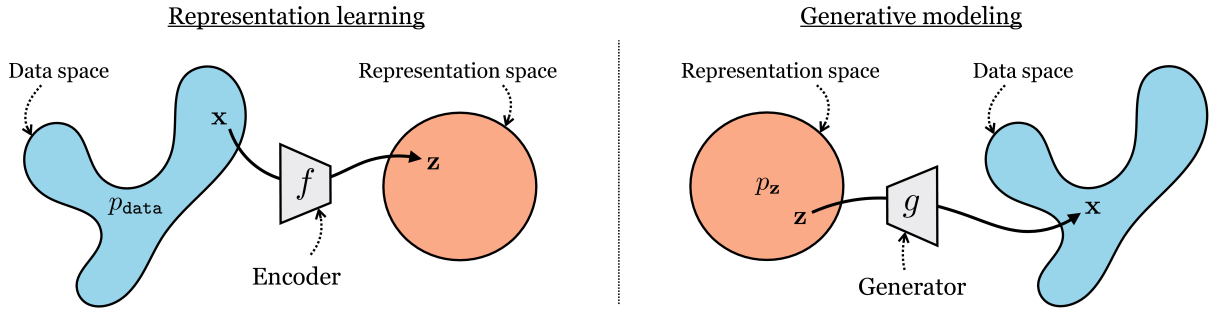


Figure 2.1: The two complementary directions: representation learning maps observations into a useful latent code, whereas generative modelling maps latent variables into the data space.

### 2.2.1 Generative Framework

In the generative framework, the world is modeled as a stochastic process: latent (hidden) sources $\boldsymbol{u}$ are drawn from a source distribution $p_{\boldsymbol{u}}$, passed through a world model $g(\boldsymbol{u};\boldsymbol{w})$, and produce observations $\boldsymbol{x}$. The learned model distribution $p(\boldsymbol{x};\boldsymbol{w})$ must match the observed distribution $p(\boldsymbol{x})$.

**Generative Model Objective**

Find parameters $\boldsymbol{w}$ such that the model distribution

$$p(\boldsymbol{x};\boldsymbol{w}) = \int_U p_{\boldsymbol{u}}(\boldsymbol{u})\,\delta(\boldsymbol{x} - g(\boldsymbol{u};\boldsymbol{w}))\,\mathrm{d}\boldsymbol{u}$$

is as close as possible to the empirical distribution $p(\boldsymbol{x})$. The loss function measures the distance between these two distributions; typically this is minimized via *maximum likelihood*.

Key examples of generative models: factor analysis, latent variable models, Boltzmann machines, hidden Markov models.

### 2.2.2 Recoding Framework

In the recoding (descriptive) framework, a model $g(\boldsymbol{x};\boldsymbol{w})$ maps observations $\boldsymbol{x} \sim p(\boldsymbol{x})$ to projections $\boldsymbol{u}$. The resulting distribution $p(\boldsymbol{u};\boldsymbol{w})$ should match a desired *target* distribution $p_t(\boldsymbol{u})$:

- **PCA:** projects to a low-dimensional space with maximal retained variance. This can be interpreted as maximal information preservation only under additional assumptions, for example Gaussian data or isotropic Gaussian reconstruction noise. The component directions are orthogonal and their scores are uncorrelated.

- **ICA:** projects into a space with statistically *independent* components (factorial code). Algorithms approximate independence by minimising mutual information, maximising non-Gaussianity, or maximising likelihood under a chosen source distribution. Merely maximising the entropy of a fixed-variance linear projection would instead favour a Gaussian and is not the ICA objective.
- **Projection Pursuit:** projects such that components are maximally non-Gaussian (since Gaussian components arise from random projections of any distribution, by the CLT; non-Gaussian components are informative).

### 2.2.3 Recoding and Generative Framework Unified

Both frameworks can be seen as two sides of the same coin. In the generative direction: sources $\boldsymbol{u} \xrightarrow{g(\boldsymbol{u};\boldsymbol{w})} \boldsymbol{x}$ (sources produce observations). In the recoding direction: observations $\boldsymbol{x} \xrightarrow{g^{-1}(\boldsymbol{x};\boldsymbol{w})} \boldsymbol{u}$ (observations are encoded into sources). When $g$ is invertible, the change-of-variables formula lets us express the same model in either direction. That does *not* make arbitrary encoder and decoder objectives equivalent: the source density, Jacobian term, and loss must match. Under the standard invertible ICA assumptions, however, INFOMAX and maximum likelihood lead to equivalent learning rules.

## 2.3 Quality of Parameter Estimation

Generative models must estimate true (but unknown) model parameters. Let the training data be $\{\boldsymbol{x}\} = \{\boldsymbol{x}^1, \ldots, \boldsymbol{x}^l\}$, collected into the matrix $\boldsymbol{X} = (\boldsymbol{x}^1, \ldots, \boldsymbol{x}^l)^\top \in \mathbb{R}^{l \times d}$. The true (unknown) parameter vector is $\boldsymbol{w}$; an estimator maps the data to $\hat{\boldsymbol{w}}$.

**Estimator Quality Measures**

$$\begin{aligned}
\text{Unbiased:} \quad & \mathbb{E}_{\boldsymbol{X}}[\hat{\boldsymbol{w}}] = \boldsymbol{w} \\
\text{Bias:} \quad & b(\hat{\boldsymbol{w}}) = \mathbb{E}_{\boldsymbol{X}}[\hat{\boldsymbol{w}}] - \boldsymbol{w} \\
\text{Variance:} \quad & \mathrm{Var}(\hat{\boldsymbol{w}}) = \mathbb{E}_{\boldsymbol{X}}\Big[(\hat{\boldsymbol{w}} - \mathbb{E}_{\boldsymbol{X}}[\hat{\boldsymbol{w}}])^\top (\hat{\boldsymbol{w}} - \mathbb{E}_{\boldsymbol{X}}[\hat{\boldsymbol{w}}])\Big] \\
\text{MSE:} \quad & \mathrm{mse}(\hat{\boldsymbol{w}}) = \mathbb{E}_{\boldsymbol{X}}\Big[(\hat{\boldsymbol{w}} - \boldsymbol{w})^\top (\hat{\boldsymbol{w}} - \boldsymbol{w})\Big]
\end{aligned}$$

The fundamental bias-variance decomposition of the MSE is:

**Bias-Variance Decomposition**

$$\mathrm{mse}(\hat{\boldsymbol{w}}) = \mathrm{Var}(\hat{\boldsymbol{w}}) + b^2(\hat{\boldsymbol{w}})$$

*Derivation sketch:* Write $\hat{\boldsymbol{w}} - \boldsymbol{w} = (\hat{\boldsymbol{w}} - \mathbb{E}[\hat{\boldsymbol{w}}]) + (\mathbb{E}[\hat{\boldsymbol{w}}] - \boldsymbol{w})$, expand the squared norm, and note that the cross-term vanishes because $\mathbb{E}_{\boldsymbol{X}}[(\hat{\boldsymbol{w}} - \mathbb{E}[\hat{\boldsymbol{w}}])(\mathbb{E}[\hat{\boldsymbol{w}}] - \boldsymbol{w})^\top] = 0$ (since $\mathbb{E}[\hat{\boldsymbol{w}}] - \boldsymbol{w}$ is a constant w.r.t. the expectation).

The objective is to minimize the MSE. Note: the MSE in the parameter estimation sense (expected squared distance to true parameter) is different from the supervised regression loss (expected squared distance to true output).

**Intuition & Mental Model**

Unbiasedness and consistency are different properties. An *unbiased* estimator has $\mathbb{E}[\hat{\boldsymbol{w}}] = \boldsymbol{w}$ for *any* dataset size. A *consistent* estimator satisfies $\hat{\boldsymbol{w}} \to \boldsymbol{w}$ as $l \to \infty$ — its variance shrinks with more data, but it may be biased for finite $l$.

The two properties are genuinely independent, and the classic exam move is to hand you an estimator that has one but not the other. Take $\frac{1}{l+1}\sum_{i=1}^{l} X_i$ as an estimator of the mean: its expectation is $\frac{l}{l+1}\mu \neq \mu$, so it is *biased* — but the bias and the variance both die off as $l \to \infty$, so it is *consistent*. Now take $X_1$, the first sample, as an estimator of the mean: $\mathbb{E}[X_1] = \mu$ makes it *unbiased*, but it never gets better no matter how much data arrives, so it is *not consistent*. Bias is about being centred on the truth; consistency is about eventually collapsing onto it.

### 2.3.1 How good can an estimator possibly get?

Suppose we restrict ourselves to unbiased estimators. Is there a floor on how small the variance can be, or can a clever enough estimator squeeze it arbitrarily close to zero? There is a floor, and it is set by how sharply the likelihood reacts to the parameter. If tweaking $\boldsymbol{w}$ barely changes the probability of the data, the data barely constrains $\boldsymbol{w}$ and any estimator will be shaky; if a small tweak in $\boldsymbol{w}$ sends the likelihood plummeting, the data pins the parameter down tightly. The quantity measuring this sensitivity is the **Fisher information**.

**Fisher Information (scalar parameter)**

For a scalar parameter $w$, the Fisher information carried by one observation is the variance of the score (the derivative of the log-likelihood):

$$\mathcal{I}(w) = \mathbb{E}\left[\left(\frac{\partial}{\partial w}\ln p(\boldsymbol{x}; w)\right)^2\right] = -\,\mathbb{E}\left[\frac{\partial^2}{\partial w^2}\ln p(\boldsymbol{x}; w)\right].$$

It measures the *curvature* of the log-likelihood at its peak: a sharp, narrow peak means high information. For $l$ iid samples the information simply adds up: $\mathcal{I}_l(w) = l\,\mathcal{I}(w)$. For a parameter vector the corresponding object is the Fisher information *matrix*, the expected outer product of score vectors.

**Cramér–Rao Lower Bound**

Under the usual differentiability and support regularity conditions, any unbiased estimator $\hat{w}$ of a scalar parameter satisfies

$$\mathrm{Var}(\hat{w}) \;\geq\; \frac{1}{\mathcal{I}_l(w)}.$$

The variance of the estimator is bounded below by the *inverse* of the Fisher information. An estimator that attains this bound — the smallest variance physically available — is called **efficient**.

**Exam trap: which quantity does Cramér–Rao bound?**

The Cramér–Rao bound relates the **variance** of $\hat{\boldsymbol{w}}$ to the inverse Fisher information — not the bias, not the expected value, not the MSE. And an estimator reaching the bound is **efficient**, not "consistent" or "unbiased" (unbiasedness is a *precondition* for the bound, not its conclusion). These three words — consistent, efficient, unbiased — get shuffled

around in exam questions constantly; keep them straight: *unbiased* = centred correctly, *consistent* = converges with more data, *efficient* = has the smallest variance the bound allows.

## 2.4 Information Theory Essentials

Information theory connects many objectives in this course, but not always in the same way. ICA can minimise mutual information and $t$-SNE explicitly minimises a KL divergence. Maximum likelihood — used by EM and Boltzmann machines — is equivalent to minimising the forward KL divergence from the empirical data distribution to the model, up to a constant. NMF only has a KL interpretation for its KL-divergence loss; the common squared-error version does not. PCA's retained variance has an information interpretation only under suitable probabilistic assumptions. So it pays to get the vocabulary straight once, here, rather than re-deriving it in every chapter.

**Entropy, Joint & Conditional Entropy, Mutual Information**

For a discrete random variable $X$ with distribution $p(x)$:

$$H(X) = -\sum_{x} p(x) \log p(x), \qquad \text{with the convention } 0 \cdot \log 0 = 0.$$

For a pair $(X, Y)$ with joint distribution $p(x, y)$:

$$H(X, Y) = -\sum_{x,y} p(x, y) \log p(x, y), \qquad H(X \mid Y) = H(X, Y) - H(Y).$$

The **mutual information** is the information the two variables share:

$$\mathrm{I}(X; Y) = \sum_{x,y} p(x, y) \, \log \frac{p(x, y)}{p(x)\, p(y)} = H(X) + H(Y) - H(X, Y).$$

$\mathrm{I}(X; Y) \geq 0$, with equality *iff* $X$ and $Y$ are independent.

Entropy is "how surprised am I, on average, by a draw from this distribution". It is zero for a deterministic variable (no surprise ever) and maximal for the uniform distribution (every outcome equally unexpected). Mutual information is then "how much does knowing $Y$ reduce my surprise about $X$" — which is why it is exactly zero when the two are independent, and why ICA hunts for the projection that drives it to zero.

**Entropy is not bounded by 1**

A perennial exam distractor claims "the entropy of a probability distribution is always $\geq 0$ and $\leq 1$". The lower bound is right, the upper bound is nonsense: a uniform distribution over $k$ outcomes has entropy $\log k$, which grows without limit. (With $\log_2$ you get 1 bit for a fair coin — probably the source of the confusion — but even then three equally likely outcomes already give $\log_2 3 \approx 1.58$.) Watch the base of the logarithm too: exams here usually ask for the *natural* logarithm, so a fair coin gives $\ln 2 \approx 0.69$, not 1.

**Computing marginals, entropies, and mutual information**

Given the joint distribution of two binary variables:

| | $Y=0$ | $Y=1$ |
|---|---|---|
| $X=0$ | 0.3 | 0.4 |
| $X=1$ | 0.2 | 0.1 |

**Marginals** are row and column sums: $p(X{=}0) = 0.3{+}0.4 = 0.7$ and $p(Y{=}0) = 0.3{+}0.2 = 0.5$.

**Entropies** (natural log) come from the marginals only:

$$H(X) = -(0.7\ln 0.7{+}0.3\ln 0.3) \approx 0.61, \qquad H(Y) = -(0.5\ln 0.5{+}0.5\ln 0.5) = \ln 2 \approx 0.69.$$

**Joint entropy** uses all four cells:

$$H(X,Y) = -(0.3\ln 0.3 + 0.4\ln 0.4 + 0.2\ln 0.2 + 0.1\ln 0.1) \approx 1.28.$$

**Mutual information** is then the leftover:

$$\mathrm{I}(X;Y) = H(X) + H(Y) - H(X,Y) \approx 0.61 + 0.69 - 1.28 = 0.02.$$

The tiny value says these two variables are *almost* independent — if they were exactly independent, every cell would equal the product of its margins ($0.7 \times 0.5 = 0.35 \neq 0.3$), and I would be exactly 0.

Finally, the workhorse of the whole course: a way to measure how far one distribution is from another.

**Kullback–Leibler Divergence**

For distributions $p$ and $q$ over the same space:

$$D_{\mathrm{KL}}(p\,\|\,q) = \sum_x p(x)\,\log\frac{p(x)}{q(x)} \qquad \text{(discrete)}, \qquad D_{\mathrm{KL}}(p\,\|\,q) = \int p(x)\,\log\frac{p(x)}{q(x)}\,\mathrm{d}x \qquad \text{(continuous)}.$$

Properties:

- $D_{\mathrm{KL}}(p\,\|\,q) \geq 0$, with equality *iff* $p = q$ (Gibbs' inequality).
- It is **not symmetric**: $D_{\mathrm{KL}}(p\,\|\,q) \neq D_{\mathrm{KL}}(q\,\|\,p)$ in general — it is a divergence, not a distance (and it violates the triangle inequality).
- It is infinite if $q(x) = 0$ somewhere that $p(x) > 0$: $p$'s support must sit inside $q$'s support.
- Mutual information is a special case: $\mathrm{I}(X;Y) = D_{\mathrm{KL}}(p(x,y)\,\|\,p(x)p(y))$ — the divergence between the true joint and the joint you would get if the variables were independent.

**Two KL computations you should be able to do cold**

**Discrete.** With $p = (0.8,\, 0.2)$ and $q = (0.5,\, 0.5)$:

$$D_{\mathrm{KL}}(p\,\|\,q) = 0.8\ln\frac{0.8}{0.5} + 0.2\ln\frac{0.2}{0.5} = 0.8(0.470) + 0.2(-0.916) \approx 0.19.$$

**Continuous (two uniforms).** Let $p_Y$ be uniform on $[0, \theta_Y]$ and $p_Z$ uniform on $[0, \theta_Z]$, with $\theta_Y \leq \theta_Z$. The integrand is nonzero only where $p_Y > 0$, and there the density ratio is the constant $\theta_Z/\theta_Y$, so the integral collapses:

$$D_{\mathrm{KL}}(p_Y \,\|\, p_Z) = \int_0^{\theta_Y} \frac{1}{\theta_Y} \ln \frac{1/\theta_Y}{1/\theta_Z} \,\mathrm{d}x = \ln \frac{\theta_Z}{\theta_Y}.$$

With $\theta_Y = e^5$ and $\theta_Z = e^{10}$ this is just $10-5 = 5$. Note the asymmetry biting: $D_{\mathrm{KL}}(p_Z \,\|\, p_Y)$ would be *infinite*, because $p_Z$ puts mass where $p_Y$ is zero.

## 2.5 Maximum Likelihood Estimator

**Likelihood**

The **likelihood** of the dataset $\{\boldsymbol{x}\} = \{\boldsymbol{x}^1, \ldots, \boldsymbol{x}^l\}$ under model parameters $\boldsymbol{w}$ is

$$\mathcal{L}(\{\boldsymbol{x}\}; \boldsymbol{w}) \;=\; p(\{\boldsymbol{x}\}; \boldsymbol{w}),$$

the probability that the model $p(\boldsymbol{x}; \boldsymbol{w})$ produces the observed data. For **iid** (independent, identically distributed) data this factorizes:

$$\mathcal{L}(\{\boldsymbol{x}\}; \boldsymbol{w}) \;=\; \prod_{i=1}^{l} p(\boldsymbol{x}^i; \boldsymbol{w}).$$

Working in log-space converts the product to a sum (negative log-likelihood, NLL):

$$-\ln \mathcal{L}(\{\boldsymbol{x}\}; \boldsymbol{w}) \;=\; -\sum_{i=1}^{l} \ln p(\boldsymbol{x}^i; \boldsymbol{w}).$$

Maximum likelihood estimation (MLE) is the dominant objective in unsupervised learning. It finds the parameters $\boldsymbol{w}$ that make the observed data most probable under the model:

$$\hat{\boldsymbol{w}}_{\mathrm{ML}} = \arg\max_{\boldsymbol{w}} \mathcal{L}(\{\boldsymbol{x}\}; \boldsymbol{w}) = \arg\min_{\boldsymbol{w}} \big[-\ln \mathcal{L}(\{\boldsymbol{x}\}; \boldsymbol{w})\big].$$

**Properties of the MLE**

Under standard regularity conditions and when the model is identifiable and correctly specified, the maximum likelihood estimator is:

- **Invariant under reparameterization:** if $\hat{\boldsymbol{w}}$ is the MLE of $\boldsymbol{w}$, then $f(\hat{\boldsymbol{w}})$ is an MLE of $f(\boldsymbol{w})$, with the usual care needed for non-injective transformations.
- **Asymptotically unbiased and efficient:** for large $l$, the MLE achieves the Cramér-Rao lower bound, i.e. it has the smallest possible variance among all unbiased estimators. It is asymptotically optimal.
- **Asymptotically consistent:** $\hat{\boldsymbol{w}} \to \boldsymbol{w}$ as $l \to \infty$.

**MLE caveats**

MLE can overfit for small datasets (it uses no prior information). For finite $l$, MLE may be biased: the classic example is the MLE of the variance of a Gaussian, $\hat{\sigma}^2 = \frac{1}{l}\sum(x_i - \bar{x})^2$, which underestimates the true variance (the unbiased estimator uses $l-1$).

## 2.6 Expectation Maximization

Many generative models include *latent (hidden) variables* $\boldsymbol{u}$ that are not directly observed. The marginal likelihood then requires integrating over all latent configurations:

$$p(\boldsymbol{x};\boldsymbol{w}) = \int_U p_{\boldsymbol{u}}(\boldsymbol{u})\,\delta(\boldsymbol{x} - g(\boldsymbol{u};\boldsymbol{w}))\,\mathrm{d}\boldsymbol{u}.$$

This integral is in general intractable due to:

- hidden states (discrete or continuous latent variables),
- many-to-one output mappings ($g$ is not injective),
- non-linearities in $g$.

The **Expectation Maximization (EM)** algorithm circumvents this by working with the *joint* distribution $p(\boldsymbol{x},\boldsymbol{u};\boldsymbol{w})$, which is easier to handle, and by constructing a tractable lower bound on the log-likelihood.

**Derivation of the lower bound.** Introduce an arbitrary distribution $Q(\boldsymbol{u}\,|\,\{\boldsymbol{x}\})$ over latent variables. Apply Jensen's inequality to the concave logarithm:

$$\begin{aligned}
\ln \mathcal{L}(\{\boldsymbol{x}\};\boldsymbol{w}) &= \ln \int_U p(\{\boldsymbol{x}\},\boldsymbol{u};\boldsymbol{w})\;\mathrm{d}\boldsymbol{u} \\
&= \ln \int_U Q(\boldsymbol{u}\,|\,\{\boldsymbol{x}\})\,\frac{p(\{\boldsymbol{x}\},\boldsymbol{u};\boldsymbol{w})}{Q(\boldsymbol{u}\,|\,\{\boldsymbol{x}\})}\;\mathrm{d}\boldsymbol{u} \\
&\geq \int_U Q(\boldsymbol{u}\,|\,\{\boldsymbol{x}\})\,\ln\frac{p(\{\boldsymbol{x}\},\boldsymbol{u};\boldsymbol{w})}{Q(\boldsymbol{u}\,|\,\{\boldsymbol{x}\})}\;\mathrm{d}\boldsymbol{u} \;\coloneqq\; \mathcal{F}(Q,\boldsymbol{w}).
\end{aligned}$$

$\mathcal{F}(Q,\boldsymbol{w})$ is the **free energy** (ELBO: evidence lower bound). Equivalently, the bound can be rewritten as

$$\mathcal{F}(Q,\boldsymbol{w}) = \ln \mathcal{L}(\{\boldsymbol{x}\};\boldsymbol{w}) - D_{\mathrm{KL}}(Q(\boldsymbol{u}\,|\,\{\boldsymbol{x}\})\,\|\,p(\boldsymbol{u}\,|\,\{\boldsymbol{x}\};\boldsymbol{w}))\,,$$

which makes the E-step obvious: to maximize $\mathcal{F}$ over $Q$ (with $\boldsymbol{w}$ fixed) one minimizes the KL divergence, which is zero iff $Q = p(\boldsymbol{u}\,|\,\{\boldsymbol{x}\};\boldsymbol{w})$.

**EM Algorithm**

EM alternates between two steps until convergence:

$$\textbf{E-step:}\quad Q_{k+1} = \arg\max_{Q}\ \mathcal{F}(Q,\boldsymbol{w}_k) \quad\Rightarrow\quad Q_{k+1} = p(\boldsymbol{u}\,|\,\{\boldsymbol{x}\};\boldsymbol{w}_k)$$

$$\textbf{M-step:}\quad \boldsymbol{w}_{k+1} = \arg\max_{\boldsymbol{w}}\ \mathcal{F}(Q_{k+1},\boldsymbol{w})$$

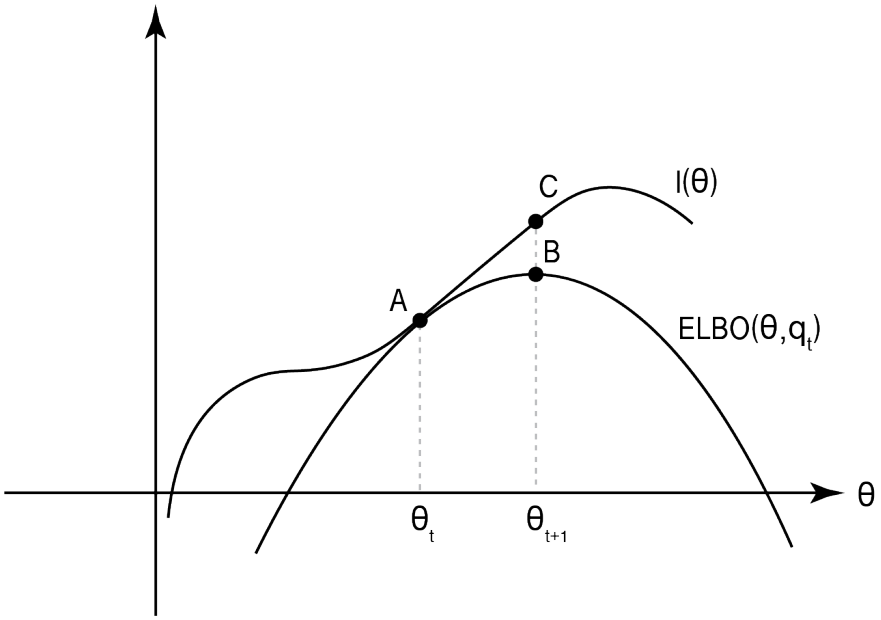


Figure 2.2: EM viewed as coordinate ascent. The E-step makes the ELBO touch the log-likelihood at the current parameters; the M-step moves to a parameter value with a higher bound and therefore a non-decreased likelihood.

**Monotone convergence.** Both steps can only increase (or leave unchanged) the lower bound $\mathcal{F}$. The E-step tightens the bound: at the beginning of the M-step, $\mathcal{F}(Q_{k+1}, \boldsymbol{w}_k) = \ln \mathcal{L}(\{\boldsymbol{x}\}; \boldsymbol{w}_k)$ (the bound is tight). The M-step then increases $\mathcal{F}$, which implies:

$$\ln \mathcal{L}(\{\boldsymbol{x}\}; \boldsymbol{w}_k) \;\leq\; \mathcal{F}(Q_{k+1}, \boldsymbol{w}_{k+1}) \;\leq\; \ln \mathcal{L}(\{\boldsymbol{x}\}; \boldsymbol{w}_{k+1}).$$

Therefore the log-likelihood is guaranteed to be non-decreasing at every EM iteration.

**Intuition & Mental Model**

The E-step does not change model parameters — it only updates the posterior distribution over latent variables. The M-step maximizes the expected complete-data log-likelihood, which for exponential family models has a closed-form solution. EM converges to a stationary point (often a local maximum, but possibly a saddle point or boundary solution), not necessarily to the global optimum.

EM is applied throughout this course: Baum-Welch algorithm for HMMs, fitting Gaussian mixtures, factor analysis, and (soft) ICA variants.

## 2.7 Maximum Entropy

Suppose you know a few facts about a distribution — its mean, maybe its variance — but nothing else, and you have to commit to a full distribution anyway. Which one do you pick? Picking any particular shape smuggles in assumptions you cannot justify. The maximum entropy principle says: pick the *least committal* distribution that still respects the facts you actually have. Since entropy measures uncertainty (Section 2.4), the most uncertain distribution consistent with your constraints is the one that assumes the least beyond them.

**Principle of Maximum Entropy**

Among all distributions consistent with the available constraints (e.g. known moments), choose the one with *maximum entropy* $H = -\sum_{k\geq 1} p_k \log p_k$.

This distribution:

- makes the minimal assumptions beyond the stated constraints,
- corresponds to the most likely empirical distribution that could give rise to the observed features,
- connects statistical mechanics (thermodynamic equilibrium) and information theory.

**Existence caveat**

Not every class of distributions contains a maximum entropy member. For example, the class of all distributions with a given mean has arbitrarily large entropy (entropy can be pushed to $\infty$). The class of distributions with mean zero, second moment one, and third moment one has entropy bounded from above but the supremum is not attained.

**Maximum entropy with moment constraints.** Given moment constraints $\sum_{i=1}^{n} p(x_i)\, f_k(x_i) = F_k$ for $k = 1, \ldots, m$ and normalization $\sum_{i=1}^{n} p(x_i) = 1$, the maximum entropy solution is the **Gibbs distribution**:

**Gibbs Distribution (MaxEnt Solution)**

$$p(x_i) = \frac{1}{Z(\lambda_1, \ldots, \lambda_m)} \exp\big(\lambda_1 f_1(x_i) + \cdots + \lambda_m f_m(x_i)\big),$$

with **partition function**

$$Z(\lambda_1, \ldots, \lambda_m) = \sum_{i=1}^{n} \exp\big(\lambda_1 f_1(x_i) + \cdots + \lambda_m f_m(x_i)\big).$$

The Lagrange multipliers $\lambda_k$ are determined by the constraint equations:

$$F_k = \frac{\partial}{\partial \lambda_k} \log Z(\lambda_1, \ldots, \lambda_m), \qquad k = 1, \ldots, m.$$

**Standard MaxEnt examples**

- **Normal distribution:** maximum entropy distribution given fixed mean and variance. Constraints: $\mathbb{E}[X] = \mu$ and $\mathbb{E}[X^2] = \mu^2 + \sigma^2$.
- **Uniform distribution:** maximum entropy on a bounded interval $[a, b]$. Only constraint: support.
- **Exponential distribution:** maximum entropy on $[0, \infty)$ given fixed mean. Constraint: $\mathbb{E}[X] = 1/\lambda$.

**Intuition & Mental Model**

The MaxEnt principle explains one precise reason Gaussian models appear so frequently: among all continuous distributions with a *fixed mean and variance*, the Gaussian has maximum differential entropy. Finite variance by itself does not make a process Gaussian,

and an energy constraint does not guarantee convergence to a Gaussian. In ICA, maximizing entropy of a nonlinear transform of the data is equivalent to maximizing likelihood of an independent-component model (INFOMAX).

# 3

# Principal Component Analysis

## 3.1 The Method

PCA answers a simple question: if the data really lives in fewer dimensions than we measured, which few directions capture most of what is going on? It rotates the data onto a new set of orthogonal axes — the *principal directions* — ordered so that the first captures the most variance, the second the most of what is left, and so on. Keep the top few and you have a faithful low-dimensional picture of the data; the rest is mostly noise or redundancy you can throw away. Everything below is just the linear algebra that makes this precise.

Let the data matrix be $\boldsymbol{X} \in \mathbb{R}^{n \times m}$, where $n$ is the number of samples and $m$ is the number of features (the rows $\boldsymbol{x}_i \in \mathbb{R}^m$ are centered so that $\sum_i \boldsymbol{x}_i = \boldsymbol{0}$).

The **sample covariance matrix** is

$$\boldsymbol{C} = \frac{1}{n} \boldsymbol{X}^\top \boldsymbol{X} \in \mathbb{R}^{m \times m}.$$

Its eigendecomposition $\boldsymbol{C} = \boldsymbol{U} \boldsymbol{\Lambda} \boldsymbol{U}^\top$ yields an orthogonal matrix $\boldsymbol{U}$ whose columns $\boldsymbol{u}_k$ are the principal directions (*loadings*), and a diagonal matrix $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_m)$ of eigenvalues sorted in decreasing order.

The **SVD** of $\boldsymbol{X}$ gives the same result: $\boldsymbol{X} = \boldsymbol{V} \boldsymbol{D} \boldsymbol{U}^\top$ with $\boldsymbol{V} \in \mathbb{R}^{n \times n}$, $\boldsymbol{D} \in \mathbb{R}^{n \times m}$ diagonal, and $\boldsymbol{U} \in \mathbb{R}^{m \times m}$ orthogonal. The eigenvalues of $\boldsymbol{C}$ are $\lambda_k = d_k^2 / n$ where $d_k$ are the singular values.

**PCA Projection**

The **principal component scores** (projections) are collected in

$$\boldsymbol{Y} = \boldsymbol{X} \boldsymbol{U} = \boldsymbol{V} \boldsymbol{D} \in \mathbb{R}^{n \times m}.$$

The $k$-th column $\boldsymbol{y}_k = \boldsymbol{X} \boldsymbol{u}_k$ is the projection of all data points onto the $k$-th principal direction. The original data can be reconstructed as the outer product sum

$$\boldsymbol{X} = \sum_{k=1}^{m} \boldsymbol{y}_k \boldsymbol{u}_k^\top.$$

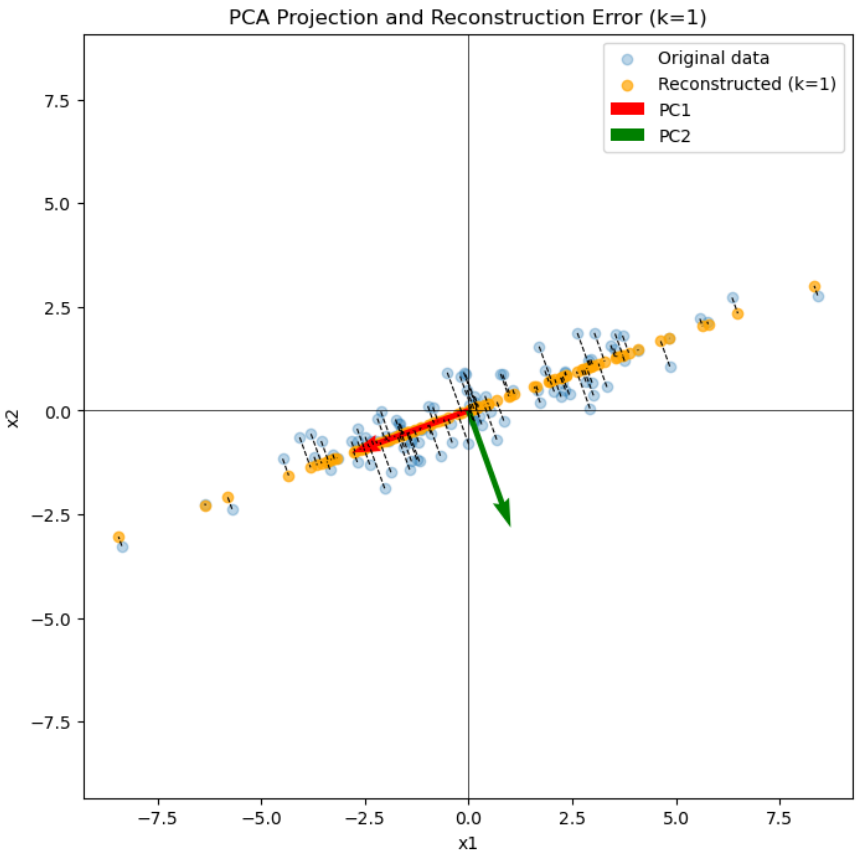


Figure 3.1: Rank-one PCA reconstruction. The blue points are the original data, the orange points their projections onto PC1, and the short connecting segments are the discarded PC2 residuals. The red and green arrows indicate the first and second principal directions.

Figure 3.1 makes the low-rank approximation geometric: retaining only PC1 replaces each sample by its orthogonal projection onto the red axis. The squared lengths of the connecting segments add up to the rank-one reconstruction error.

**Online learning: Oja's rule.** A single-component PCA can be learned online (one sample at a time) using **Oja's rule**:

$$\boldsymbol{u}^{\text{new}} = \boldsymbol{u} + \eta(t\,\boldsymbol{x} - t^2\,\boldsymbol{u}), \qquad t = \boldsymbol{u}^\top \boldsymbol{x}.$$

This is a fixed-point iteration analogous to the power method.

## 3.2 Variance Maximization

PCA can be motivated as the projection that maximizes the variance of the projected data.

The first principal direction solves

$$\boldsymbol{u}_1 = \arg\max_{\|\boldsymbol{u}\|=1} \frac{1}{n}\sum_{i=1}^{n} (\boldsymbol{u}^\top \boldsymbol{x}_i)^2 = \arg\max_{\|\boldsymbol{u}\|=1} \boldsymbol{u}^\top \boldsymbol{C}\boldsymbol{u}.$$

**Variance Maximization Solution**

Using the eigendecomposition $\boldsymbol{C} = \sum_k \lambda_k \boldsymbol{u}_k \boldsymbol{u}_k^\top$ and writing $\boldsymbol{u} = \sum_k a_k \boldsymbol{u}_k$, the objective becomes $\boldsymbol{u}^\top \boldsymbol{C} \boldsymbol{u} = \sum_k a_k^2 \lambda_k$ subject to $\sum_k a_k^2 = 1$. This is maximized by $a_1 = 1$, all others zero:

$$\boldsymbol{u}_1 = \text{eigenvector of } \boldsymbol{C} \text{ with largest eigenvalue } \lambda_1.$$

The $k$-th principal direction is the direction of maximal variance *orthogonal* to all previous $k-1$ directions. The variance explained by the $k$-th component equals $\lambda_k$.

**Why eigenvectors, of all things?**

The algebra above is short enough to hide what actually happened, so here it is in words. Writing $\boldsymbol{u} = \sum_k a_k \boldsymbol{u}_k$ expresses the candidate direction in the eigenvector basis, and the objective becomes $\sum_k a_k^2 \lambda_k$ with the budget $\sum_k a_k^2 = 1$. That is now a completely transparent problem: you have one unit of "weight" to distribute, each eigenvector pays you its eigenvalue per unit, so obviously you dump the entire budget on the highest-paying one. The maximum-variance direction *is* the top eigenvector, not by coincidence but because the eigendecomposition already sorted the directions by exactly the quantity you are maximising.

Geometrically: the covariance matrix stretches space, most along its top eigenvector. PCA just asks "which way is the data cloud longest?", and the answer to that question is what an eigendecomposition computes.

**Exam trap: singular values are not eigenvalues**

The singular values $d_k$ of the data matrix $\boldsymbol{X}$ and the eigenvalues $\lambda_k$ of $\boldsymbol{X}^\top \boldsymbol{X}$ are related by a *square root*, not equality:

$$d_k = \sqrt{\lambda_k} \qquad \text{(equivalently } \lambda_k = d_k^2\text{, or } d_k^2/n \text{ for the covariance matrix } \boldsymbol{C}\text{)}.$$

"The singular values of $\boldsymbol{X}$ equal the eigenvalues of $\boldsymbol{X}^\top \boldsymbol{X}$" is a recurring distractor — it is off by exactly that square root.

While we are here, two more that get recycled: the matrix of eigenvectors $\boldsymbol{U}$ is **orthogonal**, not symmetric; and the eigenvalues of $\boldsymbol{X}^\top \boldsymbol{X}$ are non-negative and sum to the *total variance*, not to 1.

## 3.3 Uniqueness

PCA is **unique up to sign flips** of the principal directions, provided all eigenvalues are distinct. If two or more eigenvalues are equal (degenerate case), the corresponding eigenvectors span a subspace but their individual directions are arbitrary; any orthonormal basis of that subspace is equally valid.

**Zero eigenvalues**

Zero eigenvalues mean the data lies in a lower-dimensional affine subspace (rank-deficient covariance matrix). There can be *many* zero eigenvalues: if the data of dimension $m$ spans only a $k$-dimensional subspace, then $m-k$ eigenvalues are zero. The corresponding principal directions can be discarded without any loss.

## 3.4 Properties of PCA

Let $\boldsymbol{y}^{(i)} = \boldsymbol{U}^\top \boldsymbol{x}_i$ be the projection of sample $i$.

**Key Properties of PCA Projections**

- **Zero mean:** $\sum_i y_k^{(i)} = 0$ for all $k$ (because the data are centered).
- **Uncorrelated:** $\mathrm{Cov}(y_k, y_l) = 0$ for $k \neq l$ (follows from orthogonality of $\boldsymbol{U}$).
- **Variance:** $\mathrm{Var}(y_k) = \lambda_k$ (the $k$-th eigenvalue).
- **Optimal reconstruction:** truncating to the first $l$ components gives the best rank-$l$ approximation in terms of MSE:

$$\mathrm{MSE}_l = \sum_{k=l+1}^{m} \lambda_k.$$

- **Fraction of variance** explained by the first $l$ components: $\frac{\sum_{k=1}^{l} \lambda_k}{\sum_{k=1}^{m} \lambda_k}$.

**Derivation of the MSE formula.** Retain the first $l$ components. The reconstruction of sample $\boldsymbol{x}_i$ is $\hat{\boldsymbol{x}}_i = \sum_{k=1}^{l} (\boldsymbol{u}_k^\top \boldsymbol{x}_i)\boldsymbol{u}_k$, so the residual is $\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i = \sum_{k=l+1}^{m} (\boldsymbol{u}_k^\top \boldsymbol{x}_i)\boldsymbol{u}_k$. By orthonormality of the $\boldsymbol{u}_k$:

$$\|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|^2 = \sum_{k=l+1}^{m} (\boldsymbol{u}_k^\top \boldsymbol{x}_i)^2.$$

Averaging over all $n$ samples and using $\frac{1}{n}\sum_{i=1}^{n} (\boldsymbol{u}_k^\top \boldsymbol{x}_i)^2 = \boldsymbol{u}_k^\top \boldsymbol{C} \boldsymbol{u}_k = \lambda_k$:

$$\mathrm{MSE}_l = \frac{1}{n}\sum_{i=1}^{n} \|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|^2 = \sum_{k=l+1}^{m} \boldsymbol{u}_k^\top \boldsymbol{C} \boldsymbol{u}_k = \sum_{k=l+1}^{m} \lambda_k.$$

Any other rank-$l$ projection gives a larger MSE, so the top $l$ eigenvectors are the optimal choice.

## 3.5 Examples

### 3.5.1 Iris Data Set

The Iris dataset has $n = 150$ samples (50 per species: *setosa*, *versicolor*, *virginica*) and $m = 4$ features (sepal length, sepal width, petal length, petal width).

PC1 accounts for approximately **92% of the variance** when the raw features are centred but not scaled, and clearly separates *setosa* from the other species. Distinguishing *versicolor* from *virginica* also benefits from PC2; the exact percentages and plots depend on whether the four features were standardised. The example illustrates an important warning: a high-variance direction can expose class structure, but PCA never uses class labels and does not optimise separation.

### 3.5.2 Multiple Tissue Data Set

This microarray dataset contains expression values for 5,565 genes across 102 samples from four human tissue types: **breast** (Br), **prostate** (Pr), **lung** (Lu), and **colon** (Co).

Applied to all genes simultaneously:

- PC1 (5% variance) separates prostate from the rest.
- PC2 (3%) separates colon and also breast samples.
- PC3 (3%) partially separates lung.

**Variance filtering.** Selecting only the top-variance genes before PCA is justified for microarray data (most genes are essentially constant noise). Three filtered subsets were compared:

- **101 genes** (highest variance, XMultiF1): PC1 (36%) separates prostate; PC2 (14%) separates colon; PC3 (8%) separates breast and lung. Much cleaner separation than the full set.
- **13 genes** (XMultiF2, 57% PC1): similar tissue-level separation.
- **5 genes** (XMultiF3, 77% PC1): still separates prostate clearly but other tissues become hard to distinguish. Inspection reveals 4 of the 5 selected genes are highly correlated (ACPP, KLK2, MSMB, TRGC2 — all prostate-related; KRT5 is not).

**Gene clustering.** Applying hierarchical clustering with correlation as distance metric, then selecting the gene with maximal variance per cluster (**92 uncorrelated genes of maximal variance**) gives very similar results to variance-based filtering. The approach ensures genes are not redundant (correlated genes contribute the same information to PCA) while still selecting high-variance representatives.

## 3.6 Kernel Principal Component Analysis

### 3.6.1 The Method

**Kernel PCA** (KPCA) extends PCA to nonlinear projections by performing the linear operations of PCA inside a *reproducing kernel Hilbert space* (RKHS), to which the data are first mapped non-linearly:

$$\boldsymbol{x} \;\mapsto\; \Phi(\boldsymbol{x}).$$

Assuming the mapped data are centered ($\sum_i \Phi(\boldsymbol{x}_i) = \mathbf{0}$), the covariance in feature space is $\boldsymbol{C} = \frac{1}{n}\sum_i \Phi(\boldsymbol{x}_i)\Phi(\boldsymbol{x}_i)^\top$, and the Gram (kernel) matrix has entries $K_{ij} = \Phi(\boldsymbol{x}_j)^\top \Phi(\boldsymbol{x}_i) = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$.

The eigenvalue problem $\boldsymbol{C}\boldsymbol{w} = \lambda \boldsymbol{w}$ cannot be solved in feature space directly (we only have $K_{ij}$), but the solution must lie in the span of the mapped data:

$$\boldsymbol{w} = \sum_{i=1}^{n} \alpha_i \, \Phi(\boldsymbol{x}_i).$$

Substituting and using $K_{ij}$ yields the **kernel eigenvalue problem**:

$$n\lambda \, \boldsymbol{K}\boldsymbol{\alpha} = \boldsymbol{K}^2\boldsymbol{\alpha} \quad \Rightarrow \quad n\lambda \, \boldsymbol{\alpha} = \boldsymbol{K}\boldsymbol{\alpha}.$$

Normalization $\|\boldsymbol{w}\| = 1$ requires $\|\boldsymbol{\alpha}\| = 1/\sqrt{\tilde{\lambda}}$ where $\tilde{\lambda} = n\lambda$, so

$$\boldsymbol{\alpha} = \frac{\tilde{\boldsymbol{\alpha}}}{\sqrt{\tilde{\lambda}}}.$$

The projection of a new point $\boldsymbol{x}$ onto a kernel principal direction is then

$$\boldsymbol{w}^\top \Phi(\boldsymbol{x}) = \sum_{i=1}^{n} \alpha_i \, k(\boldsymbol{x}_i, \boldsymbol{x}).$$

**Kernel PCA Algorithm**

1. **Center** the Gram matrix: $\tilde{\boldsymbol{K}} = \boldsymbol{K} - \frac{1}{n}\boldsymbol{K}\mathbf{1}\mathbf{1}^\top - \frac{1}{n}\mathbf{1}\mathbf{1}^\top\boldsymbol{K} + \frac{1}{n^2}(\mathbf{1}^\top\boldsymbol{K}\mathbf{1})\mathbf{1}\mathbf{1}^\top$.
2. **Eigenvalues**: compute eigenvectors $\boldsymbol{\alpha}$ and eigenvalues $\tilde{\lambda}$ of $\tilde{\boldsymbol{K}}$.
3. **Normalize**: $\alpha_i^{\text{new}} = \alpha_i / (\|\boldsymbol{\alpha}\|\sqrt{n\lambda})$.
4. **Project** a new vector $\boldsymbol{x}$: center its kernel vector and compute $\boldsymbol{w}^\top\Phi(\boldsymbol{x}) = \sum_i \alpha_i\, k(\boldsymbol{x}_i, \boldsymbol{x})$.

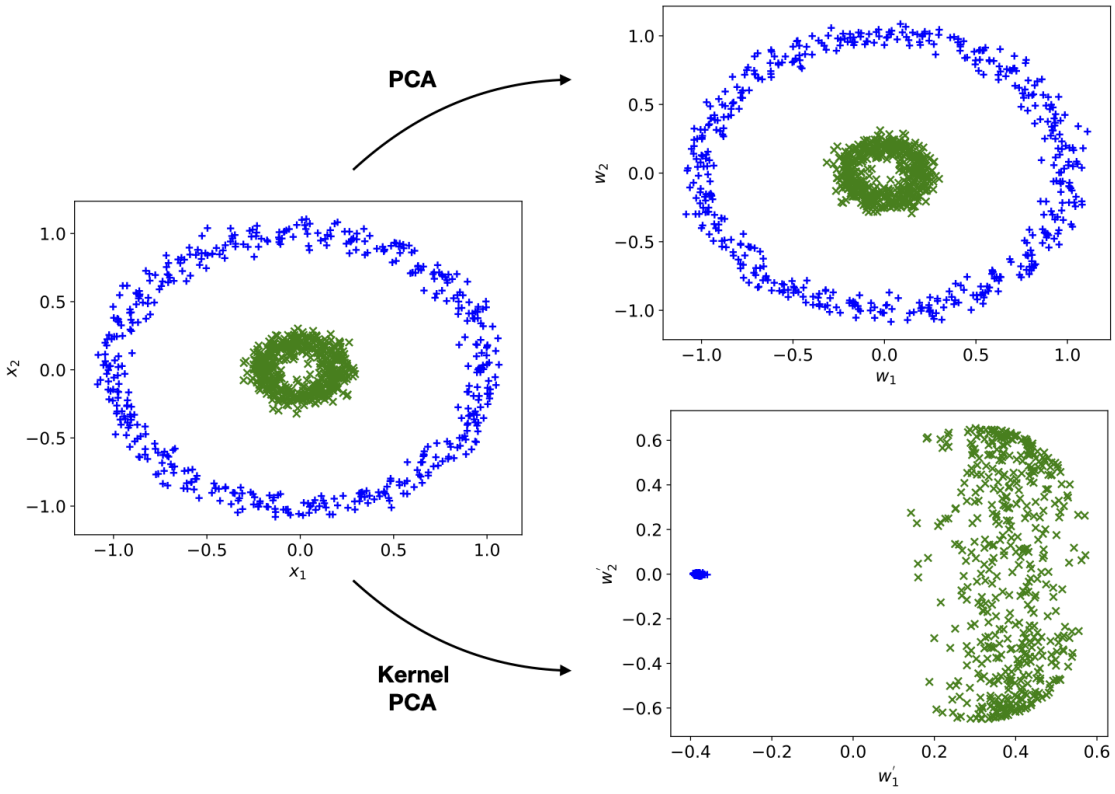


Figure 3.2: Linear PCA cannot unfold concentric rings, whereas an appropriate kernel produces coordinates in which the two groups can be separated.

**Intuition & Mental Model**

Kernel PCA can detect nonlinear structure that linear PCA misses. For example, concentric rings are linearly inseparable but perfectly separable in the RKHS of an RBF kernel. The price is that the representation depends on all training points (no compact weight vector $\boldsymbol{u}$), and choosing the right kernel function requires domain knowledge.

The entire trick is that we never touch $\Phi$ itself. Note what the projection formula $\boldsymbol{w}^\top\Phi(\boldsymbol{x}) = \sum_i \alpha_i\, k(\boldsymbol{x}_i, \boldsymbol{x})$ actually needs: only *kernel evaluations*, never the feature vectors. That is the kernel trick — the feature space may be enormous, even infinite-dimensional, and we never pay for it, because everything we want is expressible through inner products that $k$ computes for us directly.

**$K$ and $C$: same rank, different size**

Two things about the Gram matrix $\boldsymbol{K}$ and the feature covariance $\boldsymbol{C}$ are routinely confused.

**They have different dimensions.** $\boldsymbol{K}$ is $n \times n$ (one row per *data point*), while $\boldsymbol{C}$ lives in feature space and is as big as the feature space itself — possibly infinite. This is precisely why we solve the eigenproblem of $\boldsymbol{K}$ instead of $\boldsymbol{C}$.

**But they share their nonzero eigenvalues**, and therefore their rank. So the claim "$\boldsymbol{K}$ and $\boldsymbol{C}$ always have the same dimensionality *and* the same rank" is a half-truth — the rank half is right, the dimensionality half is wrong.

One more caveat worth having straight: centering $\boldsymbol{K}$ *is* required, but not because "eigenvalues of $\boldsymbol{K}$ could be negative". $\boldsymbol{K}$ is positive semidefinite, so its eigenvalues are never negative. Centering is required because the PCA derivation assumes the mapped data has zero mean in feature space, and $\Phi(\boldsymbol{x})$ generally does not.

# 4

# Independent Component Analysis

Two people are talking at the same time in a room, and you have two microphones running. Each microphone picks up *both* voices, mixed together in different proportions depending on where it sits. Can you recover the two original voices from the two mixtures, knowing nothing about the room, the speakers, or how the mixing happened?

That is the **blind source separation** problem, and remarkably the answer is yes — provided you are willing to assume the underlying sources are *statistically independent* of each other. Independent component analysis is the method that exploits that assumption. Where PCA asks "which directions carry the most variance?", ICA asks a fundamentally different question: "which directions correspond to the independent *causes* that generated this data?". PCA gives you a geometric summary; ICA tries to give you the physics.

The catch, as we will see, is that independence alone is not enough: the sources must also be *non-Gaussian*. That requirement is not a technicality — it is the entire reason the problem is solvable at all, and it is why so much of this chapter is spent measuring how far a distribution sits from a Gaussian.

**ICA Model**

**Generative (mixing) model:**

$$\boldsymbol{x} = \boldsymbol{U}\,\boldsymbol{y}, \qquad p(\boldsymbol{y}) = \prod_{j=1}^{l} p(y_j).$$

$\boldsymbol{U} \in \mathbb{R}^{m\times l}$ is the mixing matrix; the sources $y_j$ are statistically independent.

**Descriptive (demixing) model:**

$$\boldsymbol{y} = \boldsymbol{W}\,\boldsymbol{x}, \qquad \boldsymbol{W} = \boldsymbol{U}^{-1} \quad (\text{for } l = m).$$

Matrix form: $\boldsymbol{X} = \boldsymbol{Y}\boldsymbol{U}^{\top}$, with $\boldsymbol{Y}^{\top}\boldsymbol{Y} = \boldsymbol{D}_m$ (decorrelated, and in fact statistically independent). Unlike PCA, $\boldsymbol{U}$ is **not required to be orthogonal**.

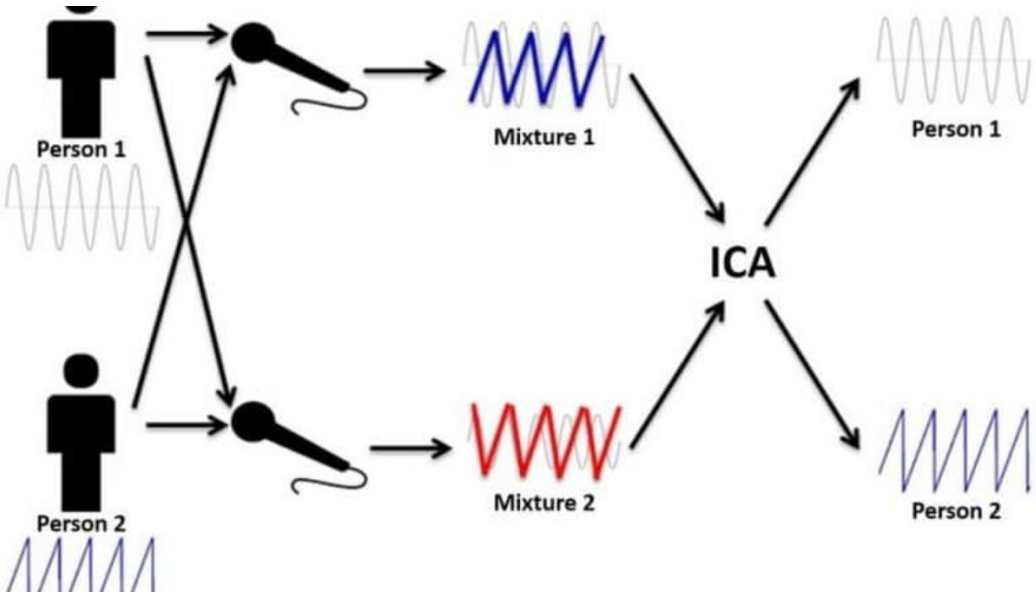


Figure 4.1: The cocktail-party problem: both microphones record different mixtures of both speakers, and ICA attempts to recover the original independent signals.

In the cocktail-party picture, $\boldsymbol{y}$ holds the original voices, $\boldsymbol{U}$ is the room (how loudly each voice reaches each microphone), and $\boldsymbol{x}$ is what the microphones actually record. Solving ICA means recovering $\boldsymbol{W} = \boldsymbol{U}^{-1}$ — the un-mixing matrix — without ever having heard the voices separately.

## 4.1 Identifiability and Uniqueness

The ICA solution is generally **not fully unique**: $\boldsymbol{x} = \boldsymbol{U}\boldsymbol{P}^{-1}\boldsymbol{P}\boldsymbol{y}$ gives another valid decomposition for any invertible $\boldsymbol{P}$. However, if $\boldsymbol{P}$ cannot mix the statistically independent components of $\boldsymbol{y}$, then $\boldsymbol{P}$ must be a product of a permutation matrix and a diagonal scaling matrix.

**Darmois' Theorem (1953) — ICA Identifiability**

Define $x_1 = \sum_j a_j y_j$ and $x_2 = \sum_j b_j y_j$ where $y_j$ are independent random variables. If $x_1$ and $x_2$ are independent, then all $y_j$ for which $a_j b_j \neq 0$ are *Gaussian*.

Consequence: for **non-Gaussian sources**, the ICA solution is unique up to permutation and scaling. The Gaussian case is the only exception.

**Why Gaussians kill ICA — and why the CLT is the key**

Take two independent standard Gaussians and plot them: you get a perfectly circular cloud. Now rotate that cloud by any angle you like. It is *still* a perfectly circular cloud of two independent standard Gaussians — you cannot tell the rotated version from the original. The independence criterion simply has nothing to grip onto, so every rotation is an equally valid "solution" and the sources are unrecoverable in principle, not just in practice.

Non-Gaussian sources break that symmetry, and the **central limit theorem** explains why they are recoverable. The CLT says that summing independent variables pushes the result *towards* a Gaussian. A mixture $\boldsymbol{a}^\top \boldsymbol{y}$ is exactly such a sum — so any mixture of the sources is *more Gaussian* than the sources themselves. Turn that around and you get

the entire ICA strategy: if mixing makes things more Gaussian, then **un-mixing must make things less Gaussian**. Hunt for the projection that is maximally non-Gaussian, and you will land on a single original source. This is why "maximise non-Gaussianity" and "recover the independent sources" are the same instruction, and it is the thread connecting kurtosis, negentropy, and every contrast function below.

**ICA assumptions.**

- Sources are **non-Gaussian** (at most one source can be Gaussian).
- $l \leq m$ (at least as many observations as sources).
- $\boldsymbol{U}$ has full rank $l$.

Source densities $p(y_i)$ are typically approximated by super-Gaussian or unimodal distributions (generative framework).

**Two criteria for independence.**

1. **Mutual information** between components.
2. **Non-Gaussianity** of components.

## 4.2 Measuring Independence

### 4.2.1 Mutual Information

The **mutual information** of a vector-valued random variable measures how far the joint distribution is from a factorial (product) distribution:

$$I(y_1, \ldots, y_l) = \sum_{j=1}^{l} H(y_j) - H(\boldsymbol{y}),$$

where $H(\boldsymbol{a}) = -\int p(\boldsymbol{a}) \ln p(\boldsymbol{a}) \, \mathrm{d}\boldsymbol{a}$ is the (differential) entropy. $I \geq 0$, with equality if and only if all components are statistically independent.

For the linear demixing $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$, using the change-of-variables formula $p(\boldsymbol{y}) = p(\boldsymbol{x})/|\boldsymbol{W}|$:

$$I(y_1, \ldots, y_m) = \sum_{j=1}^{m} H(y_j) - H(\boldsymbol{x}) - \ln |\boldsymbol{W}|.$$

Since $H(\boldsymbol{x})$ is constant in $\boldsymbol{W}$, minimizing mutual information amounts to maximizing $\ln |\boldsymbol{W}| - \sum_j H(y_j)$, which is the ICA objective.

### 4.2.2 Non-Gaussianity

**Negentropy.** The **negentropy** measures how non-Gaussian a distribution is:

$$J(\boldsymbol{y}) = H(\boldsymbol{y}_{\text{gauss}}) - H(\boldsymbol{y}),$$

where $\boldsymbol{y}_{\text{gauss}}$ is a Gaussian with the same covariance as $\boldsymbol{y}$. $J \geq 0$ (Gaussian has maximum entropy for fixed variance); maximizing negentropy minimizes mutual information. Direct estimation of negentropy is difficult in practice.

**Kurtosis.** Non-Gaussianity can be quantified via cumulants, in particular the **kurtosis** (fourth cumulant):

$$\kappa_4 = \mathbb{E}(x^4) - 3\big(\mathbb{E}(x^2)\big)^2.$$

For a Gaussian, $\kappa_3 = \kappa_4 = 0$.

- $\kappa_4 > 0$: **super-Gaussian** (heavier tails than Gaussian, e.g. Laplace distribution).
- $\kappa_4 < 0$: **sub-Gaussian** (lighter tails, e.g. uniform distribution).

Key properties:

- Additivity: $\kappa_4(x_1 + x_2) = \kappa_4(x_1) + \kappa_4(x_2)$ for independent $x_1, x_2$.
- Scaling: $\kappa_4(\alpha x) = \alpha^4 \kappa_4(x)$.
- Mixing tends to move kurtosis toward zero (towards Gaussian by the CLT). Therefore maximising **absolute** kurtosis can recover either super-Gaussian or sub-Gaussian sources; maximising signed kurtosis alone only favours the former.

Kurtosis is not robust: as a fourth-order moment, it is heavily influenced by outliers.

**Sparseness.** For super-Gaussian sources, large positive kurtosis often corresponds to **sparseness**: a sparse variable is usually near zero but occasionally takes large values. This interpretation does not extend to sub-Gaussian sources such as a uniform distribution, which have negative kurtosis but can still be separated by ICA. Sparseness does *not* imply small variance.

**Contrast functions.** Several contrast functions are used in practice to measure non-Gaussianity:

- Kurtosis: $\kappa_4(y)$.
- Mixed cumulants: $\frac{1}{12}\kappa_3^2(y) + \frac{1}{48}\kappa_4^2(y)$ (normalized to zero mean, unit variance).
- $|\mathbb{E}_y[G(y)] - \mathbb{E}_\nu[G(\nu)]|^p$ where $\nu$ is a standard Gaussian and $G$ is a nonlinearity; robust choices include $G(x) = \log\cosh(ax)$ and $G(x) = \exp(-ax^2/2)$ with $a \geq 1$.

## 4.3 Whitening and Rotation Algorithms

ICA is not unique up to scaling, so the standard approach is to first remove the scaling ambiguity by **whitening** (also called sphering), then find an orthogonal **rotation** that maximizes non-Gaussianity.

**Whitened** data satisfies $\boldsymbol{Y}^\top \boldsymbol{Y} = \boldsymbol{I}$. Starting from $\boldsymbol{X} = \boldsymbol{Y}\boldsymbol{U}^\top$ with $\boldsymbol{C} = \frac{1}{n}\boldsymbol{X}^\top \boldsymbol{X}$:

$$\boldsymbol{I} = \boldsymbol{C}^{-1/2}\,\frac{1}{n}\boldsymbol{X}^\top \boldsymbol{X}\,\boldsymbol{C}^{-1/2} = \frac{1}{n}\boldsymbol{C}^{-1/2}\boldsymbol{U}\,\boldsymbol{Y}^\top \boldsymbol{Y}\,\boldsymbol{U}^\top \boldsymbol{C}^{-1/2}.$$

This means $\hat{\boldsymbol{U}} = \frac{1}{\sqrt{n}}\boldsymbol{C}^{-1/2}\boldsymbol{U}$ is orthogonal, and the ICA solution decomposes as:

$$\boldsymbol{Y} = \frac{1}{\sqrt{n}}\,\boldsymbol{X}\,\boldsymbol{C}^{-1/2}\,\hat{\boldsymbol{U}}.$$

The two-step procedure is:

1. **Whiten:** compute $\tilde{\boldsymbol{X}} = \boldsymbol{X}\boldsymbol{C}^{-1/2}$.
2. **Rotate:** find the orthogonal matrix $\hat{\boldsymbol{U}}$ that maximizes the contrast function (super-Gaussianity / sparseness).

**Why split the search into whitening and rotation?**

A general invertible mixing matrix has $m^2$ free parameters — a big search space. But notice that ICA only cares about *independence*, and independence implies decorrelation. So we can knock out the easy part first: **whitening** (a PCA-style rescaling to unit covariance) enforces decorrelation and fixes all the stretching and scaling for free. Whatever freedom is left *after* whitening must preserve unit covariance — and the only transforms that do that are *rotations*.

That is the payoff: the search collapses from "all invertible matrices" to "rotations only", which is a far smaller and better-behaved space. Whitening does the decorrelating that second-order statistics can handle, and the rotation step then uses higher-order statistics (kurtosis, negentropy) to find the one angle that makes the components genuinely *independent*, not merely uncorrelated. The artificial example below makes this visible: mixing turns a square into a parallelogram, whitening rounds it into a circle (decorrelated but still dependent), and only the final rotation snaps it back into the axis-aligned square.

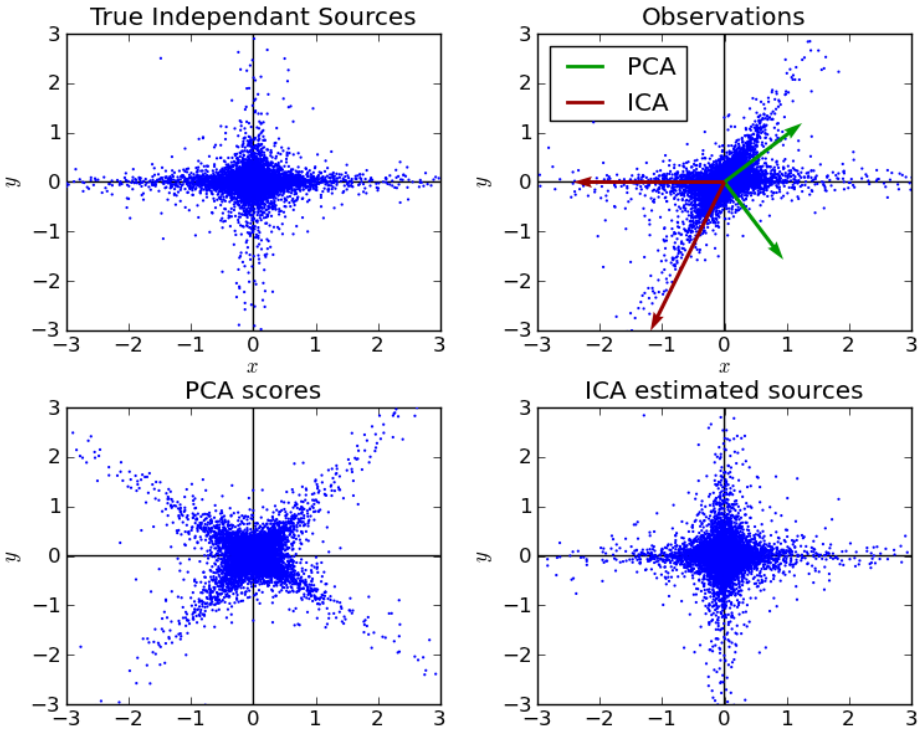


Figure 4.2: PCA decorrelates the observations but does not recover the independent axes. ICA uses higher-order structure to rotate the representation back towards the true independent sources.

## 4.4 INFOMAX Algorithm

**INFOMAX** (Bell & Sejnowski, 1995) minimizes mutual information by maximizing the entropy of a nonlinear transform of $\boldsymbol{y}$:

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}, \qquad H(\boldsymbol{g}(\boldsymbol{y})) \text{ is maximized}, \quad \boldsymbol{g}(\boldsymbol{y}) = (g(y_1), \ldots, g(y_l))^\top.$$

At maximum entropy the mutual information $I(g(y_1), \ldots, g(y_l)) = 0$, so the components are independent. The nonlinearity $g$ is chosen to approximate the CDF of the source distribution; a common choice is $g(y_i) = \tanh(y_i)$ (super-Gaussian sources).

The entropy expands as

$$H(\boldsymbol{g}(\boldsymbol{y})) \approx H(\boldsymbol{x}) + \frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{l} |\ln g'(y_{ij})| + \ln|\boldsymbol{W}|.$$

**INFOMAX Update Rules (Natural Gradient)**

**tanh** ($g(y_j) = \tanh(y_j)$):

$$\Delta \boldsymbol{W} \propto (\boldsymbol{I} - 2\,\boldsymbol{g}(\boldsymbol{y})\,\boldsymbol{y}^\top)\boldsymbol{W}$$

**sigmoid** ($g(y_j) = 1/(1+e^{-y_j})$):

$$\Delta \boldsymbol{W} \propto (\boldsymbol{I} + (1 - 2\,\boldsymbol{g}(\boldsymbol{y}))\,\boldsymbol{y}^\top)\boldsymbol{W}$$

INFOMAX is equivalent to a generative approach using **maximum likelihood**.

**Intuition & Mental Model**

The natural gradient (premultiplication by $\boldsymbol{W}^\top\boldsymbol{W}$) converts the standard gradient $(\boldsymbol{W}^\top)^{-1} \pm \ldots \boldsymbol{x}^\top$ into the more elegant update involving $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x}$. It also makes the algorithm *equivariant*: performance does not depend on the scaling or conditioning of the input.

## 4.5 EASI Algorithm

**EASI** (Equivariant Adaptive Separation via Independence, Cardoso & Laheld, 1996) uses the update rule

$$\Delta \boldsymbol{W} \propto (\boldsymbol{I} - \boldsymbol{y}\boldsymbol{y}^\top - \boldsymbol{g}(\boldsymbol{y})\boldsymbol{y}^\top + \boldsymbol{y}\,\boldsymbol{g}^\top(\boldsymbol{y}))\boldsymbol{W},$$

where $\boldsymbol{g}$ is the same class of contrast functions as in INFOMAX (tanh or sigmoid). EASI is designed for online learning and is also equivariant by construction.

## 4.6 FastICA Algorithm

**FastICA** (Hyvärinen, 1999) is probably the most widely used ICA algorithm. It is a *fixed-point* iteration (analogous to Oja's rule for PCA) that maximizes non-Gaussianity as measured by a contrast function $g$ with derivative $g'$:

**FastICA Update**

$$\boldsymbol{w}^{\text{new}} = \mathbb{E}[\boldsymbol{x}\,g(\boldsymbol{w}^\top\boldsymbol{x})] - \mathbb{E}[g'(\boldsymbol{w}^\top\boldsymbol{x})]\,\boldsymbol{w}.$$

The new $\boldsymbol{w}^{\text{new}}$ is then normalized to unit length.

Key properties:

- Whitening and rotation algorithm.
- Generalizes kurtosis maximization to arbitrary contrast functions $g$.
- Extended to extract multiple independent components simultaneously (deflation or symmetric extraction).

## 4.7 ICA Extensions

Beyond the basic square, invertible model several extensions exist:

- **Generative approach:** assumptions on the source densities $p(y_i)$, sub-Gaussian sources with specific priors.
- **Non-linear ICA:** mixing function $\boldsymbol{x} = g(\boldsymbol{y})$ is nonlinear; solutions are often not unique without additional constraints.
- **Overcomplete ICA:** more sources than observations ($l > m$); the system is underdetermined but useful for sparse representations of signals.
- **Undercomplete ICA:** fewer sources than observations ($l < m$); reduces to a lower-dimensional model.

## 4.8 ICA vs. PCA

| ICA | PCA |
|---|---|
| Seeks *causes* of the data | Seeks *geometrical abstractions* |
| Statistically independent components | Decorrelated (orthogonal) components |
| Fits an independent-source model | Captures directions of maximal variance |
| Scale invariant | Not scale invariant |
| Unique up to scaling and permutation | Unique (up to sign) |
| Requires non-Gaussian sources (at most one Gaussian) | Uses only second-order structure |
| Components not ranked | Components ranked by eigenvalue |

**Intuition & Mental Model**

PCA gives the best linear dimensionality reduction in terms of MSE; ICA finds the interpretation that is statistically most meaningful for independent source models. For Gaussian data both methods give the same result (any rotation of decorrelated Gaussians is equally independent), which is why ICA requires non-Gaussianity.

## 4.9 Artificial ICA Examples

Starting with 1,000 points from two independent **uniform distributions** (uniform = sub-Gaussian, $\kappa_4 < 0$), the three processing stages can be visualized:

1. **Original sources:** square-shaped scatter (axes-aligned).
2. **Mixing** ($\boldsymbol{X} = \boldsymbol{Y}\boldsymbol{U}^\top$): the square is rotated and sheared into a parallelogram/diamond shape — the components are now dependent.
3. **Whitening** ($\boldsymbol{X}\boldsymbol{C}^{-1/2}$): the shape becomes circular (unit covariance); correlation is removed but statistical dependence remains.
4. **Rotation** (ICA): the circle is aligned back to a square, recovering the original independent sources.

Both INFOMAX and FastICA recover the original square (up to permutation and sign). For super-Gaussian sources (Laplace distribution), the shape is a four-pointed star, and the same pipeline recovers it correctly.

## 4.10 Real World ICA Examples

### 4.10.1 Iris Data Set

On the Iris data, ICs are **ordered by their impact on the observations** (column norms of the mixing matrix $\boldsymbol{U}$), *not* by variance. The first independent component explains approximately 90% of the variance in the data and likely encodes the *size of the blossom* (a single dominant biological factor). Unlike PCA, ICA does not guarantee components are ranked by variance.

### 4.10.2 Multiple Tissue Data Set

Applied to the multiple tissue microarray data (102 samples, 4 tissue types), the first three ICs provide biologically meaningful separations:

- **IC1** (29% variance): separates *secretory/reproductive* tissue (prostate and breast) from *internal organ* tissue (colon and lung) — a biologically interpretable axis.
- **IC2** (28%): separates prostate + lung from breast + colon.
- **IC3** (27%): separates prostate + colon from breast + lung.
- All combinations of the first three ICs yield clean separations except for partial overlap between breast and lung samples.

Genes most correlated to IC1 include SERPINA7, LAMB3, **AR**, CCNG2, KLF5, CCL20, SLC39A14, ATP1B1, GSTP1, LAD1. The **androgen receptor** (AR) is the 3rd most related gene to IC1 — consistent with IC1 separating reproductive/secretory tissues, since androgen signaling drives prostate gland differentiation and also plays a role in normal breast physiology.

**Effect of increasing the number of components.** With 8 or 20 ICA components, the overall tissue-level separation does *not* improve relative to 4 components. Instead, higher-order ICs focus on progressively finer-grained sub-populations (e.g. specific sub-groups within colon samples), and the main tissue axes (IC1: prostate vs. others; IC2: prostate+lung vs. breast+colon) shift in their index but remain. This illustrates a key difference from PCA: ICA components are not ordered by variance, and adding more components reveals biological sub-structure rather than explaining more total variance.

## 4.11 Kurtosis Maximization Results in Independent Components

The theoretical justification for kurtosis-based ICA rests on a key inequality: the kurtosis of a linear mixture of independent variables is always less than or equal to the *maximum* kurtosis of the individual sources. Formally, for $x = \boldsymbol{a}^\top \boldsymbol{y}$ with $\|\boldsymbol{a}\| = 1$ and $y_j$ independent:

$$\kappa_4(x) = \sum_j a_j^4 \, \kappa_4(y_j) \;\leq\; \max_j |\kappa_4(y_j)|.$$

Equality holds when one $|a_j| = 1$ and all others are zero — i.e. when the projection aligns exactly with one source. Therefore **maximizing $|\kappa_4|$ over all unit-norm projections recovers a single source component**. Iterating (deflating each recovered component) separates all sources.

**Limitations of kurtosis**

Because kurtosis involves fourth moments, it is sensitive to outliers: a single extreme data point can dominate the estimate. More robust contrast functions such as $G(x) = \log\cosh(x)$ (used in FastICA) are preferred in practice when outliers are expected.

# 5

# Factor Analysis

PCA has a blind spot: it explains *all* the variance in the data, and it has no idea which part of that variance is real signal and which is measurement noise. Point it at a noisy sensor and it will happily devote a component to the noise.

Factor analysis fixes this by being honest about noise from the start. It splits every observed variable into two pieces: a part driven by a handful of shared hidden **factors**, and a part that is private noise belonging to that one variable alone. The crucial modelling decision is that the noise covariance $\boldsymbol{\Psi}$ is *diagonal* — noise in one measurement is independent of noise in another. The consequence is immediate and worth pausing on: since the noise cannot create any cross-variable correlation, **every correlation between observed variables must be explained by the shared factors**. That single sentence is the engine of the whole method, and it is the answer to most conceptual exam questions about FA.

So the mental split is: *factors explain what the variables have in common; noise explains what is left over, variable by variable.* (Get this backwards — "factors explain the variance, noise explains the correlations" — and you have picked the classic wrong answer.)

## 5.1 The Factor Analysis Model

**Notation warning**

This chapter writes the factor loading matrix as $\boldsymbol{\Lambda}$ and the factors as $\boldsymbol{z}$, following the standard statistics convention. The lecture slides and exam questions frequently use $\boldsymbol{U}$ for the loading matrix and $\boldsymbol{y}$ for the factor vector instead — and the FABIA chapter later uses $U$ too. The model is identical, only the letters change: $\boldsymbol{C} = \boldsymbol{\Lambda}\boldsymbol{\Lambda}^\top + \boldsymbol{\Psi}$ is the same statement as $\boldsymbol{C} = \boldsymbol{U}\boldsymbol{U}^\top + \boldsymbol{\Psi}$.

**Factor Analysis Model**

For centered data $\boldsymbol{x} \in \mathbb{R}^m$ with $l \leq m$ factors $\boldsymbol{z} \in \mathbb{R}^l$:

$$\boldsymbol{x} = \boldsymbol{\Lambda}\boldsymbol{z} + \boldsymbol{\varepsilon}, \qquad \boldsymbol{z} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Psi}),$$

where $\boldsymbol{\Lambda} \in \mathbb{R}^{m \times l}$ is the **factor loading matrix** and $\boldsymbol{\Psi} \in \mathbb{R}^{m \times m}$ is a **diagonal** noise covariance (so noise components are mutually independent). The model gives $\boldsymbol{x} \mid \boldsymbol{z} \sim \mathcal{N}(\boldsymbol{\Lambda}\boldsymbol{z}, \boldsymbol{\Psi})$.

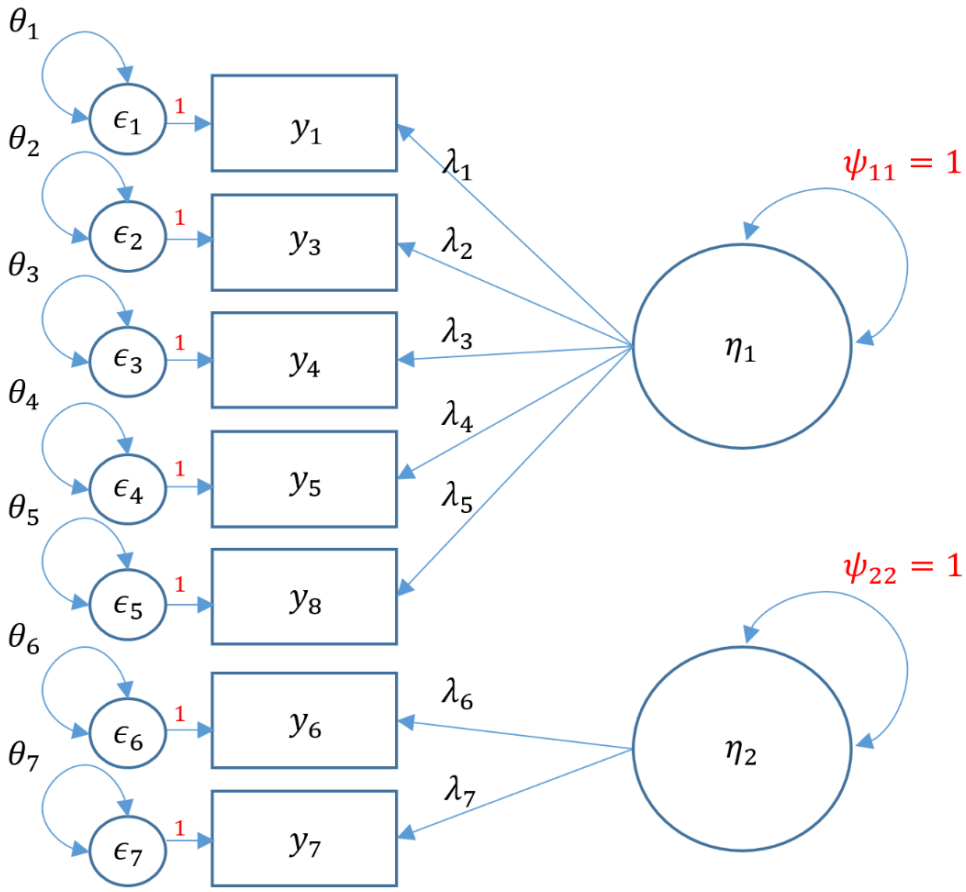


Figure 5.1: A two-factor measurement model. In the diagram, the latent variables $\eta_1, \eta_2$ correspond to our factors $\boldsymbol{z}$, the boxes $y_i$ to observed variables $x_i$, the arrows $\lambda_i$ to factor loadings, and each $\epsilon_i$ is a private error source with variance $\theta_i$.

**Covariance decomposition.** The example in Fig. 5.1 has two groups of observed variables driven by two latent factors. Shared arrows from a factor induce covariance among its observations, whereas the separate $\epsilon_i$ terms can only contribute variable-specific variance. This is the graphical version of the following covariance decomposition.

Marginalising over $\boldsymbol{z}$ gives $\boldsymbol{x} \sim \mathcal{N}(\mathbf{0},\, \boldsymbol{\Lambda}\boldsymbol{\Lambda}^\top + \boldsymbol{\Psi})$. The model-implied covariance is compared with the sample covariance $\boldsymbol{C} = \frac{1}{n}\boldsymbol{X}^\top \boldsymbol{X}$:

$$\boldsymbol{C} \;\approx\; \boldsymbol{\Lambda}\boldsymbol{\Lambda}^\top + \boldsymbol{\Psi}.$$

FA is therefore a **decomposition of the covariance matrix**: $\boldsymbol{\Lambda}$ captures the shared (dependent) variance; $\boldsymbol{\Psi}$ captures the unique (independent) variance per variable. Because $\boldsymbol{\Psi}$ is diagonal, all correlations between observations can *only* be explained by the factors.

In matrix form, $\boldsymbol{X} = \boldsymbol{Z}\boldsymbol{\Lambda}^\top + \boldsymbol{E}$ with model assumptions $\frac{1}{n}\boldsymbol{Z}^\top\boldsymbol{Z} = \boldsymbol{I}$, $\boldsymbol{Z}^\top\boldsymbol{E} = \mathbf{0}$, and $\frac{1}{n}\boldsymbol{E}^\top\boldsymbol{E} = \boldsymbol{\Psi}$.

**Factor scores (estimation).** Treating factor recovery as regression $\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{A}$ and using the model approximations $\frac{1}{n}\boldsymbol{X}^\top\boldsymbol{X} \approx \boldsymbol{\Lambda}\boldsymbol{\Lambda}^\top + \boldsymbol{\Psi}$ and $\frac{1}{n}\boldsymbol{X}^\top\boldsymbol{Y} \approx \boldsymbol{\Lambda}$:

$$\hat{\boldsymbol{A}} = (\boldsymbol{\Lambda}\boldsymbol{\Lambda}^\top + \boldsymbol{\Psi})^{-1}\boldsymbol{\Lambda}, \qquad \boldsymbol{Y} = \boldsymbol{X}(\boldsymbol{\Lambda}\boldsymbol{\Lambda}^\top + \boldsymbol{\Psi})^{-1}\boldsymbol{\Lambda}.$$

Applying the matrix inversion lemma to avoid inverting the $m \times m$ matrix:

$$\boldsymbol{Y} = \boldsymbol{X}\,\boldsymbol{\Psi}^{-1}\boldsymbol{\Lambda}\,(\boldsymbol{I} + \boldsymbol{\Lambda}^\top\boldsymbol{\Psi}^{-1}\boldsymbol{\Lambda})^{-1}.$$

**Communality.** The **communality** of observation variable $x_j$ is the proportion of its variance explained by the factors:

$$c_j = \frac{\mathrm{Var}(x_j) - \mathrm{Var}(\varepsilon_j)}{\mathrm{Var}(x_j)} = \frac{\sum_{k=1}^{l} \lambda_{jk}^2}{\Psi_{jj} + \sum_{k=1}^{l} \lambda_{jk}^2}.$$

Factor $z_t$ individually contributes $\lambda_{jt}^2/(\Psi_{jj} + \sum_k \lambda_{jk}^2)$ to $x_j$. Like PCA, projecting onto $l$ factors maximizes the variance that can be explained by $l$ factors — but FA counts only signal variance, not noise.

**Rotation non-identifiability.** The FA loading matrix is **not identifiable**: every orthogonal rotation gives the same marginal covariance and likelihood, $\boldsymbol{\Lambda z} = \boldsymbol{\Lambda V V}^\top \boldsymbol{z} = \boldsymbol{\Lambda' z'}$ for any orthogonal $\boldsymbol{V}$. Rotations are therefore chosen to improve interpretability:

- **Varimax**: simplifies columns of the loading matrix, encouraging each factor to have a small set of large loadings and many near-zero loadings.
- **Quartimax**: simplifies rows, encouraging each variable to load strongly on only a small number of factors; it can produce one broad general factor.
- **Equimax**: a compromise between Varimax and Quartimax.

## 5.2 Maximum Likelihood Factor Analysis

Since $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Lambda\Lambda}^\top + \boldsymbol{\Psi})$, the log-likelihood over $n$ samples is

$$\ln \mathcal{L}(\boldsymbol{\Lambda}, \boldsymbol{\Psi}) = \sum_{i=1}^{n} \left[ -\tfrac{m}{2} \ln(2\pi) - \tfrac{1}{2} \ln |\boldsymbol{\Lambda\Lambda}^\top + \boldsymbol{\Psi}| - \tfrac{1}{2} (\boldsymbol{x}^i)^\top (\boldsymbol{\Lambda\Lambda}^\top + \boldsymbol{\Psi})^{-1} \boldsymbol{x}^i \right].$$

Maximising this has **no closed form**, so the **EM algorithm** is used with the latent factors $\boldsymbol{z}^i$ as the hidden variables.

**FA EM Algorithm**

**E-step:** compute the posterior of the factors for each sample (using the Gaussian conditional formula):

$$\boldsymbol{z}^i \mid \boldsymbol{x}^i \sim \mathcal{N}(\boldsymbol{\mu}_{\boldsymbol{z}|\boldsymbol{x}^i}, \boldsymbol{\Sigma}_{\boldsymbol{z}|\boldsymbol{x}}),$$

$$\boldsymbol{\mu}_{\boldsymbol{z}^i|\boldsymbol{x}^i} = (\boldsymbol{x}^i)^\top (\boldsymbol{\Lambda\Lambda}^\top + \boldsymbol{\Psi})^{-1} \boldsymbol{\Lambda}, \qquad \boldsymbol{\Sigma}_{\boldsymbol{z}|\boldsymbol{x}} = \boldsymbol{I} - \boldsymbol{\Lambda}^\top (\boldsymbol{\Lambda\Lambda}^\top + \boldsymbol{\Psi})^{-1} \boldsymbol{\Lambda}.$$

Note: the posterior covariance $\boldsymbol{\Sigma}_{\boldsymbol{z}|\boldsymbol{x}}$ is the same for all data points (it does not depend on $\boldsymbol{x}^i$).

**M-step:** update the parameters by setting gradients to zero:

$$\boldsymbol{\Lambda}^{\text{new}} = \left( \tfrac{1}{n} \sum_{i=1}^{n} \boldsymbol{x}^i \, \mathbb{E}_{\boldsymbol{z}^i|\boldsymbol{x}^i}[\boldsymbol{z}^i]^\top \right) \left( \tfrac{1}{n} \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z}^i|\boldsymbol{x}^i}[\boldsymbol{z}^i (\boldsymbol{z}^i)^\top] \right)^{-1},$$

$$\boldsymbol{\Psi}^{\text{new}} = \tfrac{1}{n} \operatorname{diag}\Big( \sum_{i=1}^{n} \boldsymbol{x}^i (\boldsymbol{x}^i)^\top - \sum_{i=1}^{n} \mathbb{E}_{\boldsymbol{z}^i|\boldsymbol{x}^i}[\boldsymbol{z}^i] \, \boldsymbol{x}^i (\boldsymbol{\Lambda}^{\text{new}})^\top \Big),$$

where $\mathbb{E}[\boldsymbol{z}^i (\boldsymbol{z}^i)^\top] = \boldsymbol{\mu}_{\boldsymbol{z}^i|\boldsymbol{x}^i} \boldsymbol{\mu}_{\boldsymbol{z}^i|\boldsymbol{x}^i}^\top + \boldsymbol{\Sigma}_{\boldsymbol{z}|\boldsymbol{x}}$.

**Matrix inversion speedup.** When $m \gg l$, directly inverting the $m \times m$ matrix $(\boldsymbol{\Lambda}\boldsymbol{\Lambda}^\top + \boldsymbol{\Psi})$ is expensive. The matrix inversion lemma gives an $l \times l$ inversion instead:

$$(\boldsymbol{\Lambda}\boldsymbol{\Lambda}^\top + \boldsymbol{\Psi})^{-1} = \boldsymbol{\Psi}^{-1} - \boldsymbol{\Psi}^{-1}\boldsymbol{\Lambda}(\boldsymbol{I} + \boldsymbol{\Lambda}^\top\boldsymbol{\Psi}^{-1}\boldsymbol{\Lambda})^{-1}\boldsymbol{\Lambda}^\top\boldsymbol{\Psi}^{-1}.$$

Since $\boldsymbol{\Psi}$ is diagonal, $\boldsymbol{\Psi}^{-1}$ is trivially cheap. Furthermore the sample covariance $\boldsymbol{C}$ only needs to be computed once per EM run.

## 5.3 Factor Analysis vs. PCA and ICA

| | **FA** | **PCA / ICA** |
|---|---|---|
| Noise model | Explicit Gaussian noise $\boldsymbol{\Psi}$ | None (PCA: no noise; ICA: no noise) |
| Factors/sources | Gaussian priors $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ | PCA: none; ICA: non-Gaussian |
| Objective | Max. signal variance (covariance fit) | PCA: all variance; ICA: independence |
| Uniqueness | Rotation-indeterminate | PCA: up to sign; ICA: up to scale/permutation |
| Estimation | EM (no closed form) | PCA: eigen; ICA: gradient/fixed-point |

**Intuition & Mental Model**

FA is the right choice when you believe observations contain genuine measurement noise that should be separated from the latent signal. PCA absorbs noise into the components (it explains *all* variance, noise included), while FA explicitly models it via $\boldsymbol{\Psi}$. ICA additionally demands statistical independence of the sources and therefore requires non-Gaussian source distributions.

## 5.4 Artificial Factor Analysis Examples

To see when FA fails, consider 50-dimensional data generated from two independent **super-Gaussian** (Laplacian) latent sources. In 2-D projection the two sources form a characteristic "star" or diamond shape (uniform mass on four arms), which reflects their non-Gaussian character.

**ICA** correctly recovers this star shape: because the true sources are statistically independent and non-Gaussian, the ICA objective (contrast function or kurtosis) can identify them.

**FA** with two factors does *not* recover the star. FA's generative model assumes the latent factors $\boldsymbol{z}$ are Gaussian. It can still recover a useful low-dimensional subspace and covariance structure, but covariance alone cannot choose the particular rotation corresponding to independent non-Gaussian sources. ICA uses higher-order information to identify that rotation.

**Why FA fails for non-Gaussian sources**

FA is the maximum-likelihood model when factors are Gaussian. Forcing super-Gaussian data into a Gaussian-factor likelihood throws away the feature that identifies the axes: non-Gaussian shape. EM may fit the covariance well while returning a rotated version of the sources. ICA is preferable when the goal is specifically to recover independent, non-Gaussian causes.

## 5.5 Real World Factor Analysis Examples

### 5.5.1 Iris Data Set

In the lecture's Iris analysis, a **one-factor** model captures much of the dominant size-related variation. Its factor scores separate *setosa* strongly and order the other two species, but one should not read this as a general guarantee of clean class separation: FA is unsupervised and never sees the species labels.

### 5.5.2 Multiple Tissue Data Set

With **four factors** and no rotation the four components have distinct biological interpretations:

- **FA1** — cleanly separates prostate samples.
- **FA2** — partially separates breast from lung, but the overlap is substantial.
- **FA3** — separates colon samples.
- **FA4** — separates a subset of lung samples.

Applying **Varimax** rotation to the four factors changes the factor axes but does not change the overall fit. The pairwise scatter plots (all six pairs among FA1–FA4) show that *separation is slightly worse* than without rotation: the rotation mixes biological signal across components rather than concentrating it.

**Quartimax** rotation gives a result similar to Varimax — the separation quality is again slightly worse than the unrotated solution.

**Rotation does not help here**

Rotation methods such as Varimax and Quartimax search for a sparse, interpretable factor loading matrix. On this data the unrotated MLE already concentrates each factor on one tissue type, so rotation does not improve tissue separation and can even dilute it. Rotation is most valuable when the unrotated factors are hard to interpret biologically.

# 6

# Scaling and Projection Methods

Every method in this chapter squeezes high-dimensional data down into a few dimensions — usually two, so you can actually look at it. The reason to care is that we cannot see in 5,000 dimensions, and a great deal of data analysis begins with wanting to *look* at the thing.

What separates the methods is a single question: **what are you trying to preserve on the way down?** Everything in this chapter is an answer to that question, and once you see it that way the zoo becomes a short list:

- **Projection pursuit** — preserve *interestingness* (non-Gaussianity).
- **MDS** — preserve *pairwise distances*.
- **Isomap** — preserve pairwise distances, but measured *along the manifold* (geodesics) rather than through empty space.
- **LLE** — preserve each point's *local linear relationship* to its neighbours.
- ***t*-SNE** — preserve *neighbourhood probabilities*, with a deliberate trick to stop everything crowding together.
- **SOM / GTM** — preserve *topology*, by pinning the projection to a grid of latent points.
- **NMF** — not a projection to visualise, but a decomposition into *additive parts*.

Two warnings that recur throughout, and that exams like to probe.

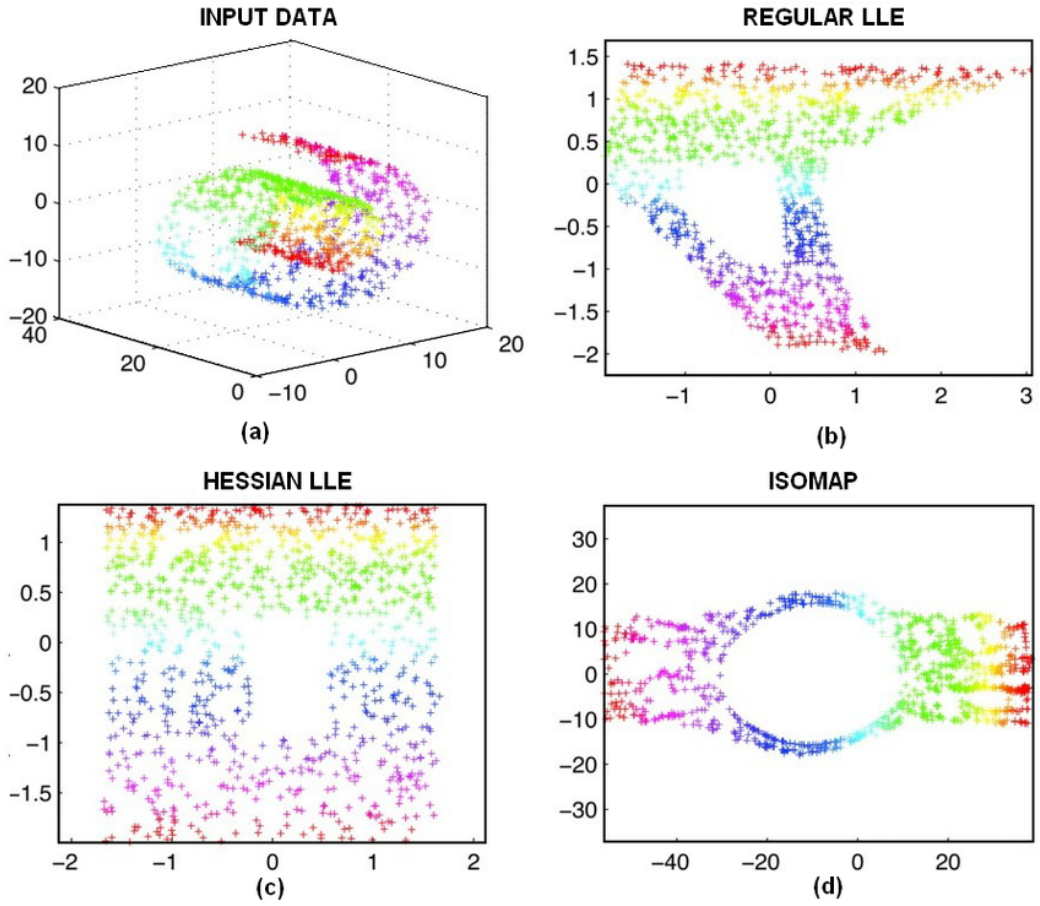


Figure 6.1: The same Swiss-roll input embedded with regular LLE, Hessian LLE, and Isomap. Colour tracks position on the original manifold. Different preservation criteria produce visibly different two-dimensional geometries; Hessian LLE is included as a variant of LLE.

Figure 6.1 is a useful warning against treating "manifold learning" as one unique operation. Regular LLE preserves local reconstruction weights, its Hessian variant imposes a different local smoothness criterion, and Isomap preserves shortest-path distances in a neighbourhood graph. Even on the same coloured input, their embeddings need not have the same global shape.

First, the projection always goes to a *lower*-dimensional space — any statement claiming these methods project *up* in order to spread points apart is a distractor. Second, a manifold method is only as good as its assumption: the last few sections show that LLE, Isomap, and $t$-SNE can expose useful structure on smooth manifolds and image data, but that the result depends on the preservation criterion and neighbourhood parameters. They then all struggle on the gene-expression data because those samples do not lie on a simple smooth low-dimensional manifold.

## 6.1 Projection Pursuit

**Projection pursuit** asks: among all linear projections $t = \boldsymbol{w}^\top \boldsymbol{x}$ (normalised to zero mean, unit variance), which direction $\boldsymbol{w}$ is most *interesting*? The classical answer is: the **least Gaussian** direction.

The rationale comes from information theory: among all distributions with a fixed mean and covariance, the Gaussian maximises entropy. Consequently, a projection that looks close to Gaussian is "boring" — it carries no structure beyond the second-order statistics already captured

by PCA. A projection with low entropy (far from Gaussian) reveals genuine non-linear or non-Gaussian structure.

**Projection Pursuit Objective**

Find $\boldsymbol{w}$ that minimises the differential entropy $H(t)$ of the projected variable $t = \boldsymbol{w}^\top \boldsymbol{x}$, subject to $\mathrm{Var}(t) = 1$.

This optimisation is difficult in practice because it requires estimating a density. Two qualitatively different types of structure can be found:

- **Unimodal super-Gaussians** — heavy-tailed, peaked distributions (single cluster with outliers).
- **Multimodal distributions** — several separated clusters.

In practice, projection pursuit uses the same contrast functions as ICA — kurtosis, negentropy approximations — as tractable surrogates for $H(t)$. Projection pursuit can therefore be seen as a precursor to ICA; ICA adds the constraint that multiple projected directions must be mutually independent.

## 6.2 Multidimensional Scaling

### 6.2.1 The Method

**Multidimensional scaling** (MDS) projects data to a low-dimensional target space $\boldsymbol{y}_i = f(\boldsymbol{x}_i; \boldsymbol{w})$ while keeping the *pairwise distances* between points as intact as possible.

Define:

$$\delta_{ij} = \|\boldsymbol{x}_i - \boldsymbol{x}_j\| \quad \text{(original distances)}, \qquad d_{ij} = \|\boldsymbol{y}_i - \boldsymbol{y}_j\| \quad \text{(projected distances)}.$$

The goal is $d \approx \delta$. Different **stress measures** formalise the mismatch:

$$R_1(d, \delta) = \frac{\sum_{i<j} (d_{ij} - \delta_{ij})^2}{\sum_{i<j} \delta_{ij}^2} \qquad \text{(Kruskal's measure — penalises large absolute errors)}$$

$$R_2(d, \delta) = \sum_{i<j} \left( \frac{d_{ij} - \delta_{ij}}{\delta_{ij}} \right)^2 \qquad \text{(fractional/relative errors)}$$

$$R_3(d, \delta) = \frac{1}{\sum_{i<j} \delta_{ij}} \sum_{i<j} \frac{(d_{ij} - \delta_{ij})^2}{\delta_{ij}} \qquad \text{(Sammon mapping — compromise)}$$

These stress objectives can be minimised iteratively, but not every MDS variant needs gradient descent: classical metric MDS has the eigendecomposition solution described below, while non-metric MDS additionally fits monotone disparities to the observed dissimilarities. The gradient of $R_1$ with respect to the projected position $\boldsymbol{y}_k$ is:

$$\frac{\partial}{\partial \boldsymbol{y}_k} R_1 = \frac{2}{\sum_{i<j} \delta_{ij}^2} \sum_{j \neq k} (d_{kj} - \delta_{kj}) \, \frac{\boldsymbol{y}_k - \boldsymbol{y}_j}{d_{kj}},$$

and analogous expressions hold for $R_2$ and $R_3$ (replacing the weight by $1/\delta_{kj}^2$ or $1/\delta_{kj}$, respectively). Viewing $R$ as a potential energy, the gradients are **forces** that push or pull each point toward its correct distance from every other point.

A special case is **metric MDS** (also called *principal coordinates analysis*): when distances are Euclidean and one seeks a linear embedding, the optimal solution can be computed analytically via eigendecomposition of the centred distance matrix — no gradient descent is needed.

### 6.2.2 Examples

MDS is applied to the **Multiple Tissue** gene-expression data set (four tissue types: colon, lung, breast, prostate). The 101 or 13 genes with the largest variance across samples are selected as features.

- **Metric MDS** (101 features): colon samples (orange) cluster in the upper-left; prostate (green) separates to the right; lung (blue) and breast (red) form an overlapping cluster in the lower region. Using only 13 features gives a similar layout.
- **Kruskal's non-metric MDS** (101 and 13 features): the separation pattern is qualitatively the same as metric MDS.
- **Sammon's non-linear mapping** (101 features): the tissue groups are less well separated than with metric or Kruskal's measure. With only 13 features the clusters are *more spread out*, but the qualitative separation is still inferior to the other variants.

Overall, metric MDS and Kruskal's non-metric MDS perform similarly on this data set, and both outperform Sammon's mapping.

## 6.3 Non-negative Matrix Factorization

### 6.3.1 The Method

**Non-negative matrix factorization** (NMF) approximates a non-negative data matrix $\boldsymbol{X} \in \mathbb{R}^{n\times m}$ ($X_{ij} \geq 0$) as

$$\boldsymbol{X} \;\approx\; \boldsymbol{Y}\boldsymbol{U}^\top \;=\; \sum_{k=1}^{l} \boldsymbol{y}_k\, \boldsymbol{u}_k^\top,$$

where $\boldsymbol{Y} \in \mathbb{R}^{n\times l}$ and $\boldsymbol{U} \in \mathbb{R}^{m\times l}$ are also required to be non-negative ($Y_{ik} \geq 0$, $U_{jk} \geq 0$).

The non-negativity constraint encourages an additive, often **parts-based representation**: every observation is reconstructed by *adding* non-negative parts, never by subtraction. Localised semantic parts are a common outcome, not a mathematical guarantee. This contrasts with PCA, whose eigenvectors can have both signs, producing holistic (whole-image) components.

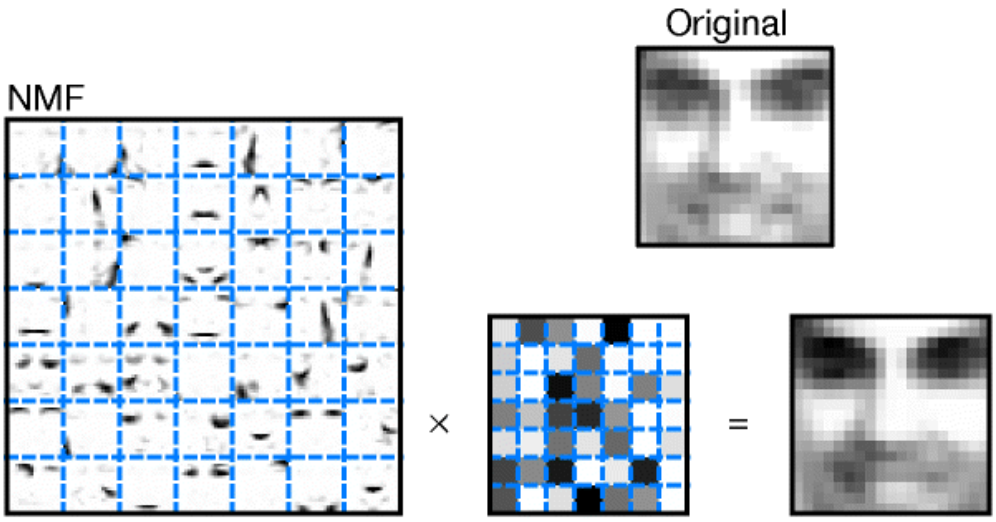


Figure 6.2: NMF reconstructs a face by combining non-negative, spatially localised basis parts with non-negative activation weights.

**Multiplicative update rules.** Two objectives are in common use.

**Objective 1 — KL divergence.** For non-negative matrices normalised to sum to one:

$$D(\boldsymbol{A}\|\boldsymbol{B}) = \sum_{ij}\Big(A_{ij}\log\frac{A_{ij}}{B_{ij}} - A_{ij} + B_{ij}\Big).$$

Minimising $D(\boldsymbol{X}\|\boldsymbol{Y}\boldsymbol{U}^\top)$ leads to multiplicative updates:

$$Y_{ik} \leftarrow Y_{ik}\frac{\sum_{j=1}^{m} U_{jk}\,X_{ij}/(\boldsymbol{Y}\boldsymbol{U}^\top)_{ij}}{\sum_{j=1}^{m} U_{jk}}, \qquad U_{jk} \leftarrow U_{jk}\frac{\sum_{i=1}^{n} Y_{ik}\,X_{ij}/(\boldsymbol{Y}\boldsymbol{U}^\top)_{ij}}{\sum_{i=1}^{n} Y_{ik}}.$$

**Objective 2 — Frobenius norm.** $\|\boldsymbol{X} - \boldsymbol{Y}\boldsymbol{U}^\top\|_F^2 = \sum_{ij}(X_{ij} - (\boldsymbol{Y}\boldsymbol{U}^\top)_{ij})^2$. Multiplicative updates are obtained by multiplying the fixed-point condition $\boldsymbol{X} = \boldsymbol{Y}\boldsymbol{U}^\top$ from the right by $\boldsymbol{U}$ (for $\boldsymbol{Y}$) or from the left by $\boldsymbol{Y}^\top$ (for $\boldsymbol{U}$):

$$Y_{ik} \leftarrow Y_{ik}\frac{(\boldsymbol{X}\boldsymbol{U})_{ik}}{(\boldsymbol{Y}\boldsymbol{U}^\top\boldsymbol{U})_{ik}}, \qquad U_{jk} \leftarrow U_{jk}\frac{(\boldsymbol{Y}^\top\boldsymbol{X})_{kj}}{(\boldsymbol{Y}^\top\boldsymbol{Y}\boldsymbol{U}^\top)_{kj}}.$$

At a fixed point the left and right sides are equal.

**Why multiplicative updates preserve non-negativity**

Both update rules multiply each entry by a ratio of non-negative quantities. Starting from a non-negative initialisation, all entries remain non-negative throughout training.

NMF can be extended to **sparse NMF**, adding sparsity penalties on $\boldsymbol{Y}$ (few parts active per sample) and/or $\boldsymbol{U}$ (each part described by few features). In gene-expression analysis this corresponds to: part = pathway, few genes participate in each pathway, and only few pathways are active per sample.

### 6.3.2 Examples

**Parts-based face representation..** Applied to a face database, NMF learns *local face parts* (eye regions, nose patches, mouth shapes). Vector quantisation (VQ) and PCA both learn holistic representations — entire faces or signed global components. The parts-based decomposition from NMF is more interpretable for many real-world objects.

**Bicluster recovery (FABIA toy data)..** A synthetic data matrix (1,000 genes, 100 samples, 13 biclusters) generated by the FABIA biclustering package provides a ground-truth test bed. The blocks of elevated expression are localised in specific gene/sample subsets; the background is near zero.

- **NMF (KL divergence, NMFDIV)**: reconstructs the block structure well with small residual noise. The absolute factors and loadings matrices each show sparse, localised activity corresponding to individual biclusters.
- **NMF (Euclidean distance, NMFEU)**: similarly good reconstruction; the residual error is small and structureless.
- **Sparse NMF (NMFSC)**: enforcing sparseness misses some blocks entirely when the sparseness parameter is too large. The loadings matrix becomes nearly uniform across all genes (columns look identical) — the method cannot distinguish which genes belong to which bicluster. The takeaway is that *the sparseness parameter is difficult to tune*; too little gives the same result as plain NMF, too much destroys the bicluster structure.
- **FABIA biclustering**: the FABIA model (Chapter 8) recovers the bicluster structure more faithfully than any NMF variant, with sharper factor and loading matrices.

## 6.4 Locally Linear Embedding

### 6.4.1 The Method

**Locally linear embedding** (LLE) computes a low-dimensional, neighbourhood-preserving embedding via a nonlinear mapping. The key insight is that on a smooth manifold, each point can be approximated by a linear combination of its nearest neighbours; LLE finds the embedding that *preserves* those local linear relationships.

**Step 1 — compute reconstruction weights..** For each point $\boldsymbol{x}_i$, find $k$ nearest neighbours and solve for weights $W_{ij}$ that best reconstruct $\boldsymbol{x}_i$ from its neighbours:

$$\varepsilon(\boldsymbol{W}) = \sum_i \left\| \boldsymbol{x}_i - \sum_{j=1}^{k} W_{ij}\, \boldsymbol{x}_j \right\|^2, \quad \text{subject to} \quad \sum_{j=1}^{k} W_{ij} = 1.$$

The sum-to-one constraint makes the weights invariant to translations; uniform rotations and rescalings multiply the local least-squares objective without changing its minimiser. For point $i$, form the centred neighbour matrix from vectors $\boldsymbol{x}_i - \boldsymbol{x}_j$ and its local Gram matrix $C^{(i)}_{jk} = (\boldsymbol{x}_i - \boldsymbol{x}_j)^\top (\boldsymbol{x}_i - \boldsymbol{x}_k)$. The optimal neighbour weights have the compact form

$$\boldsymbol{w}_i = \frac{(\boldsymbol{C}^{(i)})^{-1}\mathbf{1}}{\mathbf{1}^\top (\boldsymbol{C}^{(i)})^{-1}\mathbf{1}},$$

with all non-neighbour weights set to zero. In practice one solves this linear system rather than forming the inverse explicitly.

**Step 2 — compute the embedding..** Hold the weights $W_{ij}$ fixed and find low-dimensional coordinates $\boldsymbol{y}_i$ that minimise the same reconstruction error:

$$\Phi(\boldsymbol{Y}) = \sum_{ij} M_{ij}\, \boldsymbol{y}_i^\top \boldsymbol{y}_j, \quad \text{where} \quad M_{ij} = \delta_{ij} - W_{ij} - W_{ji} + \sum_k W_{ki} W_{kj},$$

which can be written compactly as $\boldsymbol{M} = (\boldsymbol{I} - \boldsymbol{W})^\top (\boldsymbol{I} - \boldsymbol{W})$. Because the row sums of $\boldsymbol{W}$ are one, $(\boldsymbol{I} - \boldsymbol{W})\mathbf{1} = \mathbf{0}$, so $\boldsymbol{M}$ always has a zero eigenvalue with eigenvector $\mathbf{1}$. The optimal $d$-dimensional embedding consists of the *bottom* $d$ eigenvectors of $\boldsymbol{M}$ (those with the $d$ smallest non-zero eigenvalues); the trivial zero eigenvector is discarded.

**LLE implementation notes**

- Neighbours can be found by $k$-NN (Euclidean), $\varepsilon$-ball, or KD trees for efficiency.
- If $k > m$ the local covariance $\boldsymbol{C}$ is rank-deficient; regularise with $\boldsymbol{C} \leftarrow \boldsymbol{C} + \varepsilon \boldsymbol{I}$ where $\varepsilon \approx 10^{-3}\,\mathrm{tr}(\boldsymbol{C})$.
- The matrix $\boldsymbol{M}$ is sparse (only $k$ off-diagonal entries per row), so its eigenvectors can be computed efficiently.

### 6.4.2 Examples

LLE succeeds on data that genuinely lives on a smooth low-dimensional manifold.

**Swiss Roll.** LLE "unrolls" the 3-D spiral manifold into a flat 2-D rectangle, recovering the original 2-D parameterisation of the roll.

**Face images.** Applied to a large collection of face photographs, LLE produces a 2-D embedding in which one axis encodes head pose and the other encodes expression or illumination — a smooth manifold structure in pixel space.

**"S" curve.** On a 3-D S-shaped surface LLE fails to produce a clean unrolling; the embedded scatter plot is disordered.

**Multiple Tissue (gene expression).** With $k \in \{5, 7, 9, 12\}$, LLE gives poor tissue separation in all cases. The four tissue types are not concentrated on a low-dimensional manifold in gene-expression space — they are spread throughout the high-dimensional space — so LLE's assumption is violated.

## 6.5 Isomap

### 6.5.1 The Method

**Isomap** computes a quasi-isometric low-dimensional embedding. Like LLE it is a non-linear projection, but instead of preserving local linear relationships it preserves **geodesic distances** — shortest paths along the data manifold.

**Why geodesic rather than Euclidean?.** For data lying on a curved manifold, the straight-line Euclidean distance between two far-apart points cuts through empty space and underestimates the true manifold distance. The geodesic distance follows the surface and is a better measure of "closeness" on the manifold.

**Algorithm..**

1. **Build the neighbourhood graph $G$.** Connect $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$ if $d(\boldsymbol{x}_i, \boldsymbol{x}_j) < \varepsilon$ ($\varepsilon$-Isomap) or if $i$ is among the $k$ nearest neighbours of $j$ ($k$-Isomap). Set each edge length to $d(\boldsymbol{x}_i, \boldsymbol{x}_j)$.
2. **Compute all-pairs shortest paths** using Floyd's algorithm: initialise $d_G(\boldsymbol{x}_i, \boldsymbol{x}_j) = d(\boldsymbol{x}_i, \boldsymbol{x}_j)$ for linked pairs and $\infty$ otherwise, then iterate

$$d_G(\boldsymbol{x}_i, \boldsymbol{x}_j) \leftarrow \min\big(d_G(\boldsymbol{x}_i, \boldsymbol{x}_j),\ d_G(\boldsymbol{x}_i, \boldsymbol{x}_k) + d_G(\boldsymbol{x}_k, \boldsymbol{x}_j)\big).$$

   Store the result as the geodesic distance matrix $\boldsymbol{D}_X$.
3. **Compute the embedding.** Convert distances to inner products via the doubly-centred transform

$$\tau(\boldsymbol{D}) = -\tfrac{1}{2}\,\boldsymbol{H}\,\boldsymbol{D}^2\,\boldsymbol{H}, \qquad \boldsymbol{H} = \boldsymbol{I}_n - \tfrac{1}{n}\mathbf{1}\mathbf{1}^\top,$$

   where $\boldsymbol{D}^2$ is the element-wise square of $\boldsymbol{D}$. The embedding coordinates are given by $y_{ij} = \sqrt{\lambda_p}\, v_{ji}$, where $\lambda_p$ and $v_p$ are the $p$-th largest eigenvalue and eigenvector of $\tau(\boldsymbol{D}_X)$.

The objective that is minimised is $E = \|\tau(\boldsymbol{D}_X) - \tau(\boldsymbol{D}_Y)\|_{L^2}$, where $\boldsymbol{D}_Y$ collects Euclidean distances in the projected space. Setting the embedding to the top-$l$ eigenvectors of $\tau(\boldsymbol{D}_X)$ is the exact analytic solution.

### 6.5.2 Examples

**Hand images.** $n = 2{,}000$ images of a hand opening and closing at varying wrist orientations ($k = 6$). Isomap recovers the underlying two-dimensional manifold: one axis encodes *wrist rotation*, the other *finger extension* — a semantically meaningful 2-D embedding of a high-dimensional image space.

**Tree-count data (Barro Colorado Island).** 50 one-hectare plots, 225 tree species. Classical metric MDS (cmdscale), $k$-Isomap ($k = 3$, $k = 5$), and $\varepsilon$-Isomap ($\varepsilon = 0.45$) give qualitatively different layouts of the 50 plots in 2-D. The neighbourhood parameter strongly influences the result.

**Multiple Tissue.** Results are not as good as with linear methods (metric MDS, FA) for any setting of $\varepsilon$. As with LLE, the gene-expression data do not lie on a smooth manifold, so geodesic-distance-based methods gain nothing over Euclidean methods and may even perform worse.

## 6.6 The Generative Topographic Mapping

### 6.6.1 The Method

The **Generative Topographic Mapping** (GTM) is a probabilistic, non-linear latent variable model designed as a principled alternative to Self-Organising Maps (SOMs). Like FA, GTM maps from a low-dimensional latent space to the observation space; unlike FA the mapping $\boldsymbol{x}(\boldsymbol{y}; \boldsymbol{w})$ is nonlinear.

Latent variables $\boldsymbol{y} \in \mathbb{R}^l$ are mapped to a manifold $\mathcal{S}$ embedded in $\mathbb{R}^m$ ($m > l$). Because a deterministic map from an $l$-dimensional space to an $m$-dimensional space concentrates all probability mass on a zero-volume surface (probability densities vanish), GTM places a **Gaussian ball** around each mapped point:

$$p(\boldsymbol{t} \mid \boldsymbol{y}, \boldsymbol{w}, \beta) = \left(\frac{\beta}{2\pi}\right)^{m/2} \exp\!\left(-\frac{\beta}{2}\|\boldsymbol{x}(\boldsymbol{y}, \boldsymbol{w}) - \boldsymbol{t}\|^2\right).$$

The marginal density over observations is obtained by integrating over the latent space:

$$p(\boldsymbol{t} \mid \boldsymbol{w}, \beta) = \int p(\boldsymbol{t} \mid \boldsymbol{y}, \boldsymbol{w}, \beta)\, p(\boldsymbol{y})\, \mathrm{d}\boldsymbol{y}.$$

In practice the latent prior $p(\boldsymbol{y})$ is approximated by a uniform discrete distribution over $L$ grid points $\{\boldsymbol{y}_j\}$:

$$p(\boldsymbol{y}) = \frac{1}{L}\sum_{j=1}^{L} \delta(\boldsymbol{y} - \boldsymbol{y}_j), \qquad p(\boldsymbol{t} \mid \boldsymbol{w}, \beta) = \frac{1}{L}\sum_{j=1}^{L} p(\boldsymbol{t} \mid \boldsymbol{y}_j, \boldsymbol{w}, \beta).$$

This is a **constrained Gaussian mixture model** in $\mathbb{R}^m$: the $L$ mixture centres are not free but must all lie on the manifold $\mathcal{S} = \{\boldsymbol{x}(\boldsymbol{y}_j; \boldsymbol{w})\}$.

The model is fitted by maximising the log-likelihood $\log \mathcal{L} = \sum_{i=1}^{n} \ln p(\boldsymbol{t}_i \mid \boldsymbol{w}, \beta)$ using EM.

### 6.6.2 Examples

A toy problem illustrates GTM: data are generated from a 1-D curve embedded in 2-D. The $L$ latent grid points (shown as + in the initial configuration) are initialised along a straight line and each surrounded by its Gaussian noise distribution. After fitting, the latent points align with the data curve (right panel): the model has learned the non-linear manifold, and each observation is explained by the nearest Gaussian centre on the fitted curve.

## 6.7 *t*-Distributed Stochastic Neighbor Embedding

### 6.7.1 The Method

***t*-SNE** maps each high-dimensional point to a 2- or 3-dimensional position such that similar points land nearby and dissimilar points land far apart.

**Stochastic Neighbor Embedding (SNE)..** For each pair in the original space, define the *conditional probability* that $\boldsymbol{x}_i$ would pick $\boldsymbol{x}_j$ as its neighbour:

$$p_{j|i} = \frac{\exp(-\|\boldsymbol{x}_i - \boldsymbol{x}_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|\boldsymbol{x}_i - \boldsymbol{x}_k\|^2 / 2\sigma_i^2)}.$$

The bandwidths $\sigma_i$ are set so that the *perplexity*

$$\mathrm{Perp}(P_i) = 2^{H(P_i)}, \qquad H(P_i) = -\sum_{j \neq i} p_{j|i} \log_2 p_{j|i},$$

equals a user-chosen constant (the perplexity hyperparameter, typically 5–50). Here $H(P_i)$ is the Shannon entropy of the conditional distribution $P_i = \{p_{j|i}\}$ over the observation-space neighbours of $i$. A larger perplexity considers more neighbours as relevant. In the low-dimensional space define analogously:

$$q_{j|i} = \frac{\exp(-\|\boldsymbol{y}_i - \boldsymbol{y}_j\|^2)}{\sum_{k \neq i} \exp(-\|\boldsymbol{y}_i - \boldsymbol{y}_k\|^2)}.$$

The embedding is found by minimising the KL divergence $KL(P\|Q) = \sum_{i \neq j} p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$ via gradient descent.

**Problems with SNE — and how t-SNE fixes them..**

1. **Difficult optimisation.** t-SNE *symmetrises* the joint distribution: $p_{ij} = (p_{j|i} + p_{i|j})/(2n)$, giving simpler gradients.
2. **Crowding problem.** In 10 dimensions, 11 points can be mutually equidistant; there is no faithful 2-D map for them. t-SNE uses a **heavy-tailed Student's $t$-distribution** for the low-dimensional similarities:

$$q_{ij} = \frac{(1 + \|\boldsymbol{y}_i - \boldsymbol{y}_j\|^2)^{-1}}{\sum_{k \neq l}(1 + \|\boldsymbol{y}_k - \boldsymbol{y}_l\|^2)^{-1}}.$$

   The heavier tail allows moderately distant points in the original space to be pushed further apart in 2-D without penalty, alleviating crowding.

**Why the $t$-distribution helps**

A Gaussian distribution has thin tails: forcing points that are somewhat separated in high dimensions to be far apart in 2-D is expensive. The $t$-distribution (with one degree of freedom, i.e. a Cauchy distribution) has much fatter tails, so the same KL gradient can accommodate larger spread without distorting the truly nearby points.

### 6.7.2 Examples

**MNIST digits** (6,000 points, 30-D PCA codes): t-SNE produces 10 well-separated clusters, one per digit class, with little overlap. Sammon's mapping, Isomap, and LLE all fail to achieve clean digit separation on the same data.

**Olivetti faces**: t-SNE groups images by identity into tight clusters; the other three methods produce much more mixed embeddings.

**COIL-20 objects**: each physical object, photographed under rotation, forms a closed loop (ring) in the 2-D t-SNE map — the topology of the rotation manifold is preserved. The other methods do not recover these loops clearly.

**Iris data**: PCA partially overlaps *versicolor* and *virginica*; a particular t-SNE run may display three visually separated groups. This is exploratory evidence, not proof of separability: t-SNE is stochastic and can exaggerate gaps, so results should be checked across seeds and perplexities.

**Multiple Tissue**: both perplexity settings (30 and 50) give poor tissue separation — same conclusion as LLE and Isomap. The gene-expression data do not lie on a 2-D manifold.

## 6.8 Self-Organizing Maps

### 6.8.1 The Method

The **Self-Organizing Map** (SOM, also Kohonen map) pursues two goals simultaneously: **clustering** and **dimensionality reduction (down-projection)**.

A SOM maintains a lattice of $K$ *neurons*. Each neuron $k$ has a fixed grid position $\boldsymbol{y}_k \in \mathbb{R}^l$ (equidistantly filling a hypercube) and a learnable weight vector $\boldsymbol{w}_k \in \mathbb{R}^m$ in the input space. The goal is to preserve neighbourhood relations: points that are close in input space should activate neighbouring neurons (**topologically ordered maps**, TOMs).

**Online update rule..** For each incoming sample $\boldsymbol{x}$:

1. Find the *winner* or best-matching unit: $k^* = \arg\min_s \|\boldsymbol{x} - \boldsymbol{w}_s\|^2$. The equivalent dot-product $\arg\max_s \boldsymbol{x}^\top \boldsymbol{w}_s$ is valid only when the relevant vector norms are fixed.
2. Move $k^*$ and its neighbours towards $\boldsymbol{x}$:

$$\boldsymbol{w}_k^{\text{new}} = \boldsymbol{w}_k + \eta\, \delta(\|\boldsymbol{y}_k - \boldsymbol{y}_{k^*}\|)\, (\boldsymbol{x} - \boldsymbol{w}_k),$$

   where $\eta$ is the learning rate and $\delta(\cdot)$ is the *neighbourhood (window) function*: it equals 1 for $\boldsymbol{y}_k = \boldsymbol{y}_{k^*}$ and decreases monotonically with lattice distance.

**SOM has no objective function**

For the classical online SOM, there is no generally valid single scalar objective whose gradient exactly produces every neighbourhood update (special variants and limiting cases do have energy functions). Practical consequences:

- Learning rate, neighbourhood width, and stopping rule require schedules and empirical tuning.
- No guarantee that topographic ordering is achieved.
- Convergence and comparison are less clean than for an explicit likelihood model.
- Different runs should be compared with external diagnostics such as quantisation and topographic error, not a universal training likelihood.
- The result carries no probability density and cannot be handed to a downstream probabilistic model.

GTM (Section 6.6) was developed as a principled probabilistic replacement.

### 6.8.2 Examples

**Example 1 — 1-D SOM in a 2-D circle.** Initially all weights are collapsed to a single point. Over ∼150,000 updates the 1-D chain of neurons unfolds to trace the boundary of the circle, filling the circular data distribution.

**Example 2 — 2-D SOM on a uniform square.** A $10 \times 10$ grid of neurons progressively fills the square data space; by 300,000 steps the grid covers the entire region uniformly. *With a different random initialisation* the grid develops kinks that do **not** disappear even after 400,000 steps — a local minimum with no way to detect or escape it.

**Non-uniform sampling.** When data density is higher at the centre than the border, the SOM allocates more neurons to the centre — the grid is compressed there and stretched at the boundary, reflecting the input distribution.

7

# Clustering

Clustering is the unsupervised task of splitting samples into groups so that members of the same group are more alike than members of different groups. This chapter walks through the main families and how they relate: *mixture models* give the probabilistic, soft-assignment view; *k-means* is the hard-assignment special case that falls out when you simplify a Gaussian mixture; *hierarchical* clustering builds a whole tree of nested groupings instead of committing to one number of clusters; and *similarity-based* methods need only pairwise similarities, not feature vectors at all.

## 7.1 Mixture Models

### 7.1.1 The Method

A **mixture model** defines a generative distribution over observed data $\boldsymbol{x}$ by introducing a discrete latent variable $j \in \{1, \dots, l\}$ that indexes the component:

$$p(\boldsymbol{x} \mid \theta) = \sum_{j=1}^{l} p(j)\, p(\boldsymbol{x} \mid j, \theta_j),$$

where $p(j) = w_j \geq 0$ with $\sum_j w_j = 1$ are the *mixture weights* and $p(\boldsymbol{x} \mid j, \theta_j)$ is the $j$-th component density. The model provides a flexible way to represent multi-modal or multi-cluster distributions as a convex combination of simpler component distributions.

**Bayesian cluster assignment.** Given an observation $\boldsymbol{x}$, the posterior probability of belonging to component $j$ is obtained via Bayes' theorem:

$$p(j \mid \boldsymbol{x}, \theta) = \frac{w_j\, p(\boldsymbol{x} \mid j, \theta_j)}{\sum_{t=1}^{l} w_t\, p(\boldsymbol{x} \mid t, \theta_t)}.$$

This *soft* assignment (also called *responsibility*) is the key quantity in the E-step of EM.

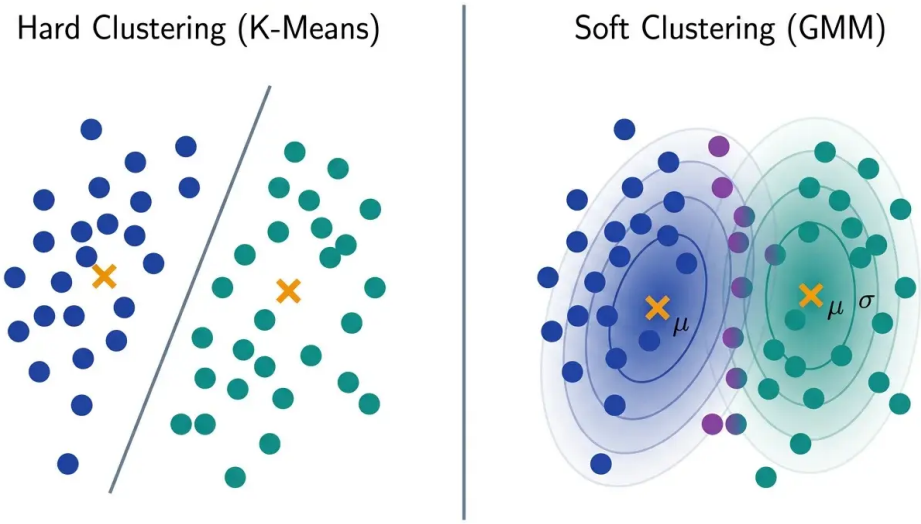


Figure 7.1: Hard versus soft clustering. $k$-means draws a hard boundary and assigns every point to exactly one centre. A Gaussian mixture instead models component densities; points in their overlap have non-trivial responsibility for both components and are shown in a blended colour.

The right panel of Fig. 7.1 visualises the posterior $p(j \mid \boldsymbol{x}, \theta)$ rather than merely the closest centre. Far inside a component the responsibility is nearly one; in the overlapping density contours it is shared between the two components. The means $\boldsymbol{\mu}_j$ locate the centres, while the elliptical contours reflect the component covariance.

It is worth naming the four pieces of that formula, because exam questions love to give you three of them and ask for the fourth:

- the **prior** $p(j) = w_j$ — how common component $j$ is overall, before seeing any data;
- the **likelihood** $p(\boldsymbol{x} \mid j, \theta_j)$ — how well component $j$ explains *this particular* observation;
- the **evidence** $p(\boldsymbol{x} \mid \theta)$ — the denominator, i.e. the total probability of the observation under the whole mixture;
- the **posterior** $p(j \mid \boldsymbol{x}, \theta)$ — what we actually want: the probability that component $j$ produced this observation.

They are tied together by $\text{posterior} = \dfrac{\text{prior} \times \text{likelihood}}{\text{evidence}}$, and any one of them can be recovered from the other three by rearranging.

**Reading Bayes' rule in both directions**

**Forward (find the posterior).** A component has prior $p(j) = 0.1$, the observation's likelihood under it is $p(\boldsymbol{x} \mid j) = 0.5$, and the evidence is $p(\boldsymbol{x} \mid \theta) = 0.2$. Then

$$p(j \mid \boldsymbol{x}) = \frac{0.1 \times 0.5}{0.2} = 0.25.$$

**Backward (find the likelihood).** Now suppose you are told the prior $p(j) = 0.1$, the *posterior* $p(j \mid \boldsymbol{x}) = 0.3$, and the evidence $p(\boldsymbol{x} \mid \theta) = 0.2$, and asked for the likelihood. Just rearrange the same equation:

$$p(\boldsymbol{x} \mid j) = \frac{p(j \mid \boldsymbol{x})\, p(\boldsymbol{x} \mid \theta)}{p(j)} = \frac{0.3 \times 0.2}{0.1} = 0.60.$$

**Sanity check on the intuition.** In the second case the posterior (0.3) is much larger than the prior (0.1) — the observation made this component *three times more plausible* than it was a priori. That can only happen if the component explains the data unusually well, so the likelihood must come out large. It does.

**Log-likelihood gradient.** The gradient of the log-likelihood with respect to component parameters $\theta_j$ factorises cleanly:

$$\frac{\partial}{\partial \theta_j} \log p(\boldsymbol{x} \mid \theta) = p(j \mid \boldsymbol{x}, \theta) \, \frac{\partial}{\partial \theta_j} \log p(\boldsymbol{x} \mid j, \theta_j).$$

The overall gradient is therefore the posterior-weighted sum of component gradients — exactly the structure exploited by EM.

### 7.1.2 Mixture of Gaussians

In a **mixture of Gaussians** (MoG) each component is a multivariate Gaussian $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$. EM iterates between computing soft assignments (E-step) and maximising the expected complete-data log-likelihood (M-step).

**E-step.** Compute the responsibility $r_{ij} = p(j \mid \boldsymbol{x}_i, \theta)$ for every sample–component pair:

$$r_{ij} = \frac{w_j \, \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\sum_{t=1}^{l} w_t \, \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_t, \boldsymbol{\Sigma}_t)}.$$

**M-step.** Update the parameters using the responsibilities:

$$w_j^{\text{new}} = \frac{1}{n} \sum_{i=1}^{n} r_{ij},$$

$$\boldsymbol{\mu}_j^{\text{new}} = \frac{\sum_i r_{ij} \, \boldsymbol{x}_i}{\sum_i r_{ij}},$$

$$\boldsymbol{\Sigma}_j^{\text{new}} = \frac{\sum_i r_{ij} (\boldsymbol{x}_i - \boldsymbol{\mu}_j^{\text{new}})(\boldsymbol{x}_i - \boldsymbol{\mu}_j^{\text{new}})^\top}{\sum_i r_{ij}}.$$

**MAP-MoG.** The ML solution can degenerate (a single-sample component with $\boldsymbol{\Sigma}_j \to \mathbf{0}$). Adding conjugate priors regularises the solution:

- **Wishart** prior on the precision: $\mathcal{W}(\boldsymbol{\Sigma}_j^{-1} \mid \alpha, \boldsymbol{\Psi})$.
- **Dirichlet** prior on the weights: $\mathcal{D}(\boldsymbol{w} \mid \gamma)$.
- **Gaussian** prior on the means: $\mathcal{N}(\boldsymbol{\mu}_j \mid \boldsymbol{\nu}_j, \eta^{-1} \boldsymbol{\Sigma}_j)$.

The MAP-EM updates replace ML formulas with:

$$w_j^{\text{new}} = \frac{\sum_i r_{ij} + \gamma - 1}{n + l(\gamma - 1)},$$

$$\boldsymbol{\mu}_j^{\text{new}} = \frac{\sum_i r_{ij}\, \boldsymbol{x}_i + \eta\, \boldsymbol{\nu}_j}{\sum_i r_{ij} + \eta},$$

$$\boldsymbol{\Sigma}_j^{\text{new}} = \frac{\sum_i r_{ij} (\boldsymbol{x}_i - \boldsymbol{\mu}_j^{\text{new}})(\boldsymbol{x}_i - \boldsymbol{\mu}_j^{\text{new}})^\top + 2\boldsymbol{\Psi} + \eta (\boldsymbol{\mu}_j^{\text{new}} - \boldsymbol{\nu}_j)(\boldsymbol{\mu}_j^{\text{new}} - \boldsymbol{\nu}_j)^\top}{\sum_i r_{ij} + 2\alpha + m + 2}.$$

Sensible defaults: $\alpha = m/2$, $\boldsymbol{\Psi} = \frac{1}{2}\boldsymbol{I}$ (or $\frac{1}{2}\operatorname{cov}(\boldsymbol{x})$), $\gamma = 1$ (uniform Dirichlet, no prior on weights), $\eta = 0$ (no mean prior), $\boldsymbol{\nu}_j = \operatorname{mean}(\boldsymbol{x})$.

**Deriving the MAP weight update**

Maximise $\log p(\boldsymbol{x} \mid \theta) + \log p(\boldsymbol{w})$ subject to $\sum_j w_j = 1$ using a Lagrange multiplier $\lambda$:

$$\frac{\partial}{\partial w_j}\Big[\textstyle\sum_j \sum_i r_{ij} \log w_j + (\gamma - 1)\log w_j + \lambda(1 - \sum_j w_j)\Big] = 0 \;\Rightarrow\; w_j = \frac{\sum_i r_{ij} + \gamma - 1}{\lambda}.$$

Summing over $j$ and using $\sum_j w_j = 1$ gives $\lambda = n + l(\gamma - 1)$.

### 7.1.3 Mixture of Poissons

When data are **non-negative integer counts** the Gaussian assumption is inappropriate. A **mixture of Poissons** uses Poisson component distributions:

$$p(x) = \sum_{j=1}^{l} w_j\, \mathcal{P}(x; \lambda_j), \qquad \mathcal{P}(x; \lambda_j) = \frac{\lambda_j^x\, e^{-\lambda_j}}{x!}.$$

The mean and variance of component $j$ are both $\lambda_j$, so the model can capture over-dispersed count distributions via a heterogeneous mixture.

**EM via Jensen's inequality.** The log-likelihood $\log \sum_j w_j \mathcal{P}(x_i; \lambda_j)$ contains a log of a sum, which is intractable to optimise directly. Introduce auxiliary variables $w_{ji} \geq 0$ with $\sum_j w_{ji} = 1$ and apply Jensen's inequality to the concave log:

$$\log \sum_j w_j \mathcal{P}(x_i; \lambda_j) = \log \sum_j w_{ji} \cdot \frac{w_j \mathcal{P}(x_i; \lambda_j)}{w_{ji}} \;\geq\; \sum_j w_{ji} \log \frac{w_j \mathcal{P}(x_i; \lambda_j)}{w_{ji}}.$$

The bound is tight when $w_{ji} \propto w_j \mathcal{P}(x_i; \lambda_j)$, which defines the E-step.

**E-step.** Set

$$w_{ji}^{\text{new}} = \frac{w_j^{\text{old}}\, \mathcal{P}(x_i; \lambda_j^{\text{old}})}{p(x_i; \boldsymbol{w}^{\text{old}}, \boldsymbol{\lambda}^{\text{old}})}.$$

**M-step.** Maximise the lower bound over $\boldsymbol{w}$ and $\boldsymbol{\lambda}$:

$$w_j^{\text{new}} = \frac{1}{n} \sum_i w_{ji}, \qquad \lambda_j^{\text{new}} = \frac{\sum_i w_{ji}\, x_i}{\sum_i w_{ji}}.$$

The $\lambda_j$ update is the responsibility-weighted sample mean — exactly as expected for the sufficient statistic of the Poisson distribution.

Regularisation: a Dirichlet prior $\mathcal{D}(\boldsymbol{w} \mid \gamma)$ on the weights and a (non-informative) uniform prior on each $\lambda_j$.

### 7.1.4 Examples

**Initialization strategies for MoG..** EM is sensitive to initialisation. Three strategies are common:

- **"em"**: run several short, low-tolerance EM passes from different random starts, then continue with a full precise run from the best solution found.
- **"rnd"**: purely random initialisations; pick the run with the highest final log-likelihood.
- **"svd"**: use the singular-value decomposition of the data matrix to derive a structured, data-adaptive starting point.

On a 2D synthetic dataset with 8 elongated clusters, the "em" and "svd" strategies reliably find the correct 8-cluster partition; "rnd" is more prone to poor local optima.

**Covariance constraints..** The shape, volume, and orientation of each Gaussian component can be constrained to regularise the model or to reflect prior knowledge. Using Mclust notation, a three-letter code describes the multivariate constraint:

- **EII**: spherical, equal volume across components.
- **VII**: spherical, unequal volume.
- **EEI / VEI / EVI / VVI**: diagonal covariance (axis-aligned clusters), with equal/varying combinations of volume and shape.
- **EEE / EEV / VEV / VVV**: full ellipsoidal covariance with equal/varying volume, shape, and orientation.

Applied to a synthetic 2D dataset with 3 true clusters (banana-shaped, curved), a fully unconstrained MoG (VVV, 3 components) correctly recovers the three cluster shapes, while spherical or diagonal constraints produce boundary errors. Increasing to 6 components with tight covariance constraints causes over-splitting: each true cluster is divided into 2 spurious sub-clusters.

**Multiple Tissue dataset..** MoG is applied to the Multiple Tissue dataset using the 76 genes with largest variance, projected onto the first two principal components. With 3 components (spherical, unequal volume) the three tissue types (liver, brain, adipose) are partially separated but brain and adipose overlap. With 4 components (spherical or diagonal) the partition improves; diagonal covariance with varying volume and shape gives a qualitatively better result, tracking the elongated liver cluster more accurately. Increasing to 5, 6, or 7 spherical components over-splits the clusters, showing that model selection (choosing $l$) is crucial.

## 7.2 $k$-Means Clustering

### 7.2.1 The Method

$k$-means is closely related to a mixture of Gaussians. Impose equal mixture weights, equal spherical covariances $\boldsymbol{\Sigma}_j = \sigma^2 \boldsymbol{I}$, and use hard assignments (or take the limit $\sigma^2 \to 0$). Then

maximum likelihood reduces to minimising within-cluster squared distances:

1. equal weights $w_j = 1/l$;
2. spherical and equal covariance $\boldsymbol{\Sigma}_j = \sigma^2 \boldsymbol{I}$;
3. *hard* (discrete) cluster membership — a sample belongs to exactly one cluster.

Under these assumptions the soft responsibility $r_{ij}$ collapses to a binary indicator: $p(j \mid \boldsymbol{x}_i, \boldsymbol{\mu}_j) = 1$ if $j = \arg\min_k \|\boldsymbol{x}_i - \boldsymbol{\mu}_k\|$, zero otherwise. The only free parameters are the $l$ cluster centres $\boldsymbol{\mu}_j$.

**$k$-means algorithm**

**Initialise**: choose $l$ cluster centres $\boldsymbol{\mu}_j$.
**Iterate** until convergence:

1. *Assignment step*: assign each $\boldsymbol{x}_i$ to the nearest centre, $c_{\boldsymbol{x}_i} = \arg\min_k \|\boldsymbol{x}_i - \boldsymbol{\mu}_k\|$.
2. *Update step*: recompute each centre as the mean of its assigned points, $\boldsymbol{\mu}_j^{\text{new}} = \frac{1}{n_j} \sum_{i:\, c_{\boldsymbol{x}_i} = j} \boldsymbol{x}_i$, where $n_j = \#\{i : c_{\boldsymbol{x}_i} = j\}$.

**$k$-means *is* EM, with the soft edges sanded off**

The two steps of $k$-means are not a coincidence — they are the E-step and the M-step of a Gaussian mixture, simplified. *Assignment step* = E-step: work out which component each point belongs to. In MoG this yields a soft responsibility spread over all components; force spherical equal covariances and hard membership and it collapses to "pick the nearest centre". *Update step* = M-step: re-estimate the parameters from those assignments. In MoG that is the responsibility-weighted mean; with hard 0/1 memberships the weighting disappears and it becomes the plain average of the assigned points.

So $k$-means inherits EM's guarantee (the objective never increases, convergence to a *local* optimum) and EM's weakness (that local optimum depends on where you started). This is also why running $k$-means several times from different initialisations is not a hack but a necessity.

**Properties**: $k$-means is fast, but it is *not* robust to outliers: because a centre is an arithmetic mean, one extreme point can move it substantially. Its squared-Euclidean objective also favours compact, roughly spherical clusters and it is sensitive to initialisation and feature scaling. A centre placed near outliers can remain stuck there, failing to model the true cluster distribution. Note that $k$ itself is *never* learned — it is a hyperparameter you choose in advance, and the algorithm will cheerfully carve your data into whatever number of clusters you asked for.

**Fuzzy $k$-means** replaces hard membership with a soft membership $u_{ij}$ controlled by a fuzzifier $b > 1$:

$$u_{ij} = \frac{\|\boldsymbol{x}_i - \boldsymbol{\mu}_j\|^{-2/(b-1)}}{\sum_{k=1}^{l} \|\boldsymbol{x}_i - \boldsymbol{\mu}_k\|^{-2/(b-1)}}.$$

The centre update becomes the responsibility-weighted mean:

$$\boldsymbol{\mu}_j^{\text{new}} = \frac{\sum_i u_{ij}^b \, \boldsymbol{x}_i}{\sum_i u_{ij}^b},$$

and the minimised objective is $\sum_j \sum_i u_{ij}^b \, \|\boldsymbol{x}_i - \boldsymbol{\mu}_j\|^2$. As $b \to 1$ this recovers hard $k$-means; as $b \to \infty$ all memberships converge to $1/l$ (fully fuzzy).

### 7.2.2 Examples

**Synthetic 2D data (5 clusters)..** On a synthetic dataset with 5 Gaussian clusters, $k$-means with $l = 5$ recovers the true partition (cluster centres at the true means) when initialised well. Local minima arise frequently: one failure mode is that one model centre explains two true clusters while a second true cluster is divided between two model centres; another is that three model centres all compete for the same true cluster while one true cluster is unmodelled. These failures highlight why multiple random restarts are necessary in practice.

**Synthetic 2D data ($k = 8$)..** With $l = 8$ and 5 true clusters the model must split some clusters. Different initialisations produce markedly different partitions, confirming the strong dependence on initialisation.

**Iris data..** $k$-means with $l = 3$ on the first two PCA components of the Iris dataset typically recovers two of the three species exactly, with errors only at the border between the two overlapping classes (*Iris versicolor*/*Iris virginica*). Occasionally a suboptimal initialisation merges the two overlapping clusters and splits the well-separated one — these solutions have higher objective values but cannot always be distinguished from the correct solution by the $k$-means objective alone.

**Multiple Tissue data..** $k$-means with $l = 4$ on the Multiple Tissue dataset correctly identifies the four tissue types in the most typical run. Suboptimal initialisations place a centre in the empty region between the adipose and brain clusters, producing a qualitatively different partition.

## 7.3 Hierarchical Clustering

### 7.3.1 The Method

**Hierarchical clustering** represents the cluster structure as a *dendrogram* — a tree whose leaves are individual samples and whose internal nodes encode the distance at which two sub-trees were merged (or split). Cutting the dendrogram at a chosen height yields a flat partition; varying the cut gives a nested family of solutions at every granularity.

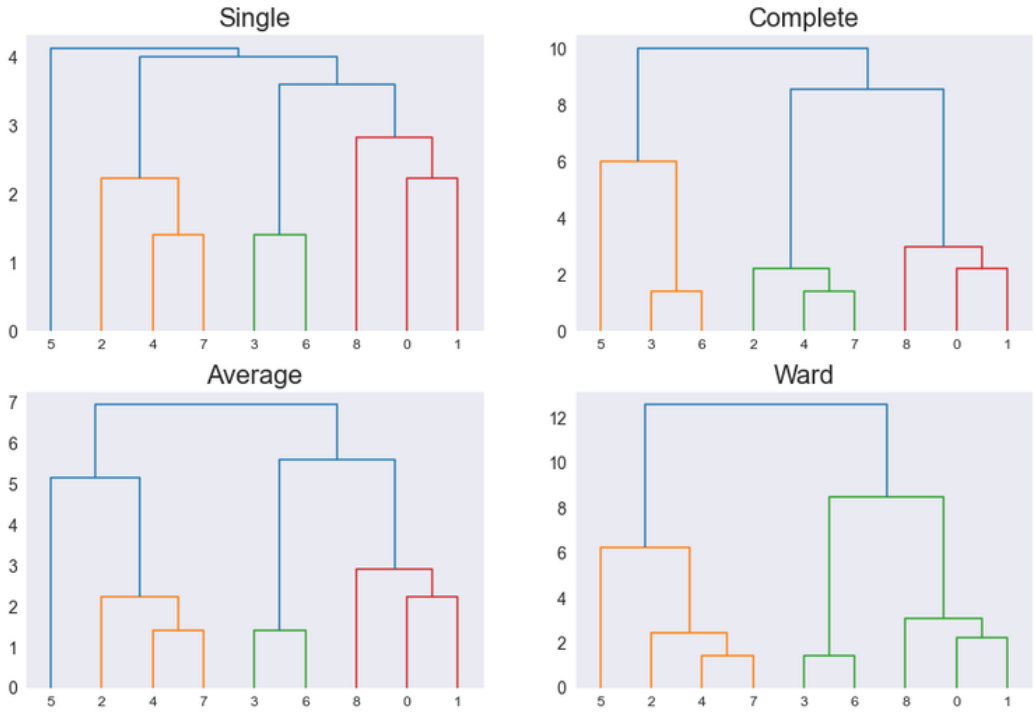


Figure 7.2: Dendrograms for the same nine objects under single, complete, average, and Ward linkage. The linkage rule changes both the merge order and the merge heights; coloured subtrees indicate the flat groups obtained at the plotting threshold.

The leaves are identical in all four panels of Fig. 7.2, yet the resulting hierarchies differ because "distance between clusters" has changed. Heights should be interpreted within a panel, not numerically compared across panels: Ward's increase in within-cluster sum of squares is on a different scale from the pairwise distances used by single, complete, and average linkage.

Two broad strategies exist:

- **Agglomerative (bottom-up)**: start with $n$ singleton clusters and greedily merge the closest pair at each step.
- **Divisive (top-down)**: start with one cluster containing all points. A common approach builds the *minimal spanning tree* of the pairwise distance graph and then removes the largest edge to split into two clusters, repeating recursively. Alternatively, remove the edge that is considerably larger than other edges within its cluster.

**Linkage criteria** define the distance between two clusters $A$ and $B$:

$$
\begin{aligned}
d_{\min}(A,B) &= \min_{\boldsymbol{a}\in A,\,\boldsymbol{b}\in B} \|\boldsymbol{a}-\boldsymbol{b}\| && \text{(single linkage)}\\
d_{\max}(A,B) &= \max_{\boldsymbol{a}\in A,\,\boldsymbol{b}\in B} \|\boldsymbol{a}-\boldsymbol{b}\| && \text{(complete linkage)}\\
d_{\text{avg}}(A,B) &= \frac{1}{n_A n_B}\sum_{\boldsymbol{a}\in A}\sum_{\boldsymbol{b}\in B}\|\boldsymbol{a}-\boldsymbol{b}\| && \text{(average linkage / UPGMA)}\\
d_{\text{mean}}(A,B) &= \|\bar{\boldsymbol{a}}-\bar{\boldsymbol{b}}\| && \text{(centroid / mean linkage)}
\end{aligned}
$$

where $n_A, n_B$ are the cluster sizes and $\bar{\boldsymbol{a}}, \bar{\boldsymbol{b}}$ are the cluster means. The element-level distance $\|\cdot\|$ can be Euclidean, Manhattan, or Mahalanobis.

**Linkage behaviour — and why single linkage chains**

**Complete linkage** avoids elongated clusters — it guarantees that the diameter (largest intra-cluster distance) stays small — but clusters may not be well-separated. **Single linkage** ensures any two points from different clusters are far apart; it is used for *leave-one-cluster-out* cross-validation where an entire new group is withheld. **Average linkage (UPGMA)** is a commonly used compromise.

The behaviour follows directly from which distance each criterion looks at. *Single* linkage judges two clusters by their **closest** pair of points, so a cluster only needs *one* point near another cluster to swallow it. Lay out a trail of stepping stones between two dense blobs and single linkage will happily walk across it, merging everything into one long straggly cluster — this is **chaining**, and it is why single linkage produces those lopsided, uninformative dendrograms in the examples below. Its saving grace is exactly the same property: it can follow genuinely elongated, non-convex shapes that centroid-based methods would slice in half.

*Complete* linkage judges by the **farthest** pair, so a merge is penalised by its worst-case diameter. It therefore refuses to build sprawling clusters and yields compact, roughly spherical blobs — but it will happily split a true elongated cluster in two, and it is sensitive to outliers (one distant point inflates the diameter and blocks an otherwise sensible merge).

**Ward's method** merges the pair of clusters whose union minimises the total within-cluster sum of squares — a variance-based criterion that tends to produce compact, spherical clusters of similar size.

### 7.3.2 Examples

**US Arrest data..** The 50 US states are clustered on four crime-rate variables. Ward's dendrogram produces two clean super-clusters (high-crime vs. low-crime states) with a clear gap, giving an unambiguous 2-cluster cut. Single linkage exhibits *chaining*: states are added one at a time to a growing chain, producing an unbalanced dendrogram with little internal structure. Complete and average linkage produce intermediate results, broadly consistent with Ward's partition. McQuitty, median, and centroid methods give similar dendrograms to average linkage on this dataset.

**Synthetic 5-cluster data..** Ward's distance produces a perfect recovery of all 5 Gaussian clusters when the dendrogram is cut at the right height. Complete and average linkage also recover the correct partition. Single linkage again suffers from chaining and produces a poor cut.

**Iris data..** With 3 clusters, Ward's and average linkage both achieve good (but not perfect) separations — a few *Iris versicolor*/*Iris virginica* points are misassigned at the class boundary. Single linkage chains through the overlapping region and fails to isolate the third species cleanly. Increasing to 5 clusters over-splits the data; the extra clusters appear at boundary points marked by misassignment arrows.

**Multiple Tissue data..** With 4 clusters, Ward's linkage correctly groups the four tissue types (liver, brain, adipose, colon/kidney mix) with only a few misassigned points. Complete and average linkage with 4 clusters produce similar quality. Single linkage with 4 clusters chains the liver samples into one huge cluster and isolates small outliers instead of identifying the biological tissue groups. Increasing to 6 clusters with Ward's or average linkage over-splits the data, creating biologically meaningless sub-clusters.

## 7.4 Similarity-Based Clustering

**Similarity-based clustering** groups objects using only pairwise similarities, without requiring an explicit feature-vector representation. Similarities can come from web links, social interactions, co-occurrence counts (co-expressed genes, co-cited papers), spatial distances, sequence alignments, or structural comparisons.

### 7.4.1 Aspect Model

The **aspect model** applies to discrete data of co-occurring pairs $(x, y)$ — for example, person $x$ buys product $y$, or word $x$ appears in document $y$. The data are counts $n(x, y)$.

**Aspect model**

Introduce a hidden class variable $z \in \{z_1, \dots, z_l\}$. Conditional on $z$, $x$ and $y$ are independent:

$$p(x, y) = \sum_z p(z)\, p(x \mid z)\, p(y \mid z).$$

The hidden factor $z$ acts as the common cause of both $x$ and $y$, explaining their correlation without direct dependence. *Example*: hot-chocolate consumption and crime rate are both driven by cold weather $(z)$, so $x \perp y \mid z$.

**EM for the aspect model.** Let $N = \sum_{x,y} n(x, y)$ and let $N_z = \sum_{x,y} n(x, y) p(z \mid x, y)$ be the expected number of pairs assigned to latent class $z$.

- **E-step**: compute the posterior $p(z \mid x, y) = \dfrac{p(z)\, p(x \mid z)\, p(y \mid z)}{\sum_{z'} p(z')\, p(x \mid z')\, p(y \mid z')}$.
- **M-step**: update the parameters using the weighted counts $n(x, y)$:

$$\begin{aligned} p(z) &= \frac{N_z}{N}, \\ p(y \mid z) &= \frac{\sum_x n(x, y)\, p(z \mid x, y)}{N_z}, \\ p(x \mid z) &= \frac{\sum_y n(x, y)\, p(z \mid x, y)}{N_z}. \end{aligned}$$

**Clustering.** Objects $x$ are clustered via the marginal posterior $p(z \mid x) \propto p(x \mid z)\, p(z)$, where $p(x) = \sum_{z,y} p(z)\, p(x \mid z)\, p(y \mid z)$. Each value of $z$ represents one cluster; analogous formulas cluster $y$ or pairs $(x, y)$.

### 7.4.2 Affinity Propagation

#### The Method

**Affinity propagation** (AP) is a similarity-based, *exemplar-based* clustering method: cluster centres must be actual data points (*exemplars* or *prototypes*). Unlike $k$-centres clustering — which also enforces this constraint but requires a good initialisation and works only for a small number of clusters — AP determines both the exemplars and the cluster memberships simultaneously via message passing, without needing the number of clusters specified in advance.

Four quantities drive the algorithm:

- **Similarities** $s(i,k)$: how well object $k$ can serve as an exemplar for object $i$ (e.g. negative squared distance, $s(i,k) = -\|\boldsymbol{x}_i - \boldsymbol{x}_k\|^2$).
- **Preferences** $s(k,k)$: a self-similarity score expressing how suitable object $k$ is as an exemplar; a uniform value controls the tendency toward clusters. *Higher preference → more cluster centres; lower preference → fewer cluster centres.*
- **Responsibilities** $r(i,k)$: message from $i$ to candidate exemplar $k$, reflecting how strongly $k$ should be the exemplar for $i$ relative to competing candidates.
- **Availabilities** $a(i,k)$: message from candidate exemplar $k$ to object $i$, reflecting how much evidence $k$ collects from other objects that it should be an exemplar.

**Affinity Propagation update rules**

Initialise $a(i,k) = 0$. Iterate, applying damping with factor $\lambda \in [0,1)$ to improve numerical stability (e.g. $\lambda = 0.5$ by default):

$$r(i,k) = s(i,k) - \max_{k',\, k' \neq k} \{a(i,k') + s(i,k')\},$$

$$a(i,k) = \min\Big\{0,\ r(k,k) + \sum_{i',\, i' \notin \{i,k\}} \max\{0, r(i',k)\}\Big\} \quad (i \neq k),$$

$$a(k,k) = \sum_{i',\, i' \neq k} \max\{0, r(i',k)\}.$$

Object $i$ is assigned to the exemplar $k^* = \arg\max_k \{a(i,k) + r(i,k)\}$.

Each message is updated as a weighted average of the new value and the previous value: $r_{\text{new}} \leftarrow (1-\lambda)\, r_{\text{computed}} + \lambda\, r_{\text{old}}$ (same for $a$). A larger $\lambda$ makes convergence slower but more stable; a smaller $\lambda$ converges faster but may oscillate.

**Intuition behind the messages**

The responsibility $r(i,k)$ drops when a competing candidate $k'$ has high similarity and availability to $i$ — it measures how uniquely suited $k$ is to represent $i$. The self-availability $a(k,k)$ grows when many other objects send positive responsibilities to $k$, confirming $k$ as a strong exemplar. Together the two messages implement a competition: candidates that gather broad support become exemplars; others do not.

Read them as a negotiation. Responsibility is object $i$ telling candidate $k$: *"here is how much I want you as my exemplar, given the other offers on the table"*. Availability is candidate $k$ replying to $i$: *"here is how qualified I am to serve, given how many others also want me"*. Iterate the conversation until it settles, and the points everyone agrees on become the exemplars.

**Exam traps: which way do preference and damping go?**

Both of AP's knobs are easy to get backwards, and exams exploit that.

**Preference** $s(k,k)$ is how eager a point is to nominate *itself* as an exemplar. Raise it and more points put themselves forward, so you get **more clusters**; lower it and you get **fewer**. ("The larger the preference, the smaller the number of cluster centres" is the wrong direction.) This is what makes AP unusual: you never state the number of clusters directly — but you have not really escaped the problem, you have only traded $k$ for a knob that controls it less predictably.

**Damping** $\lambda$ blends each new message with the previous one to stop the updates oscillating. More damping means the messages move more cautiously, so convergence is **slower but more stable** — not faster. ("The larger the damping factor, the faster the convergence" is likewise the wrong direction.)

**Limitation.** AP itself accepts arbitrary similarities and therefore does not mathematically require spherical clusters. With the common choice of negative squared Euclidean distance and one global preference, however, it often behaves as though clusters had comparable scale. Large diffuse and small compact groups can then be merged or split poorly unless the similarity and preferences encode the differing scales.

### Examples

**Toy graph (convergence illustration)..** Starting from a fully connected graph of ∼25 nodes, the messages evolve over 6 iterations: edges redistribute, 3–4 exemplars emerge (highlighted in red), and by convergence each point points exclusively to its exemplar.

**Face images..** On a face database AP selects a small number of representative faces (exemplars, highlighted in coloured boxes); all other faces point to their nearest exemplar. Compared with $k$-centres clustering run on the same data, AP finds higher-quality exemplars for the 15 hardest-to-represent images.

**Airport clustering..** AP applied to air-travel efficiency between 456 North American airports identifies 7 hub exemplars. The assignment reflects domestic vs. international flight patterns — e.g. the Canada–USA border separates Toronto and Philadelphia hubs. The dense Seattle–Vancouver corridor causes Pacific northwest cities to be assigned to the Seattle hub.

**Synthetic 2D data: balanced clusters (x7)..** On a dataset with 6 clusters (3 large / 3 small by variance), AP correctly identifies 6 exemplars and recovers the clean partition when all clusters are roughly spherical and well-separated.

**Synthetic 2D data: mixed-size clusters (x7A, x9, d6)..** When one dense small cluster sits inside or near a large diffuse cluster, AP merges them — it cannot detect the small cluster inside the large one because it assumes equal cluster variances. On the x9 dataset (6 clusters of 2 small, 2 medium, 2 large) AP conflates one pair, producing 5 clusters instead of 6. On the 6-D d6/d6A datasets (one large + three small clusters, 100 samples, PCA visualisation), AP again assigns elements of the large cluster to smaller neighbours — the large cluster has no single exemplar that would win the competition.

**Iris data..** AP with default preference ($p = 0$) produces a good 3-cluster partition with a few misassignments at the *Iris versicolor*/*Iris virginica* boundary, similar to $k$-means and MoG.

**Multiple Tissue data..** With preference $p = 0$ AP finds 4 clusters that largely match the tissue types, with a few misassigned samples. Increasing the preference to $p = 1$ over-clusters into many singleton exemplars (every point becomes its own cluster), showing sensitivity to the preference hyperparameter.

### 7.4.3 Similarity-based Mixture Models

#### Similarity-based Mixture of Gaussians

**Similarity-based mixture models** are mixture models that operate solely on pairwise similarities $k(\boldsymbol{x}_j, \boldsymbol{x}_i)$ between the $n$ training points — no explicit feature vectors are used. This is closely related to kernel methods: the input to the algorithm is the $n \times n$ kernel matrix $K_{ij} = k(\boldsymbol{x}_j, \boldsymbol{x}_i)$.

Starting from a Gaussian mixture $p(\boldsymbol{x}) = \sum_{j=1}^{K} p(j)\, k(\boldsymbol{x}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$, introduce a scalar variance $\sigma_i^2$ per component and define

$$k(\boldsymbol{x}; \boldsymbol{\mu}_i, \sigma_i^2) = \frac{1}{\sigma_i^m}\, k(\boldsymbol{x}, \boldsymbol{\mu}_i)^{1/\sigma_i^2},$$

so the model becomes

$$p(\boldsymbol{x}) = \sum_{j=1}^{K} p(j)\, \frac{1}{\sigma_j^m}\, k(\boldsymbol{x}, \boldsymbol{\mu}_j)^{1/\sigma_j^2}.$$

The component variance $\sigma_j^2$ controls the width of cluster $j$, allowing the model to handle clusters of different sizes — a key advantage over affinity propagation.

**Connection to standard MoG**

If $k(\boldsymbol{x}, \boldsymbol{\mu}_i) = \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_i, \boldsymbol{I})$ and $\sigma_i^2 = 1$ for all $i$, this reduces to a standard spherical MoG. The similarity-based formulation generalises this to arbitrary positive semi-definite kernels, and expresses all update rules solely in terms of the given similarities $k(\boldsymbol{x}_k, \boldsymbol{x}_j)$.

#### Representing the Centers

Setting $\alpha_i = p(i)$ and $\hat{\alpha}_{ik} = p(i \mid \boldsymbol{x}_k, \boldsymbol{\alpha})$, the weights satisfy

$$\alpha_i = \frac{1}{n} \sum_{k=1}^{n} \hat{\alpha}_{ik}.$$

The EM centre update requires evaluating $\|\boldsymbol{x}_l - \boldsymbol{\mu}_i\|^2$. Assuming the Gaussian kernel $k(\boldsymbol{x}_k, \boldsymbol{x}_j) = \exp(-\frac{1}{2}\|\boldsymbol{x}_k - \boldsymbol{x}_j\|^2)$, so that $\|\boldsymbol{x}_k - \boldsymbol{x}_j\|^2 = -2 \log k(\boldsymbol{x}_k, \boldsymbol{x}_j)$, one can show:

$$\|\boldsymbol{x}_l - \boldsymbol{\mu}_i\|^2 = \frac{-2 \sum_k \hat{\alpha}_{ik} \log k(\boldsymbol{x}_k, \boldsymbol{x}_l)}{\sum_k \hat{\alpha}_{ik}} + \frac{\sum_{k,j} \hat{\alpha}_{ik} \hat{\alpha}_{ij} \log k(\boldsymbol{x}_k, \boldsymbol{x}_j)}{\left(\sum_k \hat{\alpha}_{ik}\right)^2}.$$

This expresses the squared distance to the (virtual) centre entirely through pairwise similarities — no explicit $\boldsymbol{\mu}_i$ needs to be stored or updated. The updated kernel value is $k(\boldsymbol{x}_i, \boldsymbol{\mu}_i)^{\text{new}} = \exp(-\frac{1}{2}\|\boldsymbol{x}_i - \boldsymbol{\mu}_i\|^2)$.

The full covariance $\boldsymbol{\Sigma}_i$ could also be expressed via similarities, but this requires inverting an $n \times n$ matrix, which is computationally expensive and numerically unstable; only the scalar variance $\sigma_i^2$ is therefore estimated.

#### Update Rules

Define $\hat{\alpha}_i = \frac{1}{n} \sum_k \hat{\alpha}_{ik}$ and $\gamma_s = \sum_i \gamma_i$. The complete EM update cycle is:

1. **E-step**: $\hat{\alpha}_{ik} = \dfrac{\alpha_i^{\text{old}}\, k(\boldsymbol{x}_k; \boldsymbol{\mu}_i, \sigma_i^2)}{p(\boldsymbol{x}_k)}, \quad p(\boldsymbol{x}_k) = \sum_i \alpha_i^{\text{old}}\, k(\boldsymbol{x}_k; \boldsymbol{\mu}_i, \sigma_i^2).$

2. **Weight update**: $\alpha_i^{\text{new}} = \dfrac{n\hat{\alpha}_i + \gamma_i - 1}{n + \gamma_s - K}$.

3. **Variance update**: $(\sigma_i^2)^{\text{new}} = \dfrac{\sum_k \hat{\alpha}_{ik}(-2\log k(\boldsymbol{x}_k, \boldsymbol{\mu}_i)) + wS}{n\hat{\alpha}_i + w}$.

4. **Kernel update**: compute $-2\log k(\boldsymbol{x}_l, \boldsymbol{\mu}_i) = \|\boldsymbol{x}_l - \boldsymbol{\mu}_i\|^2$ from the similarity formula above, then set $k(\boldsymbol{x}_l, \boldsymbol{\mu}_i)^{\text{new}} = \exp\!\left(-\frac{1}{2}\|\boldsymbol{x}_l - \boldsymbol{\mu}_i\|^2\right)$.

Hyperparameters: $\gamma_i$ from a Dirichlet prior on $\boldsymbol{\alpha}$; $w$ and $S$ from a Wishart prior on the precisions $1/\sigma_i^2$.

### Examples

**x7 data (6 balanced clusters)..** Both affinity propagation and EMcluster (similarity-based MoG EM) correctly recover all 6 clusters. The $\alpha_i$ and $\sigma_i^2$ traces over 200 EM iterations show rapid convergence: all $\alpha_i$ converge to $\approx 1/6$ and $\sigma_i^2$ differentiates the two cluster sizes (small clusters settle to lower variance).

**x7A data (overlapping/mixed-size clusters)..** AP merges one large and one small cluster; EMcluster correctly separates all 6.

**x9 data (6 clusters: 2 small / 2 medium / 2 large)..** AP merges the two closest clusters; EMcluster identifies all 6 correctly.

**d6/d6A data (1 large + 3 small, 6-D)..** AP cannot detect the large diffuse cluster and assigns its elements to smaller neighbours. EMcluster correctly identifies the large cluster by fitting a large $\sigma^2$ to it, while the small clusters get small $\sigma^2$ values.

The key advantage of the similarity-based MoG over AP is the per-component variance parameter $\sigma_i^2$: because AP uses raw similarities without adapting cluster width, it implicitly assumes equal-size spherical clusters and fails when cluster sizes differ substantially.

# 8

# Biclustering

Ordinary clustering forces a bad assumption on gene-expression data: that a gene behaves the same way across *all* samples. Biology does not work like that. A group of genes might be co-regulated only in tumour samples, and behave completely unremarkably everywhere else. Cluster over all the samples at once and that signal is diluted into nothing.

Biclustering drops the assumption. Instead of grouping rows only, it looks for a *subset of rows that behave coherently across a subset of columns* — a rectangular block of the matrix, found by clustering rows and columns *simultaneously*. The biological reading is compelling: a bicluster is a set of genes (a pathway or regulatory module) that switch on together, but only in the particular conditions or patients where that pathway is active.

Two structural consequences separate this from ordinary clustering, and both are exam favourites. A row or column may belong to **several biclusters at once** (a gene can participate in more than one pathway), and it may belong to **none at all** (most genes are irrelevant to any given module). Ordinary clustering, by contrast, hands every point exactly one cluster.

## 8.1 Types of Biclusters

Biclustering simultaneously clusters both the rows and the columns of a data matrix. A *bicluster* is a pair $(I, J)$ where $I \subseteq \{1, \ldots, n\}$ is a subset of rows (e.g. genes) and $J \subseteq \{1, \ldots, m\}$ is a subset of columns (e.g. samples), such that the entries $X_{ij}$ for $i \in I, j \in J$ exhibit a coherent pattern. Unlike standard clustering, rows and columns can belong to multiple biclusters simultaneously, or to none at all.

The literature distinguishes six types of biclusters ordered by increasing structural complexity:

**Types of Biclusters**

Let $a_{ij}$ denote the entry in row $i$, column $j$ of the bicluster sub-matrix.

- **Constant values:** $a_{ij} = c$ for all $i, j$ — all entries identical.
- **Constant rows:** $a_{ij} = r_i$ — each row has a fixed value across all bicluster columns.
- **Constant columns:** $a_{ij} = c_j$ — each column has a fixed value across all bicluster rows.
- **Additive coherent values:** $a_{ij} = \mu + r_i + c_j$ — row and column offsets are additive; rows are shifted versions of each other.
- **Multiplicative coherent values:** $a_{ij} = \mu \cdot r_i \cdot c_j$ — rows are scalar multiples of each other; columns are scalar multiples of each other.

- **General coherent values:** rows (or columns) are related by a more general function; includes the above as special cases.

**Intuition & Mental Model**

An additive coherent bicluster with a $5 \times 5$ sub-matrix can be visualised as: each entry equals a row offset $r_i$ plus a column offset $c_j$ plus a global mean $\mu$. Subtracting row means from each row flattens all rows to the same profile, confirming the additive structure. Multiplicative coherence means all rows are proportional — dividing each row by its mean yields identical rows.

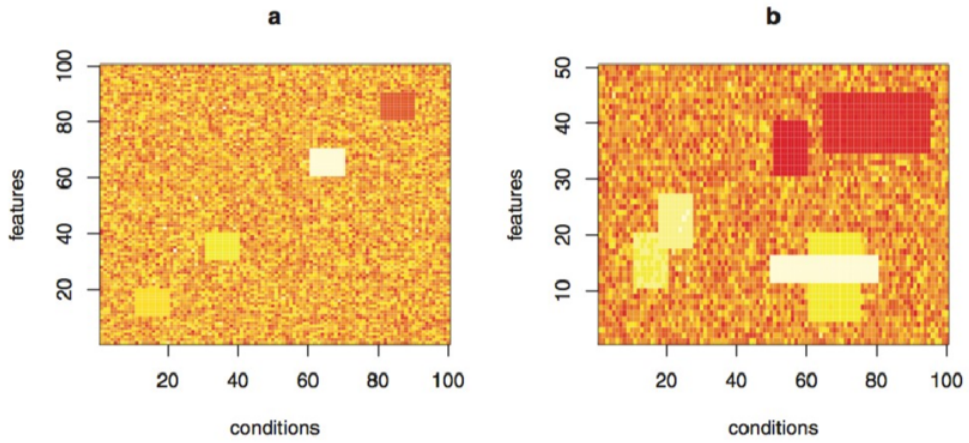


Figure 8.1: Synthetic feature-by-condition matrices with local rectangular biclusters. Panel (a) contains isolated blocks against a noisy background; panel (b) shows a more complex arrangement in which active feature and condition subsets can overlap.

Unlike an ordinary row clustering, Fig. 8.1 does not assign an entire feature to one global group. A feature participates only over the selected conditions, and a row or column may take part in multiple local blocks or in none. This local, non-exclusive membership is the defining visual signature of biclustering.

## 8.2 Overview of Biclustering Methods

Biclustering algorithms fall into four broad categories:

**Variance minimization.** These methods search for sub-matrices with low internal variance. *Cheng–Church $\delta$-biclusters* define the mean squared residue of a bicluster as

$$H(I, J) = \frac{1}{|I||J|} \sum_{i \in I, j \in J} \left(a_{ij} - a_{iJ} - a_{Ij} + a_{IJ}\right)^2,$$

and accept any bicluster with $H(I, J) \leq \delta$. Related methods include $\delta$-cluster and $\delta$-pClusters.

**Two-way clustering.** Iterative co-clustering procedures alternate between clustering rows and columns. Representative methods: CTWC (Coupled Two-Way Clustering), ITWC (Iterative Two-Way Clustering), and DCC (Double Conjugated Clustering).

**Motif and pattern recognition.** Search for biclusters defined by expression patterns or ordinal relationships. Methods include xMOTIF (expression motif), OPSM (order-preserving sub-matrices), SPEC (spectral biclustering), and ISA (Iterative Signature Algorithm).

**Probabilistic and generative models.** Fit a probabilistic model to the data and infer biclusters from the model parameters. Methods include SAMBA (probabilistic bipartite graph), PRMs, ProBic, cMonkey, Plaid model, BBC (Bayesian BiClustering), and **FABIA** (Factor Analysis for Bicluster Acquisition).

## 8.3 FABIA Biclustering

FABIA (Factor Analysis for Bicluster Acquisition) represents each bicluster as an *outer product* of two sparse vectors.

**FABIA Bicluster Representation**

A bicluster is the rank-1 matrix $\mathbf{y}_j \mathbf{u}_j^T$ where

- $\mathbf{u}_j \in \mathbb{R}^m$ is a sparse *prototype* (loading) vector — its non-zero entries indicate which features (genes) belong to bicluster $j$.
- $\mathbf{y}_j \in \mathbb{R}^n$ is a sparse *factor* vector — its non-zero entries indicate which samples belong to bicluster $j$.

The full data matrix is modelled as

$$X = \sum_{j=1}^{l} \mathbf{y}_j \mathbf{u}_j^T + \Gamma = Y U^T + \Gamma,$$

where $X, \Gamma \in \mathbb{R}^{n \times m}$, $U \in \mathbb{R}^{m \times l}$, $Y \in \mathbb{R}^{n \times l}$, and $\Gamma$ is noise. Per sample: $\mathbf{x}_i = U \tilde{\mathbf{y}}_i + \boldsymbol{\varepsilon}_i$.

### FABIA as Factor Analysis

The per-sample model $\mathbf{x} = U\tilde{\mathbf{y}} + \boldsymbol{\varepsilon}$ is a standard factor analysis (FA) model:

- $U \in \mathbb{R}^{m \times l}$ is the loading matrix (factor loadings).
- $\tilde{\mathbf{y}} \sim \mathcal{N}(\mathbf{0}, I)$ in standard FA.
- $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \Psi)$ with $\Psi$ diagonal (independent noise per feature).
- $U$ explains the *common variance*; $\Psi$ explains the *independent variance*.

### Sparseness via Laplace Priors

Standard FA does not produce sparse solutions. FABIA enforces sparseness by replacing the Gaussian prior on factors with a **Laplace prior**:

**FABIA Laplace Priors**

$$p(\tilde{\mathbf{y}}) = \left(\tfrac{1}{\sqrt{2}}\right)^{l} \prod_{j=1}^{l} \exp(-\sqrt{2}\,|\tilde{y}_j|),$$

$$p(\mathbf{u}_j) = \left(\tfrac{1}{\sqrt{2}}\right)^{m} \prod_{k=1}^{m} \exp(-\sqrt{2}\,|u_{kj}|).$$

These are Laplace (double-exponential) distributions, which have a sharp peak at zero, promoting sparse solutions.

Because the posterior under Laplace priors is intractable, FABIA uses **variational EM**: a variational approximation to the posterior is maximised in the E-step, and the model parameters are updated in the M-step.

**Intuition & Mental Model**

The Laplace prior is a sparsity-inducing prior — analogous to the $\ell_1$ penalty in LASSO. Factors and loadings are driven towards zero unless there is strong evidence in the data, so only the samples and features that genuinely participate in a bicluster receive large values. The resulting sparse outer products $\mathbf{y}_j \mathbf{u}_j^T$ cleanly delimit rectangular bicluster regions in the data matrix.

## 8.4 Examples

**Synthetic data.** On a $1000 \times 100$ dataset with 13 planted biclusters, FABIA recovers the correct bicluster structure: the absolute factor matrix $|Y|$ (samples $\times$ biclusters) and the absolute loading matrix $|U|$ (features $\times$ biclusters) both show clear block structure aligned with the planted biclusters. The reconstructed data $\hat{X} = YU^T$ visually reproduces the block pattern in the original matrix.

**Super-Gaussian 50-dimensional data.** FABIA recovers latent structure in simulated super-Gaussian data with 50 features, demonstrating robustness beyond the Gaussian assumption of standard FA.

**Iris dataset.** FABIA identifies two informative biclusters:

- *Bicluster 2* — petal features (petal length, petal width); separates *Iris setosa* from the other two species.
- *Bicluster 3* — sepal features (sepal length, sepal width); provides further separation among *Iris versicolor* and *Iris virginica*.

A biplot visualises both features and samples in the same 2D space defined by the two biclusters.

**Multiple Tissue dataset.** FABIA discovers tissue-specific expression signatures, with quality improving with more iterations:

- *500 iterations* — bicluster 2 separates prostate (green); bicluster 3 separates colon (orange); a fourth bicluster separates lung (blue).
- *2000 iterations* — bicluster 4 clearly separates prostate; bicluster 1 separates colon; biclusters 3 and 2 jointly help separate lung (blue) and breast (red). More iterations produce sparser and more clearly separated solutions.

Because the FABIA solution is sparse, the top genes of a bicluster are directly interpretable. For the prostate bicluster the most relevant genes are KLK3, ACPP, KLK2, CUTL2, and RNF41 — all strongly associated with prostate tissue: KLK3 is the *prostate-specific antigen* (PSA), ACPP is secreted by the epithelial cells of the prostate gland, and KLK2 cleaves pro-PSA into its active form. The biplot (biplots of bicluster 1 vs. 2 and of bicluster 1 vs. 3) confirms this: large circles mark the features (genes) that drive the bicluster; sparseness pushes most other features to zero, collapsing them onto the axes, so the signal is clean.

**Breast cancer dataset.** Both PCA and FABIA biclustering separate the blue subclass clearly, but both have difficulty separating the red subclass. The biplot of biclusters 2 and 3 shows the three subclasses (cyan, green, blue) well separated with distinctive marker genes (PARBL, COL14A1, CRYAB, MIA visible in the biplot).

**DLBCL dataset.** PCA separates the blue subclass; FABIA separates subclasses better because the red cluster is more clearly isolated in bicluster space than in principal component space. The biplot of biclusters 1 and 2 shows good subclass separation with NME1 as a prominent marker gene.

# 9

# Hidden Markov Models

Everything so far treated data points as an unordered bag: shuffle the rows of $\boldsymbol{X}$ and PCA, ICA, and $k$-means give exactly the same answer. But a DNA strand is not a bag of nucleotides and a sentence is not a bag of words — *order carries the meaning.* Hidden Markov models are this course's answer to sequential data.

The idea is that behind the sequence you observe, there is a second sequence you *cannot* observe. A stretch of genome is visibly just letters (A, C, G, T), but each position is secretly in some state — "this is inside an exon", "this is inside an intron" — and that hidden state is what decides which letters tend to appear. You see the emissions; you want the states.

Two assumptions make this tractable. The **Markov assumption** says the next hidden state depends only on the current one, not on the whole history. The **emission assumption** says the observed symbol depends only on the current hidden state. Both are obviously false in the strict sense, and both are useful enough to have carried an enormous amount of bioinformatics — Pfam, HMMER, gene finders. They also come at a price, and the price is the theme that closes this chapter: an HMM has *no way to remember something that happened long ago*, a limitation that will lead us to memory-augmented variants and, eventually, to the same problem LSTM was invented to solve.

Three computational questions organise everything below:

- **How likely is this sequence?** → the *forward* algorithm.
- **What hidden path most likely produced it?** → *Viterbi.*
- **How do I learn the probabilities in the first place?** → *Baum–Welch*, which is just EM wearing a sequence costume.

## 9.1 Hidden Markov Models in Bioinformatics

Hidden Markov models are the most prominent *generative model* in bioinformatics. The generative perspective means that the model defines a probability distribution over output sequences; new sequences can be scored by their likelihood and sequences can be generated that are statistically similar to the training sequences. HMMs are well suited to protein and DNA sequences, which are discrete.

**Gene prediction.** HMMs identify exons and introns along a genomic sequence. The tool GENSCAN achieves a base-pair specificity of 50%–80%.

**Profile HMMs.** A profile HMM stores a *multiple sequence alignment* in a model; new sequences are scored by their likelihood under the profile. When building a profile HMM from unaligned

sequences, training can get stuck in local maxima. In practice, profile HMMs are usually initialised from a multiple sequence alignment; annealing-style optimisation is one possible strategy when alignment and model estimation are coupled. Standard software packages: HMMER and SAM. The PFAM protein family database is built entirely on profile HMMs.

Profile HMMs have inherent limitations:

- Cannot discover long-range dependencies.
- Classical profile HMMs use a discrete residue alphabet. HMMs in general can use continuous emission densities, so this is a profile-design choice, not a limit of the HMM formalism.
- Cannot detect higher-order correlations between positions.
- Cannot use a negative training set (only model what sequences look like, not what they are not).

The first of these is the deep one, and Sean Eddy — the author of HMMER — put it most memorably: an HMM's state path

> *"has no way of remembering what a distant state generated".*

Everything the model knows about the past is squeezed into *which state it is in right now.* Keep this complaint in mind: it is the thread we pick up again in Section 9.7, where it turns out to be the same vanishing-signal problem that LSTM was designed to solve.

**Other applications.** Remote homology detection, sequence scoring, HMMs combined with phylogenetic trees, and family databases (PFAM).

## 9.2 Hidden Markov Model Basics

An HMM is a graph of connected hidden states. The hidden state at time $t$ (sequence position $t$) is $u_t \in \{1, \ldots, S\}$. Each state produces an observation according to a probabilistic output distribution. The model evolves by jumping to another state or staying in the current state at each step. The set of all possible state sequences can be visualised as a trellis: $S$ rows (states) and $T$ columns (time steps), with each left-to-right path corresponding to one possible hidden sequence.

**HMM Parameters**

- **Start probability:** $p_S(u_1)$ — probability of being in state $u_1$ at position 1.
- **Transition probability:** $p(u_t \mid u_{t-1})$ — probability of transitioning from state $u_{t-1}$ to state $u_t$.
- **Emission probability:** $p(x_t \mid u_t)$ — probability of emitting observation $x_t$ from state $u_t$.

**Markov assumption:** the next state depends only on the current state (first-order). Higher-order HMMs condition on more history, e.g. second-order: $p(u_t \mid u_{t-1}, u_{t-2})$.

The probability of a hidden state sequence $u^T = (u_1, u_2, \ldots, u_T)$ of length $T$ is:

$$p(u^T) = p_S(u_1) \prod_{t=2}^{T} p(u_t \mid u_{t-1}). \tag{9.1}$$

The joint probability of a state sequence and an observation sequence $x^T = (x_1, \ldots, x_T)$ is:

$$p(x^T, u^T) = p_S(u_1)\, p(x_1 \mid u_1) \prod_{t=2}^{T} p(u_t \mid u_{t-1})\, p(x_t \mid u_t). \tag{9.2}$$

**Intuition & Mental Model**

A simple two-state HMM has states *on* ($u = 1$) and *off* ($u = 0$) with self-loops (staying in the current state) and cross-transitions. Each state emits a characteristic output distribution. The trellis picture makes clear that computing $p(x^T)$ by summing over all $S^T$ paths is exponential — the forward algorithm provides an efficient dynamic-programming solution.

## 9.3 Expectation Maximization for HMM: Baum-Welch Algorithm

Training an HMM means choosing the start, transition, and emission probabilities so that the training sequences come out likely. We could do this by gradient ascent, but the model has an annoying feature for gradients: every probability table has to sum to one, and the hidden state is never observed. Expectation Maximization handles both cleanly. The HMM-specific version is the *Baum-Welch algorithm*, and it needs no learning rate — each iteration gives a closed-form parameter update that is guaranteed not to decrease the likelihood.

The idea is the same soft-counting story as EM everywhere: if we *knew* the hidden path for each sequence, training would be trivial bookkeeping — count how often we started in each state, how often each state emitted each symbol, how often each transition was taken, and normalise. We don't know the path, so in the E-step we compute the posterior distribution over hidden states given the data and use *expected* (soft) counts instead. The M-step then normalises those counts. Two dynamic-programming sweeps make the E-step efficient: a forward pass and a backward pass.

### Computing the likelihood: the forward pass

The likelihood of an observed sequence is the sum over all hidden paths,

$$p(x^T) = \sum_{u^T} p(x^T, u^T),$$

and there are $S^T$ such paths — hopelessly many to enumerate. The Markov property rescues us. Define the *forward variable*

$$\alpha_t(a) \;:=\; p(x^t, u_t = a),$$

the probability of having emitted the first $t$ symbols *and* sitting in state $a$ right now. It obeys a one-step recursion:

$$\begin{aligned} \alpha_1(a) &= p_S(a)\, p_E(x_1 \mid a), \\ \alpha_t(a) &= p_E(x_t \mid a) \sum_{b=1}^{S} p(u_t = a \mid u_{t-1} = b)\, \alpha_{t-1}(b). \end{aligned}$$

At the end, summing out the last state gives the likelihood, $p(x^T) = \sum_a \alpha_T(a)$. Each step costs a sum over $S$ predecessor states, repeated for $S$ current states and $T$ time steps, so the whole pass is $O(TS^2)$ instead of $O(S^T)$.

**Intuition & Mental Model**

Why does summing over $S^T$ paths collapse to $O(TS^2)$? Because the future only cares *which* state you are in now, not *how* you got there. So every path that arrives at state $a$ at time $t$ can be merged into a single running number $\alpha_t(a)$ and carried forward together. That merging is the Markov assumption cashing out — and it is exactly the same trick dynamic programming uses to align sequences.

### The backward pass

The forward variable looks at the past. For the E-step we also need the future. Define the *backward variable*

$$\beta_t(a) \;:=\; p(x_{t+1\leftarrow T} \mid u_t = a),$$

the probability of emitting the *remaining* symbols $x_{t+1}, \ldots, x_T$ given that we are in state $a$ at time $t$. It runs from the end backwards:

$$\begin{aligned} \beta_T(a) &= 1, \\ \beta_t(a) &= \sum_{b=1}^{S} p_E(x_{t+1} \mid b)\, p(u_{t+1} = b \mid u_t = a)\, \beta_{t+1}(b). \end{aligned}$$

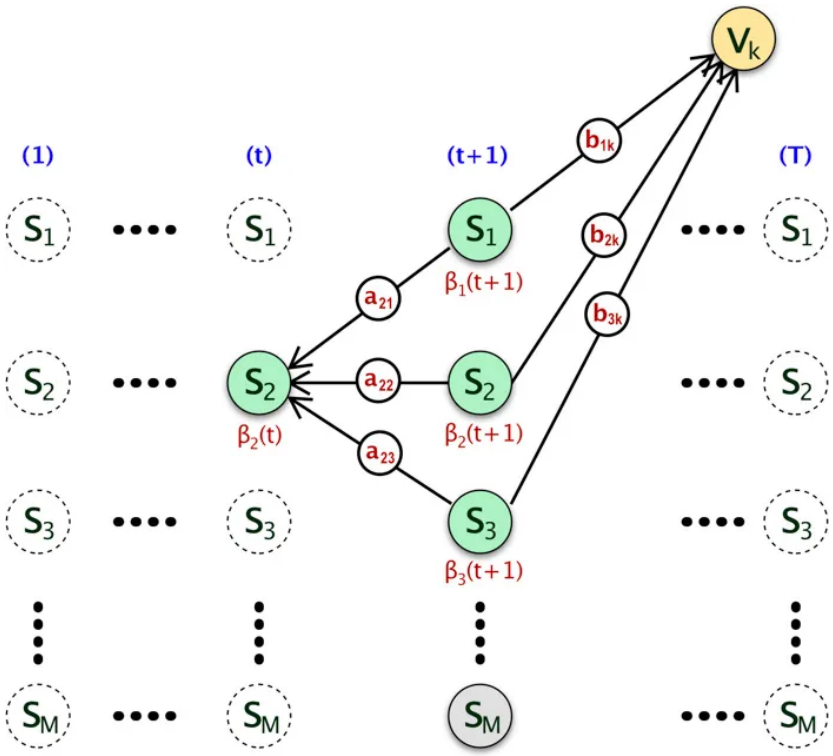


Figure 9.1: One step of the HMM backward recursion. From the current state $S_2$ at time $t$, each possible successor $S_j$ contributes its transition probability $a_{2j}$, the probability $b_{jk}$ of emitting the next observed symbol $V_k$, and its future message $\beta_j(t+1)$.

Thus Fig. 9.1 expands the recurrence as $\beta_2(t) = \sum_j a_{2j} b_{jk} \beta_j(t+1)$. The arrows are drawn backwards because the already-computed information at time $t+1$ is being accumulated into the message for time $t$; the underlying Markov transitions themselves still run forward from $t$ to $t+1$.

### E-step: posterior state and transition probabilities

With both sweeps in hand, the posteriors fall out by multiplying "evidence from the past" by "evidence from the future" and renormalising. The probability of being in state $a$ at time $t$ is

$$\gamma_t(a) \;\coloneqq\; p(u_t = a \mid x^T) = \frac{\alpha_t(a)\,\beta_t(a)}{p(x^T)},$$

and the probability of taking the transition $b \to a$ at time $t$ is

$$\xi_t(a,b) \;\coloneqq\; p(u_t = a, u_{t-1} = b \mid x^T) = \frac{\alpha_{t-1}(b)\,p(u_t = a \mid u_{t-1} = b)\,p_E(x_t \mid a)\,\beta_t(a)}{p(x^T)}.$$

These are consistent: summing a transition over its source gives the state posterior, $\gamma_t(a) = \sum_b \xi_t(a,b)$.

### M-step: re-estimate the parameters

The M-step is just "normalise the expected counts". Maximising the expected complete-data log-likelihood under the sum-to-one constraints (a one-line Lagrange multiplier argument) gives the rule $w_a = (\sum_t c_{ta})/(\sum_{a'}\sum_t c_{ta'})$ for every probability table — i.e. each parameter becomes its share of the soft counts:

**Baum-Welch re-estimation (M-step)**

$$p_S(a) = \gamma_1(a), \qquad p_E(x \mid a) = \frac{\sum_{t=1}^{T} \mathbf{1}\{x_t = x\}\,\gamma_t(a)}{\sum_{t=1}^{T}\gamma_t(a)}, \qquad p(a \mid b) = \frac{\sum_{t=2}^{T}\xi_t(a,b)}{\sum_{t=2}^{T}\gamma_{t-1}(b)}.$$

In words: the new start probability is how often we expect to start in $a$; the new emission probability is the fraction of $a$'s "time" spent emitting $x$; the new transition probability is the share of departures from $b$ that go to $a$.

**Intuition & Mental Model**

The whole algorithm is "counting, but soft". If the hidden path were known you would tally integer counts and divide. Because it is hidden, you tally *expected* counts $\gamma$ and $\xi$ — fractional visits and fractional transitions — and divide those instead. Alternate E and M and the likelihood climbs monotonically to a local maximum.

**Several sequences, and what the likelihood means**

With a whole training set $\{x_i^T\}$, run the forward and backward passes per sequence and *accumulate* the numerators and denominators across all sequences *before* normalising once at the end of the M-step. Also note that for discrete HMMs the likelihood and the probability $p(x^T)$ are the same object — there is no density-vs-probability subtlety here.

## 9.4 Viterbi Algorithm

The forward pass answers "how probable is this observation?" by summing over *all* hidden paths. Often we want a different answer: the single *most likely* hidden path that explains the observation,

$$(u^T)^* = \arg\max_{u^T} p(u^T \mid x^T) = \arg\max_{u^T} p(u^T, x^T).$$

This matters whenever the hidden states carry meaning — labelling a genomic region as exon or intron, or aligning a new sequence against a profile stored in an HMM. The best path *is* that alignment, which is why, like alignment, it is found by dynamic programming.

The Viterbi algorithm is the forward pass with two changes: replace the sum by a maximum, and remember where each maximum came from. Let $V_{t,a}$ be the probability of the best path of length $t$ that ends in state $a$:

$$V_{1,a} = p_S(a)\, p_E(x_1 \mid a),$$
$$V_{t,a} = p_E(x_t \mid a) \max_b \; p(u_t = a \mid u_{t-1} = b)\, V_{t-1,b},$$

with the best total probability $\max_{u^T} p(u^T, x^T) = \max_a V_{T,a}$. To recover the path itself, store at each cell the predecessor that achieved the maximum,

$$b(t, a) = \arg\max_b \; p(u_t = a \mid u_{t-1} = b)\, V_{t-1,b},$$

then back-trace from $\arg\max_a V_{T,a}$ to the start. Like the forward pass, the cost is $O(TS^2)$.

**Intuition & Mental Model**

Forward and Viterbi share one grid; only the operator differs. Sum over predecessors ⇒ "how probable overall" (marginal likelihood). Max over predecessors ⇒ "what is the single best story" (most likely path). Swap $\sum$ for max, keep a back-pointer, and the forward algorithm becomes Viterbi.

Viterbi also gives a cheap way to refine a multiple alignment: (1) initialise the HMM, (2) align every sequence to it with Viterbi, (3) make per-column frequency counts and re-estimate the transition and emission probabilities, (4) repeat until convergence. This "Viterbi training" is a hard-assignment cousin of Baum-Welch.

## 9.5 Input Output Hidden Markov Models

An *input output HMM* (IOHMM) generates an output sequence $x^T$ while *conditioned* on an input sequence $y^T$ of the same length. Concretely, every probability now also depends on the current input: the start probability is $p_S(u_1 \mid y_1)$, the transitions are $p(u_t \mid y_t, u_{t-1})$, and the emissions are $p_E(x_t \mid y_t, u_t)$. Learning is unchanged — maximise the likelihood with EM — just with every table conditioned on the input.

The payoff is that IOHMMs can finally use *negative examples*, one of the things a plain HMM cannot do. Feed the class label through the input (e.g. set $y_T = +1$ for the positive class and $y_T = -1$ for the negative class, with the other inputs set to a fixed or don't-care value); the model can then learn to discriminate, or at least to separate the slice of the negative class that sits closest to the positives. The cost is that the number of parameters grows with the size of the input alphabet, so with limited data the probabilities become harder to estimate reliably.

## 9.6 Factorial Hidden Markov Models

A single state variable taking $S$ values is a clumsy way to represent a process that is really several things happening at once — you would need one state per *combination* of factors. A *factorial HMM* instead splits the hidden state into several parallel chains $u^1, u^2, \ldots, u^h$. In the standard factorial HMM, each chain evolves from its own previous state and the observation depends jointly on the current states of all chains. Coupled or hierarchical variants may additionally

let chains influence one another. The chains can represent simultaneous causes or processes operating at different time scales.

The price is combinatorial. With $h$ chains of $S$ values each, the joint emission distribution has $P\,S^h$ entries, so exact learning is expensive. This is why factorial HMMs are trained with approximate (e.g. variational) inference rather than plain Baum-Welch.

## 9.7 Memory Input Output Factorial Hidden Markov Models

Recall Sean Eddy's complaint from Section 9.1 — an HMM's state path "has no way of remembering what a distant state generated". Long-range dependencies are the standard HMM's blind spot, and it is worth seeing exactly why.

The *only* way a plain HMM can carry information forward in time is to enter a state and refuse to leave it: a non-escaping (absorbing) state with

$$p(u_t = a \mid u_{t-1} = a) = 1.$$

Once the chain takes value $a$ it stays there, so the fact "we once saw the event that pushed us into $a$" is preserved. In principle the model could *learn* to do this, but the likelihood of successfully maintaining the memory shrinks exponentially with the storage duration. That vanishing signal is swamped by the ordinary local minima coming from input/output statistics, so learning to store over long lags is effectively impossible. The practical fix is to *hardwire* the memory: fix $p(a \mid a) = 1$ and never update it. But then the unit is frozen — after storing, it can neither track new dynamics nor record a second event.

The *Memory-based Input-Output Factorial HMM* (MIOFHMM, Hochreiter & Mozer, 2001) resolves this by combining the two previous extensions: use a factorial HMM so that *some* chains are dedicated absorbing memory units while others keep modelling the system's dynamics, and wrap it in the input/output architecture so that inputs can *trigger* the memory. All chains start "uncommitted"; an input event flips a memory chain into an absorbing value that thereafter flags "this event occurred". The experiment in the lecture (number of weight updates needed to learn to remember an input until the end of the sequence, as a function of sequence length $T$) shows the ordinary IOHMM blowing up with $T$, the factorial IOFHMM doing better, and the MIOFHMM staying low and nearly flat.

**Intuition & Mental Model**

This is the long-term-memory problem in probabilistic clothing: the signal for *keeping* a memory decays exponentially with the lag — the same exponential decay that plagues gradients in plain recurrent networks and that LSTM was built to defeat. The remedy here is identical in spirit to the LSTM memory cell: stop hoping the model learns to hold on, and build in a unit that simply cannot forget.

## 9.8 Tricks of the Trade

A handful of practical adjustments make HMMs behave on real bioinformatics data:

- **Non-emitting (delete) states.** Bioinformatics models often include states that consume no symbol. The forward pass has to be adjusted to skip over them rather than advancing the sequence position.
- **Comparing variable-length sequences.** Likelihood shrinks exponentially with sequence length (and there are simply more long sequences than short ones), so raw likelihoods are

not comparable across lengths. Normalise, or compare against a length-matched random model.

- **Work in log-space.** Products of many small probabilities underflow; computing everything as sums of logs keeps the numbers sane.
- **Floor the probabilities.** Keep every probability above a small threshold $\epsilon$. Hard zeros make some regions of parameter space permanently unreachable.
- **Zero out afterwards, not during.** EM can never quite reach an exact zero. As post-processing, snap all probabilities $\leq \epsilon$ to zero; this often improves generalisation from short training sequences to much longer test sequences.
- **Beware local minima.** HMMs (especially when built from unaligned sequences) are riddled with them. Global strategies such as deterministic annealing help escape.

## 9.9 Profile Hidden Markov Models

A *profile HMM* encodes a multiple sequence alignment as a position-specific scoring model that you can run against a database to find remote homologues. It is the workhorse behind tools like HMMER and SAM and the Pfam protein-family database. The architecture (HMMER's) uses three kinds of state per alignment column:

- **Match states** $M_x$ — the consensus columns of the alignment, each with its own emission distribution over the alphabet.
- **Insert states** $I_x$ — emit extra residues inserted *between* consensus columns.
- **Delete states** $D_x$ — non-emitting states that let a sequence *skip* a consensus column.

Begin and end states ($B$, $E$) bracket the pattern. A new sequence is scored by its likelihood; HMMER actually reports a log-odds score against a random model, which makes the score's significance easy to read.

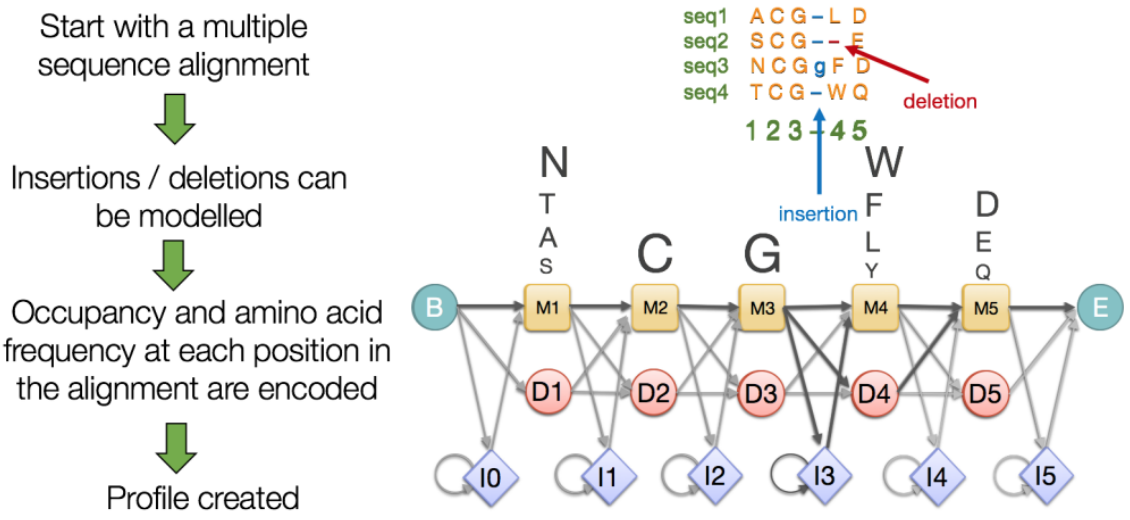


Figure 9.2: Constructing a profile HMM from a multiple sequence alignment. Consensus columns become match states $M_1, \ldots, M_5$; gaps are represented by silent delete states $D_i$, and extra residues between consensus columns by insert states $I_i$. The alignment marks a deletion in red and an insertion in blue.

In Fig. 9.2, the gap in sequence 2 skips a consensus column through a delete state, whereas the lower-case residue in sequence 3 is emitted by an insert state between two match columns. Occupancy determines whether an alignment column becomes a match position, and the residue counts in that column determine its position-specific emission distribution.

Building a profile from *unaligned* positive examples runs straight into the local-minimum problem, so one either pays for global optimisation (deterministic annealing) or — as is now standard — *initialises* the HMM from a multiple alignment of the positives and only fine-tunes from there. Profile HMMs are pervasive: Pfam is organised around profile HMMs for protein families. Coverage percentages change with every database release and should be looked up rather than memorised from an old slide. The same machinery, outside the profile setting, is also used for tasks such as splice-site detection.

# 10

# Boltzmann Machines

## 10.1 The Boltzmann Machine

A Boltzmann machine is a stochastic recurrent neural network, introduced by Geoffrey Hinton and Terry Sejnowski in 1985. Think of it as a probabilistic cousin of the Hopfield network: a collection of binary units that flip on and off at random, biased so that "good" configurations show up more often. The name comes from the Boltzmann distribution in statistical mechanics, which supplies both the probability of a state and the recipe for sampling. In its fully connected form it never became practical, but its restricted variant (the RBM, Section 10.3) later turned out to be very useful for deep learning.

Each unit $i$ has a binary state $s_i \in \{0, 1\}$, and the network assigns an *energy* to a complete configuration of all units:

$$E = -\left(\sum_{i<j} w_{ij}\, s_i s_j + \sum_i \theta_i\, s_i\right),$$

where $w_{ij}$ is the connection strength between units $i$ and $j$ and $\theta_i$ is unit $i$'s bias (its activation threshold). The connections are constrained to be symmetric with no self-loops,

$$w_{ii} = 0 \quad \forall i, \qquad w_{ij} = w_{ji} \quad \forall i, j,$$

so the weight matrix $\boldsymbol{W}$ is symmetric with a zero diagonal (and in general indefinite).

How does a unit decide its state? Define the local field as the energy advantage of turning unit $i$ on,

$$h_i \;\coloneqq\; E(s_i = 0) - E(s_i = 1) = \sum_j w_{ij}\, s_j + \theta_i.$$

Under the Boltzmann distribution the energy of a state is proportional to its negative log probability, so the log-odds of the unit being on are exactly $h_i/T$:

$$\frac{h_i}{T} = \ln p_{i=\text{on}} - \ln p_{i=\text{off}} = \ln \frac{p_{i=\text{on}}}{1 - p_{i=\text{on}}}.$$

Solving for the probability gives the logistic (sigmoid) activation,

$$p_{i=\text{on}} = \frac{1}{1 + \exp(-h_i/T)},$$

where $T$ is the temperature. The logistic function is therefore not an arbitrary modelling choice — it drops straight out of the Boltzmann distribution.

The network runs by repeatedly picking a unit and resampling its state from this probability (Gibbs sampling). Once it reaches *thermal equilibrium* the distribution over global states has

settled and no longer changes. In practice one starts hot — a high $T$ makes flips nearly random, which lets the system jump out of poor local minima — and then gradually cools, biasing the sampler toward low-energy configurations. This is simulated annealing; only idealised infinitely slow schedules carry global-optimum guarantees, so practical runs can still settle in a poor basin.

**Intuition & Mental Model**

Picture an energy landscape over all $2^n$ configurations. Low-energy configurations are the high-probability ones the machine "likes". Temperature controls how freely it explores: hot means it wanders and ignores small energy bumps; cold means it greedily rolls downhill. Annealing is "shake hard, then let it settle" — shake to escape bad valleys, settle to lock in a good one.

## 10.2 Learning in the Boltzmann Machine

The units split into *visible* units $V$, which receive input from the environment (the training data is a set of binary vectors over $V$), and *hidden* units $H$, which act as latent features. Write $P^+(V)$ for the distribution of the training data and $P^-(V)$ for the machine's own marginal distribution over the visibles once it has reached equilibrium (the hidden units summed out). Learning means making the model's $P^-(V)$ look like the data's $P^+(V)$, which we measure with the Kullback-Leibler divergence

$$D_{\mathrm{KL}}\left(P^+ \,\|\, P^-\right) = \sum_v P^+(v) \ln \frac{P^+(v)}{P^-(v)}.$$

This is a function of the weights, so we minimise it by gradient descent.

Just like EM, training alternates two phases:

- **Positive (clamped) phase:** pin the visible units to a training vector, let the hidden units settle, and measure the correlation $p^+_{ij}$ — the probability that units $i$ and $j$ are both on at equilibrium.
- **Negative (free) phase:** let the whole network run freely, with no data clamped, and measure the corresponding correlation $p^-_{ij}$.

At temperature $T = 1$, the gradient of the divergence is the negative difference of these two correlations:

$$\frac{\partial D}{\partial w_{ij}} = -\left(p^+_{ij} - p^-_{ij}\right), \qquad \frac{\partial D}{\partial \theta_i} = -\left(p^+_i - p^-_i\right).$$

Gradient descent therefore uses $\Delta w_{ij} = \eta(p^+_{ij} - p^-_{ij})$ and $\Delta\theta_i = \eta(p^+_i - p^-_i)$, where $\eta$ is the learning rate.

**Intuition & Mental Model**

The rule reads as "fit the correlations". Raise the weight between two units that fire together *when clamped to the data*; lower it between two units that fire together *when the model dreams freely*. When the data-driven and model-driven correlations match, the gradient is zero and learning stops. Note that each update uses only the joint activity of the two units it connects — a *local* learning rule, which is part of why the Boltzmann machine is often called biologically plausible.

The catch is the negative phase. Two problems make the unrestricted Boltzmann machine impractical:

- collecting reliable equilibrium statistics can require exponentially long mixing times in the worst case, especially with strong weights and separated energy modes;
- connections are most plastic when their units have activation probabilities midway between 0 and 1, causing a *variance trap*: noise makes the weights random-walk until activities finally saturate.

## 10.3 The Restricted Boltzmann Machine

The *restricted* Boltzmann machine (RBM) removes the intralayer connections: units form a bipartite graph with a visible layer and a hidden layer, and connections run only *between* the layers, never within one. This single restriction is what makes inference tractable. Writing $a_i$ for visible biases, $b_j$ for hidden biases, and $w_{ij}$ for the visible-to-hidden weight, the energy is

$$E(\boldsymbol{v}, \boldsymbol{h}) = -\sum_i a_i v_i - \sum_j b_j h_j - \sum_{i,j} h_j v_i w_{ij} = -\boldsymbol{a}^\top \boldsymbol{v} - \boldsymbol{b}^\top \boldsymbol{h} - \boldsymbol{h}^\top \boldsymbol{W} \boldsymbol{v},$$

and the joint and marginal distributions are

$$p(\boldsymbol{v}, \boldsymbol{h}) = \frac{1}{Z} e^{-E(\boldsymbol{v}, \boldsymbol{h})}, \qquad p(\boldsymbol{v}) = \frac{1}{Z} \sum_{\boldsymbol{h}} e^{-E(\boldsymbol{v}, \boldsymbol{h})},$$

where the partition function $Z = \sum_{\boldsymbol{v}, \boldsymbol{h}} e^{-E(\boldsymbol{v}, \boldsymbol{h})}$ normalises over *all* configurations — the expensive term that makes exact learning hard.

The bipartite structure buys a crucial simplification: given the visible layer the hidden units are mutually independent, and vice versa,

$$p(\boldsymbol{h} \mid \boldsymbol{v}) = \prod_j p(h_j \mid \boldsymbol{v}), \qquad p(\boldsymbol{v} \mid \boldsymbol{h}) = \prod_i p(v_i \mid \boldsymbol{h}),$$

with each conditional given by a sigmoid,

$$p(h_j = 1 \mid \boldsymbol{v}) = \sigma\Big(b_j + \sum_i w_{ij} v_i\Big), \qquad p(v_i = 1 \mid \boldsymbol{h}) = \sigma\Big(a_i + \sum_j w_{ij} h_j\Big).$$

So a whole layer can be sampled in one parallel shot, conditioned on the other.

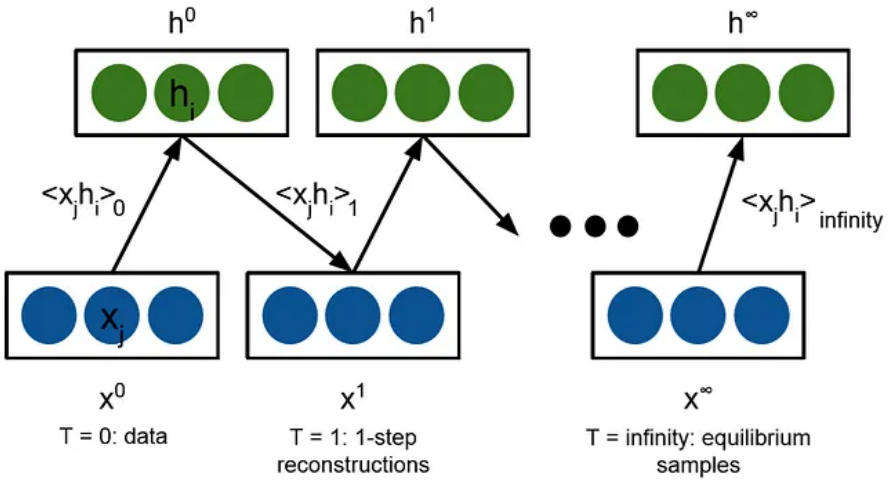


Figure 10.1: RBM sampling statistics. The left pair is clamped to the data at step 0, the middle pair is the one-step reconstruction used by CD-1, and the right pair represents samples after running the chain to equilibrium. The image writes $x$ for the visible vector denoted $\boldsymbol{v}$ in the text.

The two layers in Fig. 10.1 have only cross-layer connections, which is why all hidden units can be sampled together given the visibles and vice versa. Exact maximum-likelihood learning would compare the data statistics at step 0 with the equilibrium statistics at the far right. CD-1 replaces that expensive equilibrium term by the reconstruction statistics after one Gibbs step in the middle. Here the labels $T = 0, 1, \infty$ count sampling steps; they do not denote the thermodynamic temperature used earlier.

### Contrastive divergence

Training maximises the likelihood of the data, $\arg\max_{\boldsymbol{W}} \prod_{v \in V} p(v)$, equivalently the expected log-likelihood. The gradient again needs the model's free-running statistics, which depend on the intractable $Z$. The standard workaround is *contrastive divergence* (CD), which replaces the full free phase with a single short Gibbs step started from the data. One step (CD-1) for a training sample is:

1. Take a training vector $\boldsymbol{v}$ and sample the hidden activations $\boldsymbol{h} \sim p(\boldsymbol{h} \mid \boldsymbol{v})$. The outer product $\boldsymbol{v}\boldsymbol{h}^\top$ is the positive-phase statistic.
2. Sample a reconstruction $\boldsymbol{v}' \sim p(\boldsymbol{v} \mid \boldsymbol{h})$, then resample $\boldsymbol{h}' \sim p(\boldsymbol{h} \mid \boldsymbol{v}')$.
3. The outer product $\boldsymbol{v}'\boldsymbol{h}'^\top$ is the negative-phase statistic.
4. Update $\Delta w_{ij} = \eta\,(\,\text{positive} - \text{negative}\,) = \eta\,(\langle v_i h_j \rangle_{\text{data}} - \langle v'_i h'_j \rangle_{\text{recon}})$.

**Intuition & Mental Model**

Contrastive divergence sidesteps exactly the problem that sank the full Boltzmann machine: instead of running the free phase to equilibrium, it takes a single reconstruction step away from the data and asks "which way did the model drift?" — then nudges the weights to undo that drift. Cheap, and it works remarkably well in practice.

Finally, RBMs *stack*. After training one RBM, its hidden activations are a learned representation of the input, so they can serve as the visible layer of a second, higher-level RBM. Stacking RBMs and training them greedily one layer at a time builds a deep network of features without any labels — a classic unsupervised pretraining strategy, and a large part of why RBMs mattered for the early deep-learning revival.